

Porter Logistics Case Study

About Porter

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's \$40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers

Business Problem

Porter works with a wide range of restaurants for delivering their items directly to the people. Porter has a number of delivery partners available for delivering the food, from various restaurants and wants to get an estimated delivery time that it can provide the customers on the basis of what they are ordering, from where and also the delivery partners. Porter wants to estimate the delivery time for a given order and improve the customer experience by providing accurate delivery time estimates. The company has provided a dataset containing information about past orders, including the time taken for each delivery. The goal is to build a regression model that can predict the delivery time based on various features such as order details, restaurant information, and delivery partner availability. This dataset has the required data to train a regression model that will do the delivery time estimation, based on all those features

Metrics

We will use Mean Squared Error (MSE) as the evaluation metric for our regression model. MSE is a common metric used to measure the average squared difference between the predicted and actual values. A lower MSE indicates better performance of the model.

We will also use MAE (Mean Absolute Error) as an additional metric to evaluate the model's performance. MAE measures the average absolute difference between the predicted and actual values, providing a more interpretable measure of prediction accuracy.

Data Dictionary

Each row in this file corresponds to one unique delivery. Each column corresponds to a feature as explained below.

1. market_id : integer id for the market where the restaurant lies
2. created_at : the timestamp at which the order was placed
3. actual_delivery_time : the timestamp when the order was delivered
4. store_primary_category : category for the restaurant
5. order_protocol : integer code value for order protocol(how the order was placed ie: through porter, call to restaurant, pre booked, third part etc)
6. total_items subtotal : final price of the order
7. num_distinct_items : the number of distinct items in the order
8. min_item_price : price of the cheapest item in the order
9. max_item_price : price of the costliest item in order
10. total_onshift_partners : number of delivery partners on duty at the time order was placed
11. total_busy_partners : number of delivery partners attending to other tasks
12. total_outstanding_orders : total number of orders to be fulfilled at the moment

```
In [1]: import polars as pl
import duckdb as db
import numpy as np
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler

In [2]: df = pl.read_csv("../data/raw/dataset.csv")
```

Data Exploration

```
In [3]: df.head()
```

Out[3]: shape: (5, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshif
f64	str	str	str	str	f64	i64	i64	i64	i64	i64	
1.0	"2015-02-06 22:24:17"	"2015-02-06 23:27:16"	"df263d996281d984952c07998dc543...	"american"	1.0	4	3441	4	557	1239	
2.0	"2015-02-10 21:49:25"	"2015-02-10 22:56:29"	"f0ade77b43923b38237db569b016ba...	"mexican"	2.0	1	1900	1	1400	1400	
3.0	"2015-01-22 20:39:28"	"2015-01-22 21:09:09"	"f0ade77b43923b38237db569b016ba...	null	1.0	1	1900	1	1900	1900	
3.0	"2015-02-03 21:21:45"	"2015-02-03 22:13:00"	"f0ade77b43923b38237db569b016ba...	null	1.0	6	6900	5	600	1800	
3.0	"2015-02-15 02:40:36"	"2015-02-15 03:20:26"	"f0ade77b43923b38237db569b016ba...	null	1.0	3	3900	3	1100	1600	

In [4]: df.shape

Out[4]: (197428, 14)

In [5]: df.describe()

Out[5]: shape: (9, 15)

statistic	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_pri
str	f64	str	str	str	str	f64	f64	f64	f64	f64	f
"count"	196441.0	"197428"	"197421"	"197428"	"192668"	196433.0	197428.0	197428.0	197428.0	197428.0	197428
"null_count"	987.0	"0"	"7"	"0"	"4760"	995.0	0.0	0.0	0.0	0.0	(
"mean"	2.978706	null	null	null	null	2.882352	3.196391	2682.331402	2.670791	686.21847	1159.588
"std"	1.524867	null	null	null	null	1.503771	2.666546	1823.093688	1.630255	522.038648	558.4113
"min"	1.0	"2014-10-19 05:24:15"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...	"afghan"	1.0	1.0	0.0	1.0	-86.0	(
"25%"	2.0	null	null	null	null	1.0	2.0	1400.0	1.0	299.0	800
"50%"	3.0	null	null	null	null	3.0	3.0	2200.0	2.0	595.0	1095
"75%"	4.0	null	null	null	null	4.0	4.0	3395.0	3.0	949.0	1395
"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffedf5be3a86e2ee281d54cdc97bc1...	"vietnamese"	7.0	411.0	27100.0	20.0	14700.0	14700

In [6]: dict(zip(df.columns, df.dtypes))

Out [6]: {'market_id': Float64, 'created_at': String, 'actual_delivery_time': String, 'store_id': String, 'store_primary_category': String, 'order_protocol': Float64, 'total_items': Int64, 'subtotal': Int64, 'num_distinct_items': Int64, 'min_item_price': Int64, 'max_item_price': Int64, 'total_onshift_partners': Float64, 'total_busy_partners': Float64, 'total_outstanding_orders': Float64}

In [7]: df.null_count()

Out [7]: shape: (1, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
987	0	7	0	4760	995	0	0	0	0	0	16262	162	162

In [8]: db.sql("""select * from df where total_onshift_partners IS NULL limit 20""")

Out [8]:

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders
double	varchar	varchar	varchar	varchar	double	int64	int64	int64	double	double	double	double	double
int64	int64	double	double	double	double	double	double	double	double	double	double	double	double
225	6.0	2015-02-06 01:11:56	2015-02-06 01:42:51	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	2	575	2	2	2	2	2
185	6.0	2015-02-14 02:07:47	2015-02-14 03:17:37	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	5	1415	3	3	3	3	3
650	6.0	2015-01-31 21:58:30	2015-01-31 22:55:32	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	1	650	1	1	1	1	1
225	6.0	2015-02-08 03:28:59	2015-02-08 05:32:11	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	5	1550	5	5	5	5	5
185	6.0	2015-01-23 19:29:17	2015-01-23 20:25:25	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	6	1110	5	5	5	5	5
150	6.0	2015-02-03 22:27:24	2015-02-03 23:09:31	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	8	2115	8	8	8	8	8
889	6.0	2015-01-31 01:35:23	2015-01-31 02:16:43	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	2	1741	2	2	2	2	2
225	6.0	2015-01-24 23:17:11	2015-01-25 00:24:49	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	7	2250	2	2	2	2	2
3900	6.0	2015-02-03 21:15:23	2015-02-03 22:38:32	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	1	3900	1	1	1	1	1
400	6.0	2015-02-15 19:57:36	2015-02-15 20:49:04	4a06d868d044c50af0cf9bc82d2fc19f	sandwich	1.0	4	3575	4	4	4	4	4
979	6.0	2015-01-25 19:15:56	2015-01-25 19:40:38	4a06d868d044c50af0cf9bc82d2fc19f	sandwich	1.0	2	2374	2	2	2	2	2
0	6.0	2015-01-28 19:50:12	2015-01-28 20:41:22	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	7	2503	4	4	4	4	4
179	6.0	2015-01-28 21:34:15	2015-01-28 22:01:36	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	5	1997	5	5	5	5	5
0	6.0	2015-02-17 19:38:04	2015-02-17 20:02:27	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	3	1274	3	3	3	3	3
899	6.0	2015-01-29 21:18:12	2015-01-29 21:43:05	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	2	1798	1	1	1	1	1
999	6.0	2015-02-14 21:20:04	2015-02-14 21:54:10	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	2	2497	2	2	2	2	2
299	6.0	2015-02-03 23:33:16	2015-02-04 00:07:15	e143c01e314f7b950daca31188cb5d0f	sandwich	4.0	2	1597	2	2	2	2	2
699	6.0	2015-02-07 03:40:27	2015-02-07 04:06:48	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	1	1048	1	1	1	1	1
300	6.0	2015-02-07 21:28:56	2015-02-07 22:00:43	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	2	1199	2	2	2	2	2
1095	6.0	2015-02-09 05:22:41	2015-02-09 05:51:35	1f0643149dd5c86751215fd0040675aa	korean	1.0	2	2390	2	2	2	2	2
20 rows	14 columns												

In [9]: db.sql("""select * from df where store_primary_category IS NULL limit 20""")

Table 1: Order Details														
market_id		created_at		actual_delivery_time		store_id		store_primary_category		order_protocol	total_items	subtotal	num_distinct_items	min_
item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders										
double	varchar	varchar	double	double	double	double	varchar			double	int64	int64	int64	
int64	int64		double	double	double	double	varchar			double	int64	int64	int64	
1900	3.0	2015-01-22 20:39:28	2015-01-22 21:09:09	f0ade77b43923b38237db569b016ba25	NULL					1.0	1	1900	1	
600	3.0	2015-02-03 21:21:45	2015-02-03 22:13:00	f0ade77b43923b38237db569b016ba25	NULL					1.0	6	6900	5	
1100	3.0	2015-02-15 02:40:36	2015-02-15 03:20:26	f0ade77b43923b38237db569b016ba25	NULL					1.0	3	3900	3	
1500	3.0	2015-01-28 20:30:38	2015-01-28 21:08:58	f0ade77b43923b38237db569b016ba25	NULL					1.0	3	5000	3	
1200	3.0	2015-01-31 02:16:36	2015-01-31 02:43:00	f0ade77b43923b38237db569b016ba25	NULL					1.0	2	3900	2	
750	3.0	2015-02-12 03:03:35	2015-02-12 03:36:20	f0ade77b43923b38237db569b016ba25	NULL					1.0	4	4850	4	
700	3.0	2015-02-18 01:15:45	2015-02-18 02:08:57	f0ade77b43923b38237db569b016ba25	NULL					1.0	2	2100	2	
1200	3.0	2015-02-02 19:22:53	2015-02-02 20:09:19	f0ade77b43923b38237db569b016ba25	NULL					4.0	4	4300	4	
600	3.0	2015-02-16 04:19:33	2015-02-16 06:34:00	f0ade77b43923b38237db569b016ba25	NULL					1.0	2	2200	2	
1900	3.0	2015-02-07 01:34:31	2015-02-07 02:17:14	f0ade77b43923b38237db569b016ba25	NULL					1.0	1	1900	1	
699	3.0	2015-01-25 01:50:51	2015-01-25 02:28:53	f0ade77b43923b38237db569b016ba25	NULL					4.0	4	4986	4	
275	1.0	2015-01-28 20:33:04	2015-01-28 21:04:14	50905d7b2216bfeccb5b41016357176b	NULL					NULL	3	1765	3	
975	1.0	2015-02-12 20:27:42	2015-02-12 21:18:29	50905d7b2216bfeccb5b41016357176b	NULL					2.0	1	975	1	
175	1.0	2015-01-28 21:37:38	2015-01-28 22:11:53	50905d7b2216bfeccb5b41016357176b	NULL					2.0	3	1425	3	
825	1.0	2015-01-21 20:35:27	2015-01-21 21:01:26	50905d7b2216bfeccb5b41016357176b	NULL					2.0	2	1750	2	
825	1.0	2015-02-16 19:19:17	2015-02-16 20:16:43	50905d7b2216bfeccb5b41016357176b	NULL					2.0	8	7870	6	
825	1.0	2015-01-27 20:26:19	2015-01-27 21:03:53	50905d7b2216bfeccb5b41016357176b	NULL					2.0	2	1725	2	
395	1.0	2015-01-22 20:19:08	2015-01-22 21:07:43	50905d7b2216bfeccb5b41016357176b	NULL					2.0	3	2015	3	
825	1.0	2015-02-10 20:05:17	2015-02-10 20:34:43	50905d7b2216bfeccb5b41016357176b	NULL					2.0	1	1220	1	
194	3.0	2015-01-23 19:22:18	2015-01-23 19:48:50	51b7dae1031b20174cacc7e69d6e4bf0	NULL					4.0	3	582	1	

```
db.sql("""
select * from df where order_protocol IS NULL limit 20
""")
```

market_id		created_at		actual_delivery_time		store_id		store_primary_category		order_protocol	total_items	subtotal	num_distinct_items	min
item_price	max_item_price	total	onshift_partners	total_busy_partners	total_outstanding_orders									
double	varchar	double	varchar	double	varchar	double	varchar			double	int64	int64	int64	
int64	int64		double		double		double							
275	1.0	2015-01-28 20:33:04	2015-01-28 21:04:14	50905d7b2216bfeccb5b41016357176b	21.0	24.0	NULL	NULL	3	1765	3			
325	4.0	2015-01-24 19:48:45	2015-01-24 20:31:14	a87ff679a2f3e71d9181a6b7b542122c	60.0	59.0	mediterranean	NULL	2	2070	2			
379	2.0	2015-02-10 19:36:38	2015-02-10 20:24:14	26324d8e2cc1957b8e581568a089a51c	38.0	68.0	NULL	NULL	9	4781	8			
1000	NULL	2015-02-17 02:17:43	2015-02-17 03:15:14	fe8c15fed5f808006ce95eddb7366e35	36.0	52.0	NULL	NULL	3	3400	3			
1295	4.0	2015-02-09 02:10:29	2015-02-09 02:59:20	6ea9ab1baa0efb9e19094440c317e21b	66.0	121.0	NULL	NULL	2	2790	2			
1995	5.0	2015-01-30 20:27:37	2015-01-30 21:17:51	b597976c3ce6012f3a07e9f5c71a3c8c	10.0	12.0	NULL	NULL	1	1995	1			
400	3.0	2015-01-23 02:21:51	2015-01-23 03:29:52	ed71773d43d53fa70ecf593c6582d9cc	7.0	7.0	sushi	NULL	6	5275	4			
1399	NULL	2015-02-17 03:49:46	2015-02-17 04:21:27	86311dbe35f1b6c5166365165602f54d	40.0	69.0	pizza	NULL	1	1699	1			
595	NULL	2015-02-07 02:11:12	2015-02-07 03:10:11	29cb2d8a436d82bf5dbcca7ea09edc5c	40.0	71.0	NULL	NULL	5	5240	5			
920	NULL	2015-01-23 04:09:03	2015-01-23 05:14:00	9f8684e630c4c30cad7b1f0935cd62ab	NULL	NULL	NULL	NULL	2	2240	1			
775	4.0	2015-02-10 23:44:10	2015-02-11 00:34:39	cf2226ddd41b1a2d0ae51dab54d32c36	12.0	21.0	fast	NULL	2	1850	1			
189	4.0	2015-01-29 01:09:59	2015-01-29 02:01:41	cf2226ddd41b1a2d0ae51dab54d32c36	70.0	82.0	NULL	NULL	3	1967	3			
160	NULL	2015-01-29 02:46:37	2015-01-29 03:47:41	fd8c07a31f8a85910ad8476f5f7efb27	0.0	0.0	chinese	NULL	2	890	2			
2575	2.0	2015-02-16 02:05:20	2015-02-16 02:44:08	0bb0846327772451045bd30dd347821b	54.0	89.0	NULL	NULL	1	2575	1			
200	1.0	2015-02-07 03:42:08	2015-02-07 05:00:30	35b47299c1130953d286c976e9c608dc	25.0	38.0	british	NULL	2	1750	2			
500	NULL	2015-01-21 20:42:48	2015-01-21 21:20:59	58e4d44e550d0f7ee0a23d6b02d9b0db	24.0	28.0	american	NULL	3	2150	2			
175	5.0	2015-02-15 18:24:36	2015-02-15 18:50:49	bb838bfff2c2120b9237cac812c1019f8	6.0	6.0	NULL	NULL	6	1670	4			
100	3.0	2015-02-07 23:05:03	2015-02-07 23:35:05	8de0c3085da54b8e957220b9c8de8aca	8.0	13.0	NULL	NULL	6	3538	3			
1099	2.0	2015-01-24 05:14:44	2015-01-24 05:55:35	34be90676412d14c80f85809d8e3b97b	27.0	47.0	NULL	NULL	4	4696	2			
695	1.0	2015-01-29 00:57:22	2015-01-29 01:40:19	f187a23c3ee681ef6913f31fd6d6446b	7.0	6.0	pizza	NULL	1	995	1			
20 rows 14 columns														

```
db.sql("""
select market_id, store_primary_category, count(*) from df group by market_id, store_primary_category order by count(*) desc limit 20
""")
```


Out[11]:

market_id double	store_primary_category varchar	count_star() int64
2.0	mexican	6647
2.0	american	5700
1.0	american	5228
2.0	pizza	4859
1.0	pizza	4081
4.0	pizza	3840
2.0	sandwich	3751
2.0	burger	3340
4.0	mexican	3250
2.0	dessert	3115
4.0	chinese	2847
3.0	american	2743
1.0	japanese	2628
4.0	burger	2603
4.0	dessert	2601
4.0	vietnamese	2541
3.0	mexican	2487
1.0	mexican	2439
4.0	japanese	2403
5.0	american	2386
20 rows		3 columns

In [12]:

df["store_primary_category"].value_counts().sort("count", descending=True)

Out[12]: shape: (75, 2)

store_primary_category	count
str	u32
"american"	19399
"pizza"	17321
"mexican"	17099
"burger"	10958
"sandwich"	10060
...	...
"lebanese"	9
"belgian"	2
"indonesian"	2
"alcohol-plus-food"	1
"chocolate"	1

In [13]:

df["order_protocol"].value_counts().sort("count", descending=True)

Out[13]: shape: (8, 2)

order_protocol	count
f64	u32
1.0	54725
3.0	53199
5.0	44290
2.0	24052
4.0	19354
null	995
6.0	794
7.0	19

In [14]:

df["market_id"].value_counts().sort("count", descending=True)

Out[14]: shape: (7, 2)

market_id	count
f64	u32
2.0	55058
4.0	47599
1.0	38037
3.0	23297
5.0	18000
6.0	14450
null	987

In [15]:

df=df.with_columns(
 pl.col("min_item_price").abs(),
 pl.col("max_item_price").abs(),
 pl.col("total_onshift_partners").abs(),
 pl.col("total_busy_partners").abs(),
 pl.col("total_outstanding_orders").abs(),
)

In [16]:

df.describe()

Out[16]: shape: (9, 15)

statistic	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_pri
str	f64	str	str	str	str	f64	f64	f64	f64	f64	f
"count"	196441.0	"197428"	"197421"	"197428"	"192668"	196433.0	197428.0	197428.0	197428.0	197428.0	197428
"null_count"	987.0	"0"	"7"	"0"	"4760"	995.0	0.0	0.0	0.0	0.0	(
"mean"	2.978706	null	null	null	null	2.882352	3.196391	2682.331402	2.670791	686.222339	1159.588
"std"	1.524867	null	null	null	null	1.503771	2.666546	1823.093688	1.630255	522.033561	558.4113
"min"	1.0	"2014-10-19 05:24:15"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...	"afghan"	1.0	1.0	0.0	1.0	0.0	(
"25%"	2.0	null	null	null	null	1.0	2.0	1400.0	1.0	299.0	800
"50%"	3.0	null	null	null	null	3.0	3.0	2200.0	2.0	595.0	1095
"75%"	4.0	null	null	null	null	4.0	4.0	3395.0	3.0	949.0	1395
"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffedf5be3a86e2ee281d54cdc97bc1...	"vietnamese"	7.0	411.0	27100.0	20.0	14700.0	14700

In [17]: df.null_count()

Out[17]: shape: (1, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
987	0	7	0	4760	995	0	0	0	0	0	16262	162

Null Values Imputation

In [18]: df = df.drop_nulls(subset=["total_onshift_partners", "total_busy_partners", "total_outstanding_orders", "actual_delivery_time",\
"market_id","store_primary_category","order_protocol"\
])

In [19]: df.describe()

Out[19]: shape: (9, 15)

statistic	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price
str	f64	str	str	str	str	f64	f64	f64	f64	f64	f64
"count"	180240.0	"181159"	"181159"	"181159"	"176944"	180242.0	181159.0	181159.0	181159.0	181159.0	181159.0
"null_count"	919.0	"0"	"0"	"0"	"4215"	917.0	0.0	0.0	0.0	0.0	0.0
"mean"	2.748702	null	null	null	null	2.895923	3.208469	2698.524296	2.677659	684.854095	1160.7526
"std"	1.331715	null	null	null	null	1.514983	2.673052	1829.304441	1.627291	521.286542	561.8386
"min"	1.0	"2015-01-21 15:22:03"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...	"afghan"	1.0	1.0	0.0	1.0	0.0	0.0
"25%"	2.0	null	null	null	null	1.0	2.0	1415.0	2.0	299.0	79
"50%"	2.0	null	null	null	null	3.0	3.0	2225.0	2.0	595.0	109
"75%"	4.0	null	null	null	null	4.0	4.0	3411.0	3.0	942.0	139
"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffeabd223de0d4eacb9a3e6e53e544...	"vietnamese"	7.0	411.0	26800.0	20.0	14700.0	1470

In [20]: db.sql("""
select * from df where max_item_price = 0 and min_item_price = 0
""")

Out[20]:

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price
double	varchar	varchar	varchar	varchar	double	int64	int64	int64	double
int64	int64	double	double	double	double	int64	int64	int64	double
1.0	2015-02-09 18:23:31	2015-02-09 19:31:54	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	575	1	0.0
1.0	2015-01-22 17:57:08	2015-01-22 18:54:37	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	575	1	0.0
1.0	2015-02-13 18:41:31	2015-02-13 19:30:39	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	575	1	0.0
1.0	2015-01-21 19:36:57	2015-01-21 20:23:45	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	560	1	0.0
1.0	2015-02-13 02:37:10	2015-02-13 03:13:44	3e91970f771a2c473ae36b60d1146068	dim-sum	5.0	1	595	1	0.0
5.0	2015-01-27 02:26:45	2015-01-27 02:56:56	648f4baa45889f9c5f4f7add35862841	pizza	3.0	1	2418	1	0.0
1.0	2015-01-21 19:50:42	2015-01-21 20:24:07	da5e8bfed9bdb84595be92afeb3fd378	sandwich	1.0	2	1090	2	0.0
0.0	0.0	21.0	21.0	31.0	0.0	0.0	0.0	0.0	0.0

In [21]: df=df.with_columns(
pl.when(pl.col("max_item_price") == 0).then(pl.col("max_item_price").mean()).otherwise(pl.col("max_item_price")).alias("max_item_price"),
pl.when(pl.col("min_item_price") == 0).then(pl.col("min_item_price").mean()).otherwise(pl.col("min_item_price")).alias("min_item_price")
)

In [22]: db.sql("""
select * from df where max_item_price < min_item_price
""")

Out [22]:

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min
item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders					
double	varchar	varchar	varchar	varchar	double	int64	int64	int64	
double	double	double	double	double					
1314.0	4.0	2015-01-24 03:03:18	2015-01-24 04:02:30	a87ff679a2f3e71d9181a67b7542122c	mexican	1.0	1	1403	1
	1292.0		129.0	127.0	205.0				
	1.0	2015-02-03 03:36:16	2015-02-03 05:13:04	c56a4706337730e0e15da875405fa1c5	fast	4.0	7	1223	5 684.
8540950215005	649.0		22.0	22.0	48.0				
	1.0	2015-01-25 21:38:45	2015-01-25 22:14:22	c56a4706337730e0e15da875405fa1c5	fast	4.0	6	958	6 684.
8540950215005	599.0		17.0	9.0	9.0				
	5.0	2015-01-31 20:10:47	2015-01-31 20:49:29	b597976c3ce6012f3a07e9f5c71a3c8c	NULL	1.0	1	1289	1
1437.0	1400.0		21.0	13.0	19.0				
	5.0	2015-01-30 02:20:58	2015-01-30 03:14:42	b597976c3ce6012f3a07e9f5c71a3c8c	NULL	1.0	1	1828	1
2031.0	1935.0		24.0	21.0	23.0				
	2.0	2015-01-27 04:09:54	2015-01-27 04:56:33	f806c5d2707545d718717be03e69a8d4	fast	4.0	14	1305	3 684.
8540950215005	190.0		65.0	93.0	77.0				
	2.0	2015-02-03 03:22:21	2015-02-03 04:14:00	f806c5d2707545d718717be03e69a8d4	fast	4.0	13	985	4 684.
8540950215005	390.0		96.0	89.0	154.0				
	5.0	2015-02-08 03:42:21	2015-02-08 04:20:11	f806c5d2707545d718717be03e69a8d4	nepalese	1.0	31	2820	7 684.
8540950215005	355.0		116.0	120.0	184.0				
	2.0	2015-01-31 19:28:49	2015-01-31 20:17:29	f806c5d2707545d718717be03e69a8d4	fast	4.0	40	3000	2 684.
8540950215005	150.0		48.0	45.0	51.0				
	2.0	2015-02-07 05:47:42	2015-02-07 06:26:49	f806c5d2707545d718717be03e69a8d4	fast	4.0	26	2320	5 684.
8540950215005	250.0		39.0	43.0	67.0				
.	.		.	.	.	.	.	.	.
.	.		.	.	.	.	.	.	.
.	.		.	.	.	.	.	.	.
.	.		.	.	.	.	.	.	.
.	.		.	.	.	.	.	.	.
	3.0	2015-02-06 20:06:59	2015-02-06 20:40:50	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	1	1299	1
805.0	765.0		6.0	6.0	9.0				
	4.0	2015-02-03 20:36:37	2015-02-03 21:04:49	4cc39b42f22ee5987819b83fc055cb33	pizza	3.0	1	625	1
553.0	540.0		31.0	35.0	31.0				
	4.0	2015-02-07 20:52:15	2015-02-07 21:28:58	4cc39b42f22ee5987819b83fc055cb33	pizza	3.0	1	3174	1
1513.0	1403.0		38.0	31.0	32.0				
	4.0	2015-02-16 02:34:06	2015-02-16 03:20:42	17e62166fc8586dfa4d1bc0e1742c08b	vietnamese	5.0	2	2086	1
904.0	878.0		85.0	77.0	152.0				
	4.0	2015-02-03 02:57:44	2015-02-03 03:41:32	17e62166fc8586dfa4d1bc0e1742c08b	vietnamese	5.0	2	2082	2
1020.0	928.0		88.0	87.0	150.0				
	1.0	2015-02-15 22:15:36	2015-02-15 22:42:14	1763ea5a7e72dd7ee64073c2dda7a7a8	fast	4.0	9	946	5 684.
8540950215005	299.0		24.0	17.0	17.0				
	1.0	2015-02-09 05:16:26	2015-02-09 05:48:49	1763ea5a7e72dd7ee64073c2dda7a7a8	fast	4.0	6	348	3 684.
8540950215005	169.0		20.0	20.0	18.0				
	1.0	2015-01-25 03:06:25	2015-01-25 03:32:31	1763ea5a7e72dd7ee64073c2dda7a7a8	fast	4.0	11	1351	5 684.
8540950215005	169.0		75.0	66.0	83.0				
	1.0	2015-01-30 19:24:55	2015-01-30 20:03:21	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	1	1142	1
737.0	701.0		22.0	19.0	32.0				
	1.0	2015-02-09 20:32:27	2015-02-09 21:09:10	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	1	437	1
497.0	448.0		36.0	26.0	27.0				
1778 rows (20 shown)									
14 columns									

```
In [23]: df=df.with_columns(  
    max_price_temp=pl.col('max_item_price'),  
    min_price_temp=pl.col('min_item_price'),  
)  
  
In [24]: df=df.with_columns(  
    max_item_price=pl.when(pl.col('max_price_temp') < pl.col('min_price_temp'))  
        .then(pl.col('min_price_temp'))  
        .otherwise(pl.col('max_price_temp')),  
    min_item_price=pl.when(pl.col('min_price_temp') > pl.col('max_price_temp'))  
        .then(pl.col('max_price_temp'))  
        .otherwise(pl.col('min_price_temp'))  
).drop('max_price_temp', 'min_price_temp')  
  
In [25]: db.sql("""  
    select * from df where  total_onshift_partners = 0  
    """)
```

Out[25]:

market_id	item_price	created_at		actual_delivery_time		store_id		store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_
		max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders								
		double	varchar	varchar	varchar								
		double	double	double	double								
1.0	2015-01-27 02:29:00	2015-01-27 03:30:45	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1150	1					
1150.0	1.0	2015-02-08 00:56:26	2015-02-08 01:50:48	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	4	4805	2				
1095.0	1.0	2015-01-24 04:01:10	2015-01-24 04:28:57	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5200	5				
225.0	1.0	2015-02-01 06:20:26	2015-02-01 06:55:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2910	3				
595.0	1.0	2015-02-14 03:10:56	2015-02-14 04:12:41	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2495	2				
1095.0	1.0	2015-02-04 19:28:18	2015-02-04 20:11:13	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2345	2				
1150.0	1.0	2015-01-25 03:22:11	2015-01-25 04:14:22	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1250	1				
1150.0	1.0	2015-02-08 23:06:13	2015-02-08 23:42:58	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	1190	2				
595.0	1.0	2015-01-23 02:45:55	2015-01-23 03:32:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5035	5				
195.0	1.0	2015-01-23 05:03:26	2015-01-23 05:56:30	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2065	3				
195.0	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
3.0	2015-01-28 16:18:30	2015-01-28 17:33:30	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1640	2					
615.0	3.0	2015-02-17 19:03:53	2015-02-17 19:51:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2500	3				
500.0	3.0	2015-02-14 17:31:55	2015-02-14 18:24:49	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2700	3				
650.0	3.0	2015-02-10 19:03:48	2015-02-10 20:26:52	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3				
245.0	3.0	2015-02-12 21:07:28	2015-02-12 21:55:41	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3				
245.0	3.0	2015-01-23 17:53:47	2015-01-23 18:42:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	1	975	1				
975.0	3.0	2015-02-07 15:31:33	2015-02-07 16:24:02	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	5	2430	4				
270.0	3.0	2015-02-08 16:11:00	2015-02-08 17:36:23	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	4	2960	4				
480.0	3.0	2015-01-21 17:13:16	2015-01-21 17:51:09	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1175	2				
345.0	3.0	2015-01-24 05:56:06	2015-01-24 06:27:21	10e4d7889812f78893b86aeb04111871	mexican	3.0	1	2599	1				
2599.0													
3615 rows (20 shown)													
14 columns													

In [26]:

```
db.sql("""
    select * from df where total_busy_partners = 0 and total_onshift_partners = 0 and total_outstanding_orders = 0
""")
```

Out[26]:

market_id	item_price	created_at		actual_delivery_time		store_id		store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_
		max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders								
		double	varchar	varchar	varchar								
		double	double	double	double								
1150.0	1.0	2015-01-27 1150.0	02:29:00	2015-01-27 03:30:45	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1150	1			
1095.0	1.0	2015-02-08 1195.0	00:56:26	2015-02-08 01:50:48	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	4	4805	2			
225.0	1.0	2015-01-24 1095.0	04:01:10	2015-01-24 04:28:57	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5200	5			
595.0	1.0	2015-02-01 1095.0	06:20:26	2015-02-01 06:55:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2910	3			
1095.0	1.0	2015-02-14 1150.0	03:10:56	2015-02-14 04:12:41	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2495	2			
1150.0	1.0	2015-02-04 1195.0	19:28:18	2015-02-04 20:11:13	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2345	2			
1150.0	1.0	2015-01-25 1150.0	03:22:11	2015-01-25 04:14:22	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1250	1			
595.0	1.0	2015-02-08 595.0	23:06:13	2015-02-08 23:42:58	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	1190	2			
195.0	1.0	2015-01-23 1150.0	02:45:55	2015-01-23 03:32:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5035	5			
195.0	1.0	2015-01-23 1150.0	05:03:26	2015-01-23 05:56:30	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2065	3			
.	.	.	.	.	.	.	.	.	.	.			
.	.	.	.	.	.	.	.	.	.	.			
.	.	.	.	.	.	.	.	.	.	.			
.	.	.	.	.	.	.	.	.	.	.			
480.0	3.0	2015-01-24 950.0	17:11:50	2015-01-24 18:29:10	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2455	3			
615.0	3.0	2015-01-28 925.0	16:18:30	2015-01-28 17:33:30	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1640	2			
500.0	3.0	2015-02-17 975.0	19:03:53	2015-02-17 19:51:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2500	3			
650.0	3.0	2015-02-14 975.0	17:31:55	2015-02-14 18:24:49	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2700	3			
245.0	3.0	2015-02-10 810.0	19:03:48	2015-02-10 20:26:52	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3			
245.0	3.0	2015-02-12 810.0	21:07:28	2015-02-12 21:55:41	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3			
975.0	3.0	2015-01-23 975.0	17:53:47	2015-01-23 18:42:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	1	975	1			
270.0	3.0	2015-02-07 790.0	15:31:33	2015-02-07 16:24:02	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	5	2430	4			
480.0	3.0	2015-02-08 925.0	16:11:00	2015-02-08 17:36:23	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	4	2960	4			
345.0	3.0	2015-01-21 790.0	17:13:16	2015-01-21 17:51:09	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1175	2			
3415 rows (20 shown)													
14 columns													

In [27]:

```
df=df.filter(
    ~(pl.col("total_onshift_partners") == 0)
)
```

In [28]:

```
df.describe()
```


Out [28]: shape: (9, 15)

	statistic	market_id	created_at	actual_delivery_time		store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_pri
	str	f64	str	str		str	str	f64	f64	f64	f64	f64	ff
	"count"	176642.0	"177544"	"177544"		"177544"	"173383"	176640.0	177544.0	177544.0	177544.0	177544.0	177544
	"null_count"	902.0	"0"	"0"		"0"	"4161"	904.0	0.0	0.0	0.0	0.0	C
	"mean"	2.767864	null	null		null	null	2.898687	3.205588	2701.713919	2.675618	692.093454	1164.9864
	"std"	1.329898	null	null		null	null	1.513229	2.67464	1829.211488	1.62441	516.496735	559.2193
	"min"	1.0	"2015-01-21 15:22:03"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...		"afghan"	1.0	1.0	0.0	1.0	1.0	52
	"25%"	2.0	null	null		null	null	1.0	2.0	1420.0	2.0	300.0	799
	"50%"	2.0	null	null		null	null	3.0	3.0	2228.0	2.0	599.0	1095
	"75%"	4.0	null	null		null	null	4.0	4.0	3421.0	3.0	943.0	1397
	"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffeabd223de0d4eacb9a3e6e53e544...		"vietnamese"	7.0	411.0	26800.0	20.0	14700.0	14700

```
In [29]: category_per_store = df.groupby("store_id").agg(
        pl.col("store_primary_category").mode().first().alias("store_primary_category_name")
    )
category_per_store = category_per_store.fill_null("other")
```

```
In [30]: df = df.join(category_per_store, on="store_id", how="left")
```

```
In [31]: df = df.with_columns(
        pl.when(pl.col("store_primary_category").is_null()).then(pl.col("store_primary_category_name")).otherwise(pl.col("store_primary_category")).alias("store_primary_category")
    ).drop("store_primary_category_name")
```

```
In [32]: market_id_per_store = df.groupby("store_id").agg(
        pl.col("market_id").mode().first().alias("market_id_num")
    )
market_id_per_store = market_id_per_store.fill_null(pl.col("market_id_num").mode().first())
```

```
In [33]: df = df.join(market_id_per_store, on="store_id", how="left")
```

```
In [34]: df = df.with_columns(
        pl.when(pl.col("market_id").is_null()).then(pl.col("market_id_num")).otherwise(pl.col("market_id")).alias("market_id")
    ).drop("market_id_num")
```

```
In [35]: df.null_count()
```

Out [35]: shape: (1, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
0	0	0	0	0	0	904	0	0	0	0	0	0

```
In [36]: df = df.with_columns(
        pl.col("created_at").str.to_datetime(),
        pl.col("actual_delivery_time").str.to_datetime()
    )
```

Time based feature generation

```
In [40]: df=df.with_columns(
        pl.col("created_at").dt.offset_by('-5h30m'),
        pl.col("actual_delivery_time").dt.offset_by('-5h30m')
    )
```

```
In [41]: df = df.with_columns(
        pl.col("created_at").dt.year().alias("created_at_year"),
        pl.col("created_at").dt.month().alias("created_at_month"),
        pl.col("created_at").dt.day().alias("created_at_day"),
        pl.col("created_at").dt.hour().alias("created_at_hour"),
        pl.col("created_at").dt.minute().alias("created_at_minute"),
        pl.col("created_at").dt.second().alias("created_at_second"),
        pl.col("actual_delivery_time").dt.year().alias("actual_delivery_time_year"),
        pl.col("actual_delivery_time").dt.month().alias("actual_delivery_time_month"),
        pl.col("actual_delivery_time").dt.day().alias("actual_delivery_time_day"),
        pl.col("actual_delivery_time").dt.hour().alias("actual_delivery_time_hour"),
        pl.col("actual_delivery_time").dt.minute().alias("actual_delivery_time_minute"),
        pl.col("actual_delivery_time").dt.second().alias("actual_delivery_time_second")
    )
```

```
In [42]: df
```

Out [42]: shape: (177,544, 26)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshi
f64	datetime[μs]	datetime[μs]	str	str	f64	i64	i64	i64	f64	f64	
1.0	2015-02-06 16:54:17	2015-02-06 17:57:16	"df263d996281d984952c07998dc543...	"american"	1.0	4	3441	4	557.0	1239.0	
2.0	2015-02-10 16:19:25	2015-02-10 17:26:29	"f0ade77b43923b38237db569b016ba...	"mexican"	2.0	1	1900	1	1400.0	1400.0	
3.0	2015-01-22 15:09:28	2015-01-22 15:39:09	"f0ade77b43923b38237db569b016ba...	"other"	1.0	1	1900	1	1900.0	1900.0	
3.0	2015-02-03 15:51:45	2015-02-03 16:43:00	"f0ade77b43923b38237db569b016ba...	"other"	1.0	6	6900	5	600.0	1800.0	
3.0	2015-02-14 21:10:36	2015-02-14 21:50:26	"f0ade77b43923b38237db569b016ba...	"other"	1.0	3	3900	3	1100.0	1600.0	
...	...	...	...	...	...	...	...	...	...	...	...
1.0	2015-02-16 18:49:41	2015-02-16 19:54:48	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4.0	3	1389	3	345.0	649.0	
1.0	2015-02-12 18:31:59	2015-02-12 19:28:22	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4.0	6	3010	4	405.0	825.0	
1.0	2015-01-23 23:16:08	2015-01-24 00:06:16	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4.0	5	1836	3	300.0	399.0	
1.0	2015-02-01 12:48:15	2015-02-01 13:53:22	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1.0	1	1175	1	535.0	535.0	
1.0	2015-02-08 13:54:33	2015-02-08 14:31:41	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1.0	4	2605	4	425.0	750.0	

```
In [43]: cols=[
        'order_protocol',
        'total_items',
        'subtotal',
        'num_distinct_items',
        'min_item_price',
        'max_item_price',
        'total_onshift_partners',
    ]
```



```
'total_busy_partners',
'total_outstanding_orders',
'created_at_year',
'created_at_month',
'created_at_day',
'created_at_hour',
'created_at_minute',
'actual_delivery_time_year',
'actual_delivery_time_month',
'actual_delivery_time_day',
'actual_delivery_time_hour',
'actual_delivery_time_minute'
]
```

```
In [44]: # scaler = StandardScaler()
# scaled_features = scaler.fit_transform(df[cols].to_numpy())
knn_imputer = KNNImputer(n_neighbors=3)
imputed_features = knn_imputer.fit_transform(df[cols].to_numpy())
```

```
In [45]: # imputed_features = scaler.inverse_transform(imputed_features)
imputed_features = pl.DataFrame(imputed_features)
imputed_features.columns = cols
imputed_features
```

Out[45]: shape: (177_544, 19)

order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_year	created_at_month	created_at_d
f64	f64	f64	f64	f64	f64	f64	f64	f64	f64	f64	f
1.0	4.0	3441.0	4.0	557.0	1239.0	33.0	14.0	21.0	2015.0	2.0	f
2.0	1.0	1900.0	1.0	1400.0	1400.0	1.0	2.0	2.0	2015.0	2.0	10
1.0	1.0	1900.0	1.0	1900.0	1900.0	1.0	0.0	0.0	2015.0	1.0	20
1.0	6.0	6900.0	5.0	600.0	1800.0	1.0	1.0	2.0	2015.0	2.0	30
1.0	3.0	3900.0	3.0	1100.0	1600.0	6.0	6.0	9.0	2015.0	2.0	10
...	...	...	...	...	...	...	...	...	...	...	f
4.0	3.0	1389.0	3.0	345.0	649.0	17.0	17.0	23.0	2015.0	2.0	10
4.0	6.0	3010.0	4.0	405.0	825.0	12.0	11.0	14.0	2015.0	2.0	10
4.0	5.0	1836.0	3.0	300.0	399.0	39.0	41.0	40.0	2015.0	1.0	20
1.0	1.0	1175.0	1.0	535.0	535.0	7.0	7.0	12.0	2015.0	2.0	30
1.0	4.0	2605.0	4.0	425.0	750.0	20.0	20.0	23.0	2015.0	2.0	40

```
In [46]: df=df.with_columns(
    pl.when(pl.col("order_protocol").is_null())
      .then(imputed_features["order_protocol"])
      .otherwise(pl.col("order_protocol"))
      .cast(pl.Int8)
      .alias("order_protocol")
)
```

```
In [47]: df.write_parquet("../data/cleaned/dataset.parquet")
```

Feature Generation

```
In [54]: df=pl.read_parquet("../data/cleaned/dataset.parquet")
```

```
In [55]: df[["created_at"].max(), df[["created_at"].min()]
```

```
Out[55]: (datetime.datetime(2015, 2, 18, 0, 30, 44),
datetime.datetime(2015, 1, 21, 9, 52, 3))
```

```
In [56]: df=df.with_columns(
    pl.col("created_at").dt.strftime("%b").alias("created_at_month_name"),
    pl.col("actual_delivery_time").dt.strftime("%b").alias("actual_delivery_time_month_name"),
    pl.col("created_at").dt.weekday().alias("created_at_day_of_week"),
    pl.col("actual_delivery_time").dt.weekday().alias("actual_delivery_time_day_of_week"),
    pl.col("created_at").dt.strftime("%a").alias("created_at_day_name"),
    pl.col("actual_delivery_time").dt.strftime("%a").alias("actual_delivery_time_day_name"),
    pl.col("created_at").dt.week().alias("created_at_week_number"),
    pl.col("actual_delivery_time").dt.week().alias("actual_delivery_time_week_number"),
    ((pl.col("created_at").dt.day() - 1) // 7 + 1).cast(pl.Int16).alias("created_at_week_of_month"),
    ((pl.col("actual_delivery_time").dt.day() - 1) // 7 + 1).cast(pl.Int16).alias("actual_delivery_time_week_of_month")
)
```

```
In [57]: df=df.with_columns(
    ((pl.col("actual_delivery_time") - pl.col("created_at")).dt.total_seconds())
      .alias("delivery_time_seconds"),
    ((pl.col("actual_delivery_time") - pl.col("created_at")).dt.total_minutes())
      .alias("delivery_time_minutes"),
)
```

```
In [58]: df.with_columns(
    pl.col("market_id").cast(pl.Int8).alias("market_id"),
    pl.col("total_items").cast(pl.Int16).alias("total_items"),
    pl.col("subtotal").cast(pl.Int16).alias("subtotal"),
    pl.col("num_distinct_items").cast(pl.Int16).alias("num_distinct_items"),
    pl.col("min_item_price").cast(pl.Int16).alias("min_item_price"),
    pl.col("max_item_price").cast(pl.Int16).alias("max_item_price"),
)
```

Out [58]: shape: (177_544, 38)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshi
i8	datetime[μs]	datetime[μs]	str	str	i8	i16	i16	i16	i16	i16	
1	2015-02-06 16:54:17	2015-02-06 17:57:16	"df263d996281d984952c07998dc543...	"american"	1	4	3441	4	557	1239	
2	2015-02-10 16:19:25	2015-02-10 17:26:29	"f0ade77b43923b38237db569b016ba...	"mexican"	2	1	1900	1	1400	1400	
3	2015-01-22 15:09:28	2015-01-22 15:39:09	"f0ade77b43923b38237db569b016ba...	"other"	1	1	1900	1	1900	1900	
3	2015-02-03 15:51:45	2015-02-03 16:43:00	"f0ade77b43923b38237db569b016ba...	"other"	1	6	6900	5	600	1800	
3	2015-02-14 21:10:36	2015-02-14 21:50:26	"f0ade77b43923b38237db569b016ba...	"other"	1	3	3900	3	1100	1600	
...	...	...	...	...	...	...	...	...	...	...	
1	2015-02-16 18:49:41	2015-02-16 19:54:48	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4	3	1389	3	345	649	
1	2015-02-12 18:31:59	2015-02-12 19:28:22	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4	6	3010	4	405	825	
1	2015-01-23 23:16:08	2015-01-24 00:06:16	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4	5	1836	3	300	399	
1	2015-02-01 12:48:15	2015-02-01 13:53:22	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1	1	1175	1	535	535	
1	2015-02-08 13:54:33	2015-02-08 14:31:41	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1	4	2605	4	425	750	

In [59]: df.write_parquet("../data/cleaned/dataset.parquet")

Exploratory Data Analysis

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
import duckdb as db
```

```
In [2]: df=pd.read_parquet("../data/cleaned/dataset.parquet")
df['index'] = df.index
```

```
In [3]: df.dtypes
```

```
Out[3]: market_id                float64
created_at                datetime64[us]
actual_delivery_time      datetime64[us]
store_id                  object
store_primary_category    object
order_protocol            int8
total_items               int64
subtotal                  int64
num_distinct_items        int64
min_item_price            float64
max_item_price            float64
total_onshift_partners    float64
total_busy_partners       float64
total_outstanding_orders  float64
created_at_year           int32
created_at_month          int8
created_at_day            int8
created_at_hour           int8
created_at_minute         int8
created_at_second         int8
actual_delivery_time_year  int32
actual_delivery_time_month int8
actual_delivery_time_day   int8
actual_delivery_time_hour  int8
actual_delivery_time_minute int8
actual_delivery_time_second int8
created_at_month_name      object
actual_delivery_time_month_name object
created_at_day_of_week     int8
actual_delivery_time_day_of_week int8
created_at_day_name        object
actual_delivery_time_day_name object
created_at_week_number     int8
actual_delivery_time_week_number int8
created_at_week_of_month   int16
actual_delivery_time_week_of_month int16
delivery_time_seconds      int64
delivery_time_minutes      int64
index                     int64
dtype: object
```

```
In [4]: df.head()
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	...	actual_delivery_time_c
0	1.0	2015-02-06 16:54:17	2015-02-06 17:57:16	df263d996281d984952c07998dc54358	american	1	4	3441	4	557.0	...	
1	2.0	2015-02-10 16:19:25	2015-02-10 17:26:29	f0ade77b43923b38237db569b016ba25	mexican	2	1	1900	1	1400.0	...	
2	3.0	2015-01-22 15:09:28	2015-01-22 15:39:09	f0ade77b43923b38237db569b016ba25	other	1	1	1900	1	1900.0	...	
3	3.0	2015-02-03 15:51:45	2015-02-03 16:43:00	f0ade77b43923b38237db569b016ba25	other	1	6	6900	5	600.0	...	
4	3.0	2015-02-14 21:10:36	2015-02-14 21:50:26	f0ade77b43923b38237db569b016ba25	other	1	3	3900	3	1100.0	...	

5 rows × 39 columns

```
In [5]: df.describe()
```

	market_id	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	...	actual_delivery_time_
count	177544.000000	177544	177544	177544.000000	177544.000000	177544.000000	177544.000000	177544.000000	177544.000000	177544.000000	...	177544.0
mean	2.767601	2015-02-04 14:02:31.056284	2015-02-04 14:50:05.157701	2.897192	3.205588	2701.713919	2.675618	692.093454	1164.986471	45.719624	...	29.0
min	1.000000	2015-01-21 09:52:03	2015-01-21 10:28:11	1.000000	1.000000	0.000000	1.000000	1.000000	52.000000	1.000000	...	0.0
25%	2.000000	2015-01-28 20:10:10.750000	2015-01-28 20:56:42.750000	1.000000	2.000000	1420.000000	2.000000	300.000000	799.000000	18.000000	...	14.0
50%	2.000000	2015-02-04 20:58:15.500000	2015-02-04 21:58:34.500000	3.000000	3.000000	2228.000000	2.000000	599.000000	1095.000000	38.000000	...	29.0
75%	4.000000	2015-02-11 19:19:06.250000	2015-02-11 19:59:20.250000	4.000000	4.000000	3421.000000	3.000000	943.000000	1397.000000	66.000000	...	44.0
max	6.000000	2015-02-18 00:30:44	2015-02-19 17:15:31	7.000000	411.000000	26800.000000	20.000000	14700.000000	14700.000000	171.000000	...	59.0
std	1.329823	NaN	NaN	1.511135	2.674640	1829.211488	1.624410	516.496735	559.219301	34.271843	...	17.0

8 rows × 33 columns

Univariate Analysis

```
In [78]: print(f"There are {df.shape[0]} rows in the dataset")
print(f"First orders were placed on {df['created_at'].min()} and last orders were placed on {df['created_at'].max()}")
# print number of days
print(f"There are {df['created_at'].dt.date.nunique()} unique days in the dataset")
```

There are 177544 rows in the dataset
First orders were placed on 2015-01-21 09:52:03 and last orders were placed on 2015-02-18 00:30:44
There are 29 unique days in the dataset

Market ID

```
In [26]: plt.figure(figsize=(10, 6))
df["market_id"].value_counts().plot(kind="bar", title="Number of orders per market");
plt.xlabel("Market ID")
plt.ylabel("Number of orders")
plt.show();
```

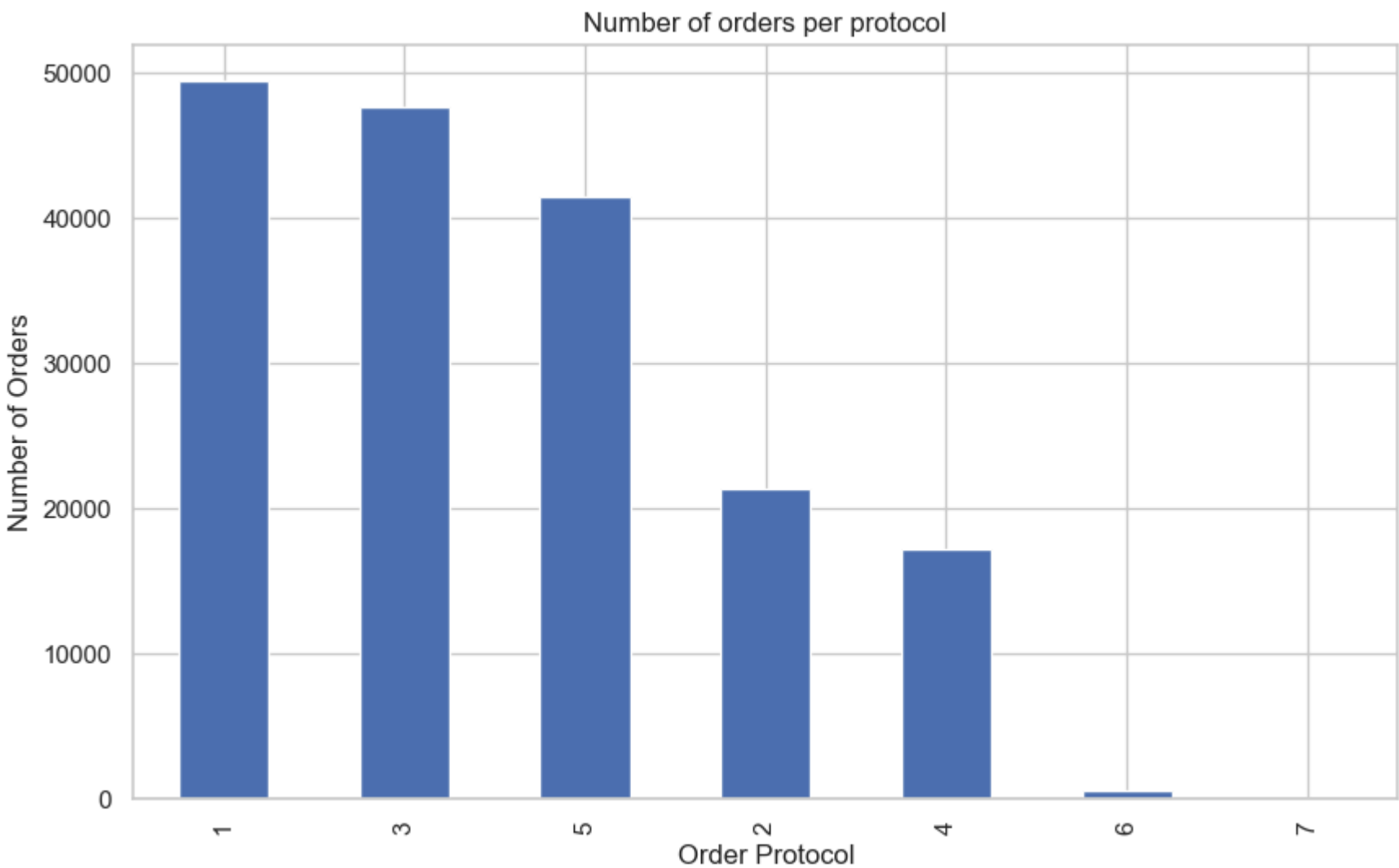



Observations

- Majority of the orders are from market id 2
- Market id 6 has the least number of orders

Order Protocol

```
In [27]: plt.figure(figsize=(10, 6))
df["order_protocol"].value_counts().plot(kind="bar", title="Number of orders per protocol");
plt.xlabel("Order Protocol")
plt.ylabel("Number of Orders");
```



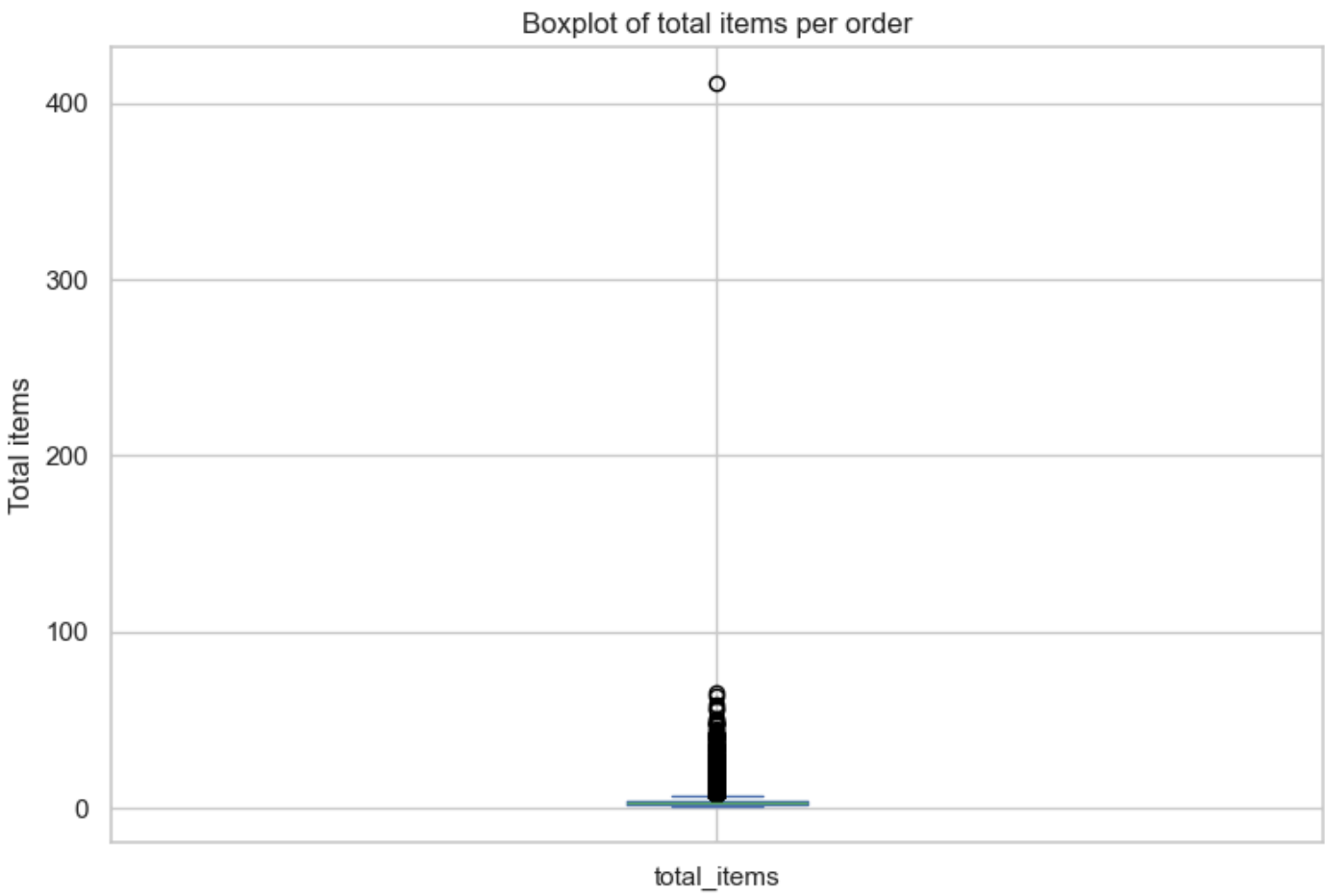
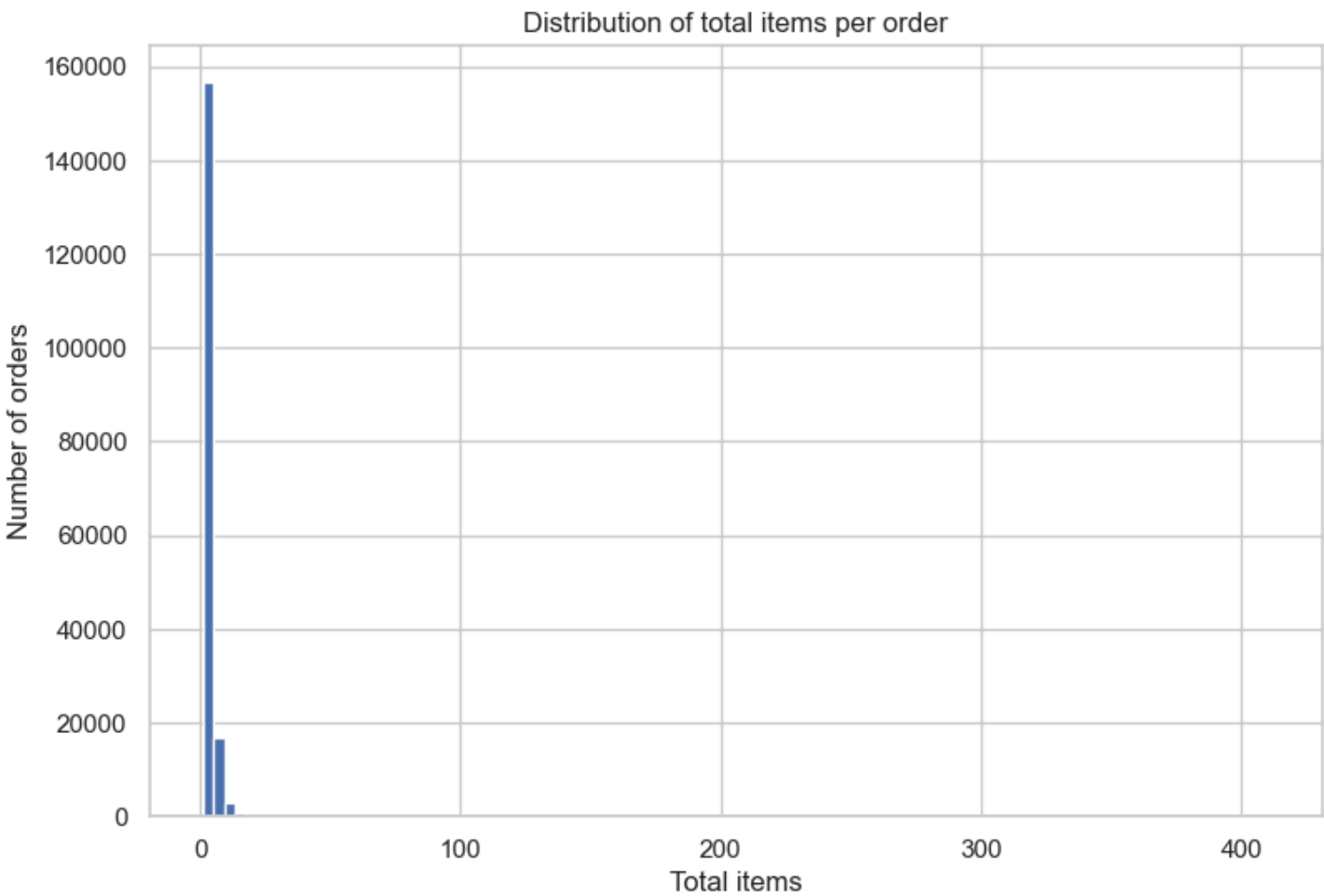
Observations

- Majority of the orders are from Order Protocol 1
- Order Protocol 7 has the least number of orders

Total Items

```
In [31]: fig, ax = plt.subplots(1,2, figsize=(20, 6))

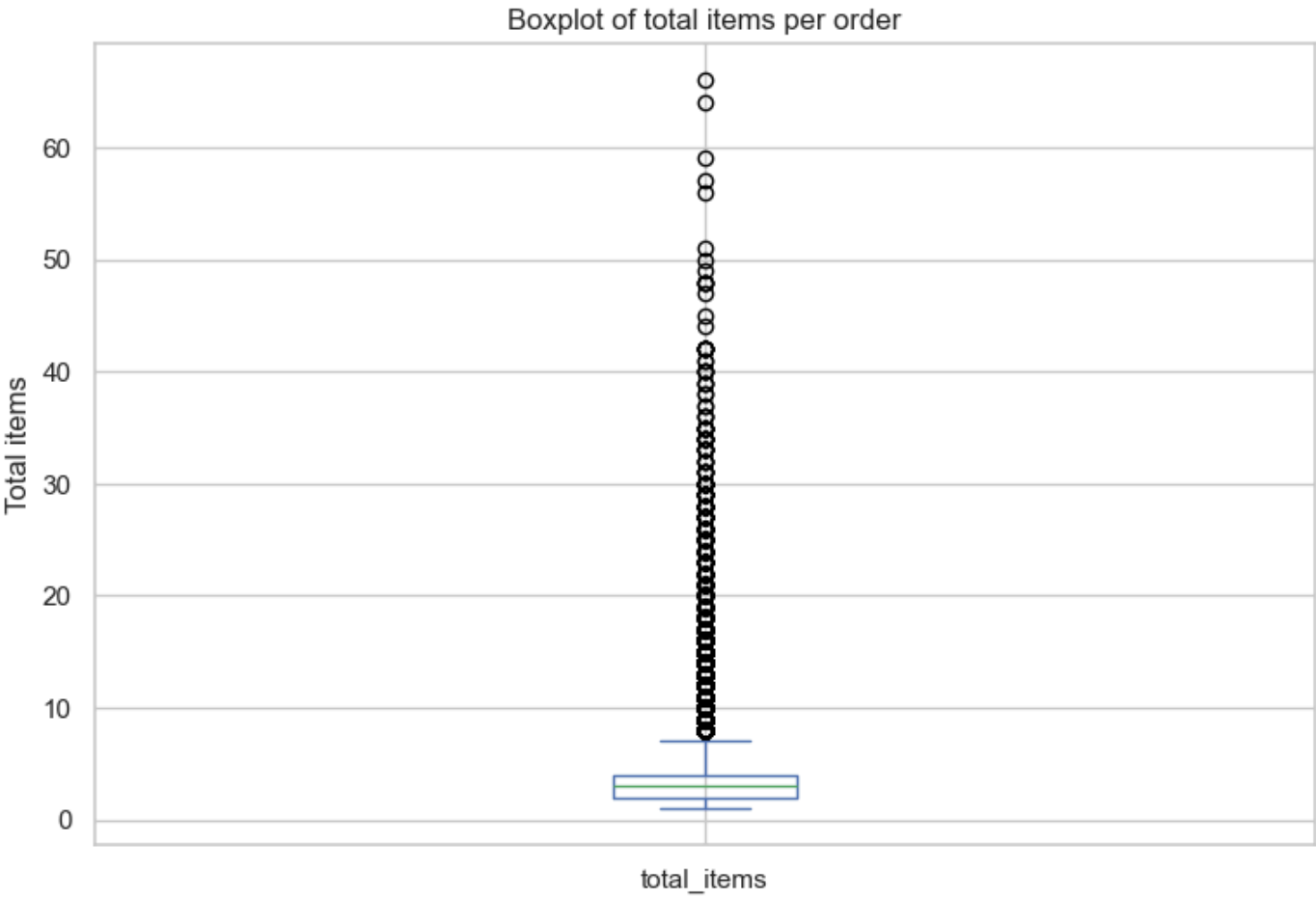
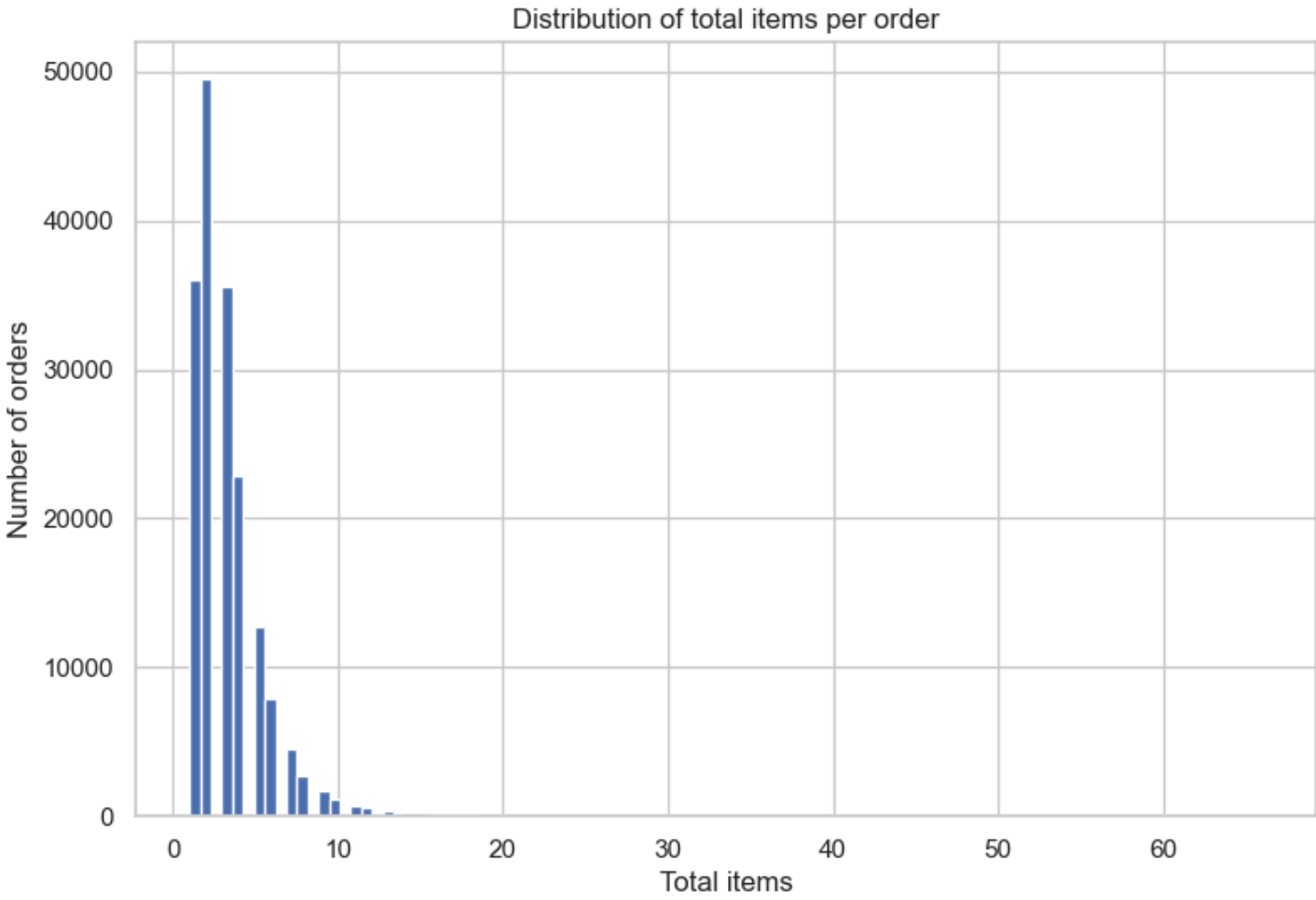
df["total_items"].hist(bins=100, ax=ax[0])
ax[0].set_title("Distribution of total items per order")
ax[0].set_xlabel("Total items")
ax[0].set_ylabel("Number of orders")
df["total_items"].plot(kind="box", ax=ax[1])
ax[1].set_title("Boxplot of total items per order")
ax[1].set_ylabel("Total items")
plt.show();
```



Observations

- We can see that there is an outlier in the data

```
In [8]: fig, ax = plt.subplots(1,2, figsize=(20, 6))
        _df[df["total_items"]<100]
        _["total_items"].hist(bins=100, ax=ax[0])
        ax[0].set_title("Distribution of total items per order")
        ax[0].set_xlabel("Total items")
        ax[0].set_ylabel("Number of orders")
        _["total_items"].plot(kind="box", ax=ax[1])
        ax[1].set_title("Boxplot of total items per order")
        ax[1].set_ylabel("Total items")
        plt.show();
```



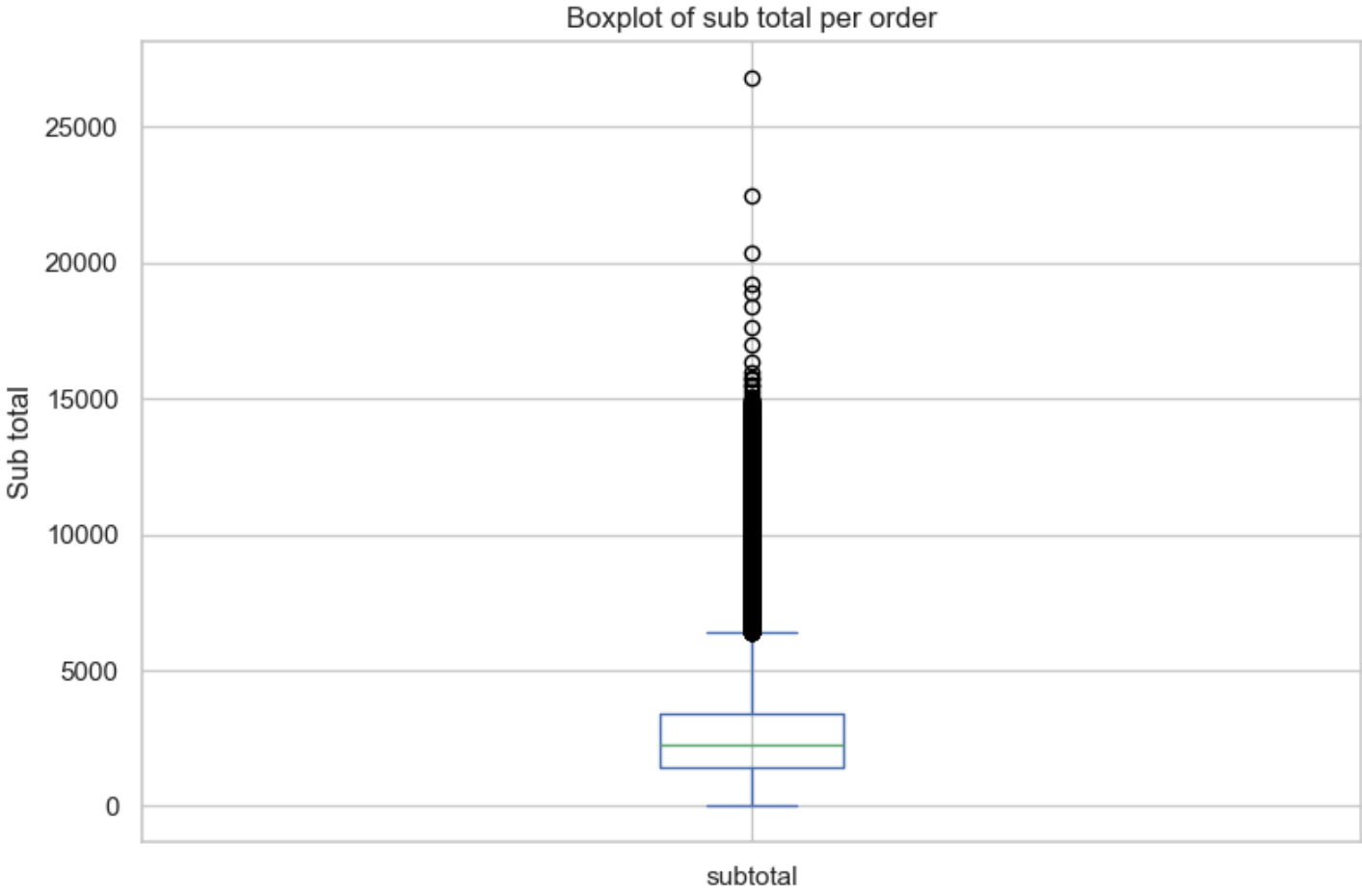
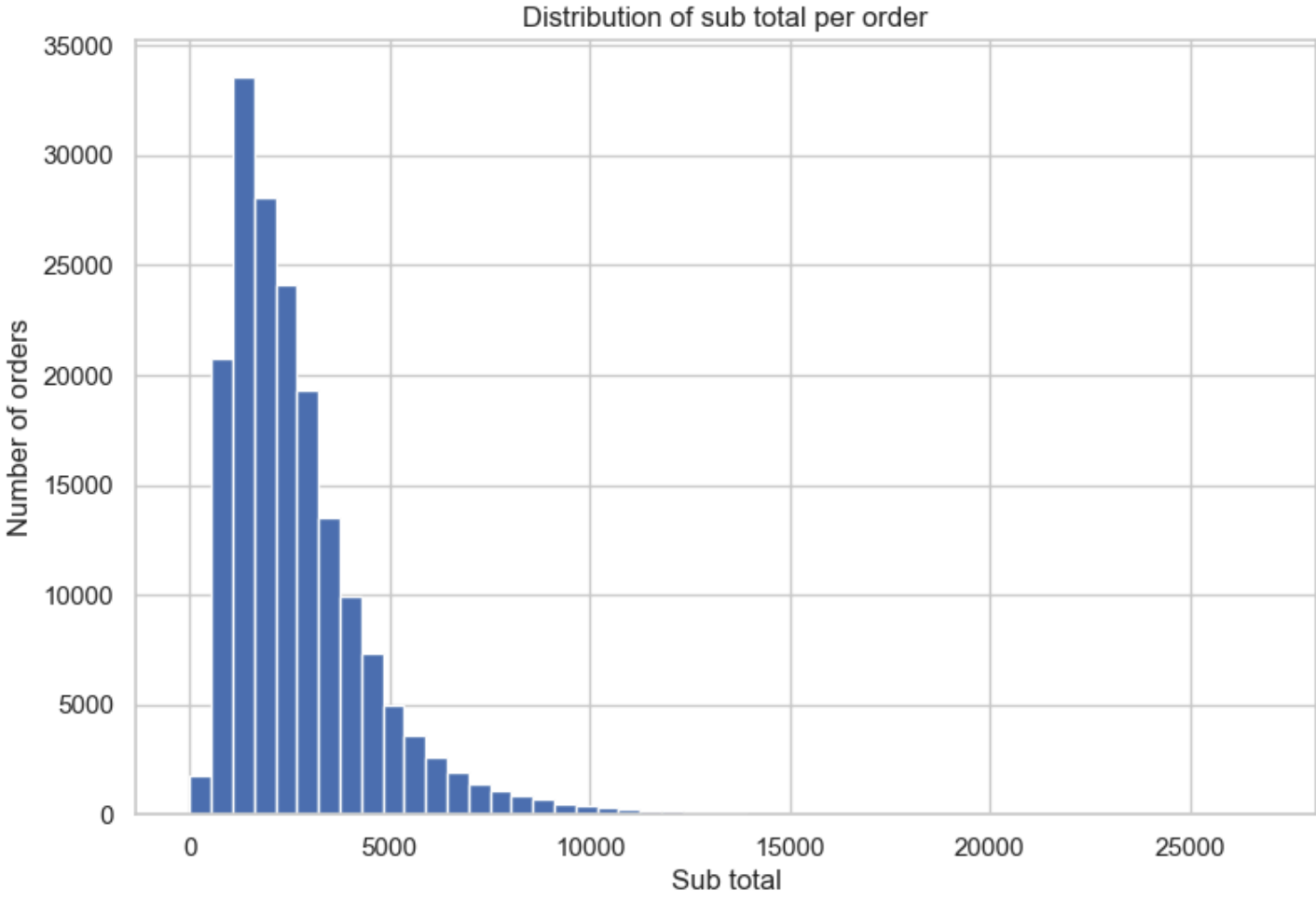
Observations

- From above plot we can see that most of the total items count is between 0 and 15

Sub Total

```
In [6]: fig, ax = plt.subplots(1,2, figsize=(20, 6))

        df["subtotal"].hist(bins=50, ax=ax[0])
        ax[0].set_title("Distribution of sub total per order")
        ax[0].set_xlabel("Sub total")
        ax[0].set_ylabel("Number of orders")
        df["subtotal"].plot(kind="box", ax=ax[1])
        ax[1].set_title("Boxplot of sub total per order")
        ax[1].set_ylabel("Sub total")
        plt.show();
```



Observations

- We can see that most of the subtotal values are under 3000

Min Price and Max Price

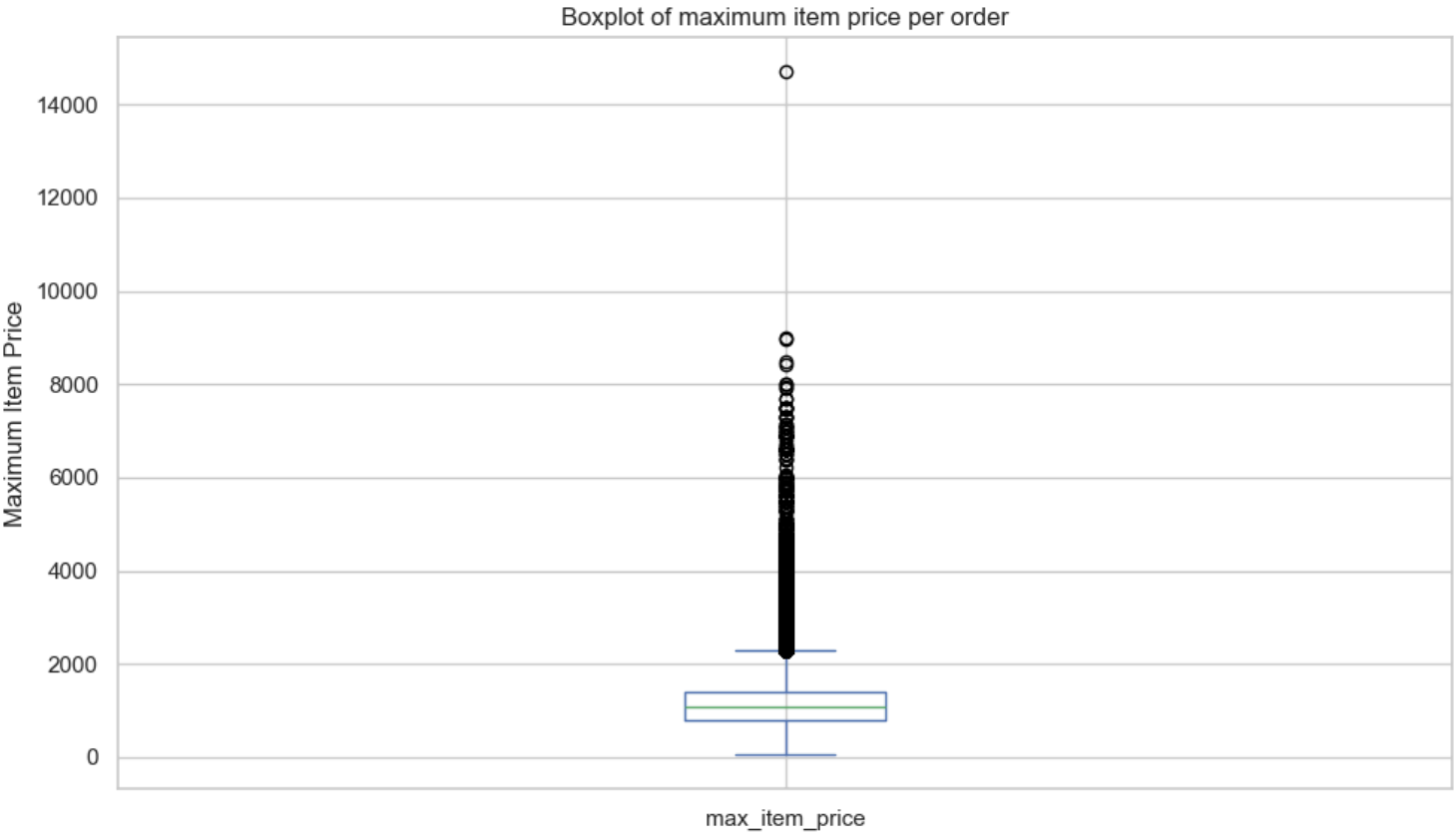
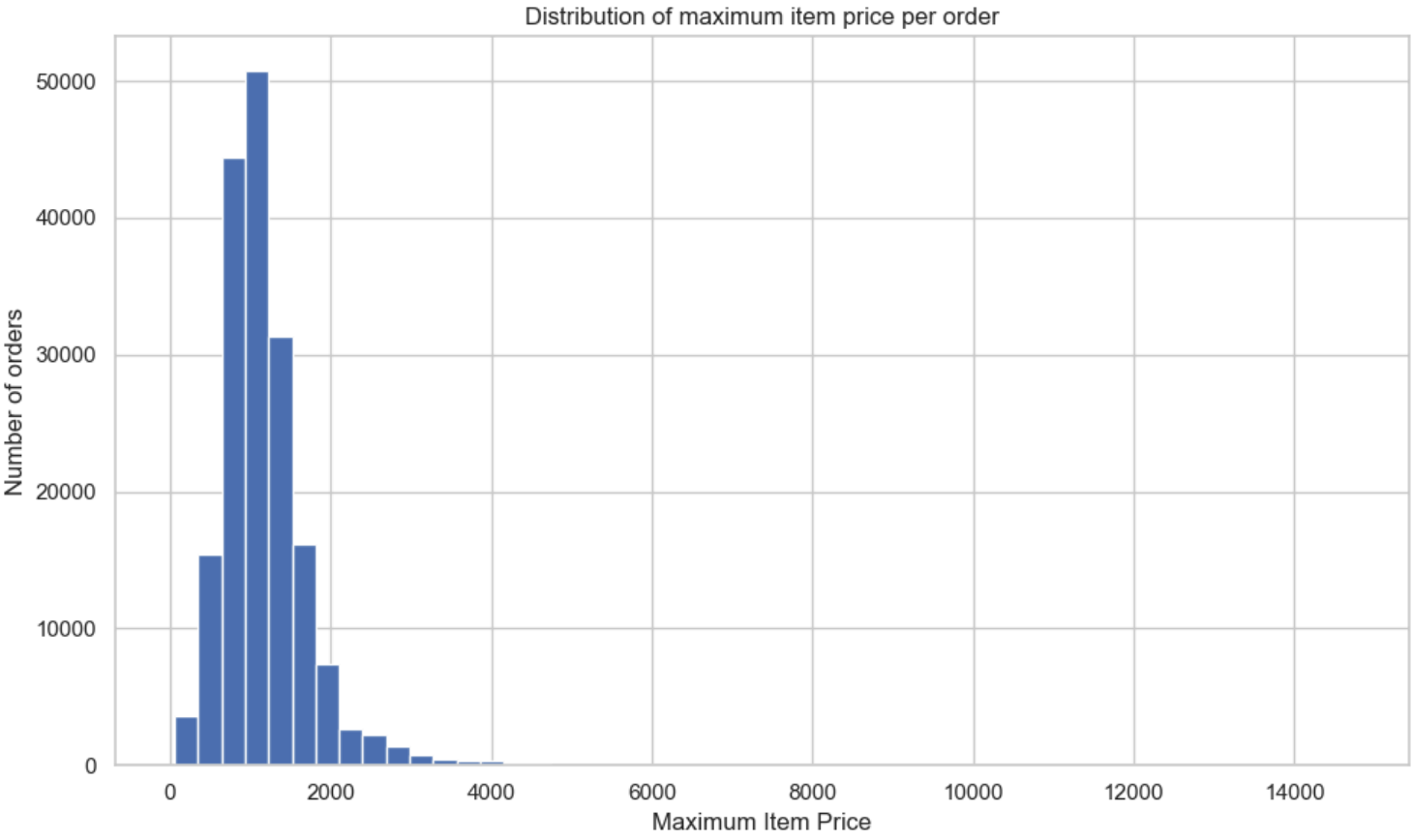
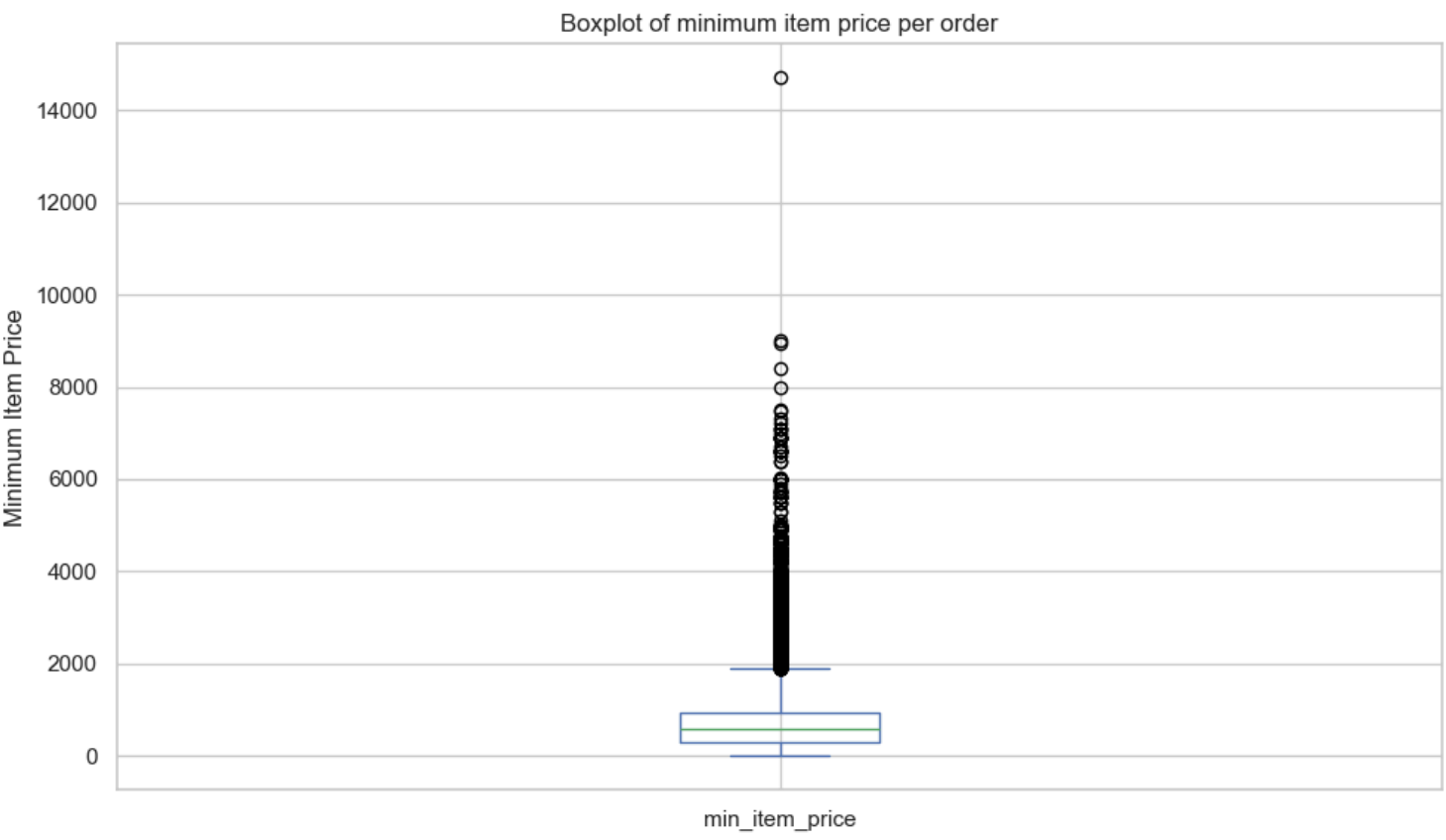
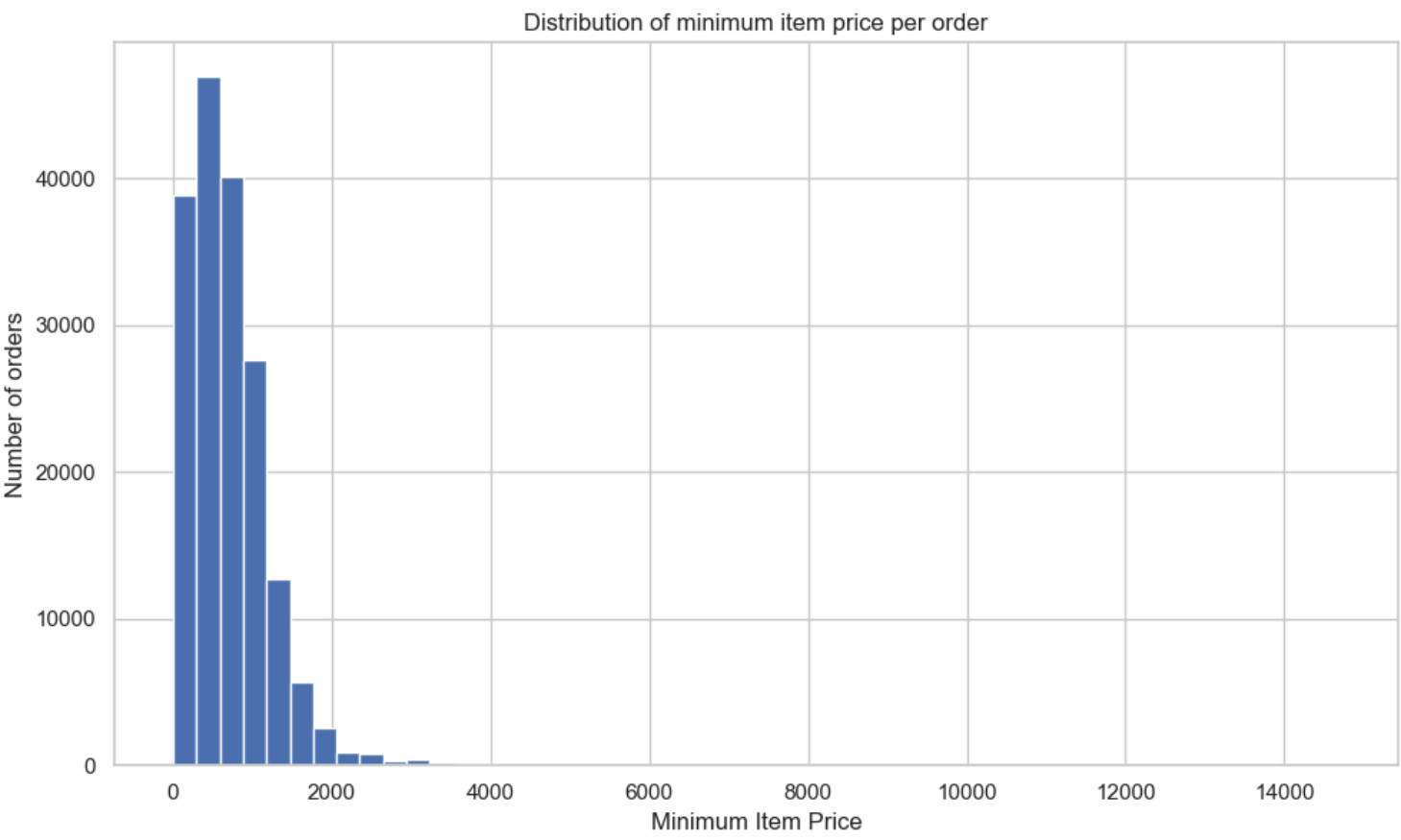
```
In [5]: fig, ax = plt.subplots(2,2, figsize=(20, 12))

        df["min_item_price"].hist(bins=50, ax=ax[0][0])
        ax[0][0].set_title("Distribution of minimum item price per order")
        ax[0][0].set_xlabel("Minimum Item Price")
        ax[0][0].set_ylabel("Number of orders")

        df["min_item_price"].plot(kind="box", ax=ax[0][1])
        ax[0][1].set_title("Boxplot of minimum item price per order")
        ax[0][1].set_ylabel("Minimum Item Price")

        # max_item_price
        df["max_item_price"].hist(bins=50, ax=ax[1][0])
        ax[1][0].set_title("Distribution of maximum item price per order")
        ax[1][0].set_xlabel("Maximum Item Price")
        ax[1][0].set_ylabel("Number of orders")
        df["max_item_price"].plot(kind="box", ax=ax[1][1])
        ax[1][1].set_title("Boxplot of maximum item price per order")
        ax[1][1].set_ylabel("Maximum Item Price")

        plt.tight_layout()
        plt.show();
```

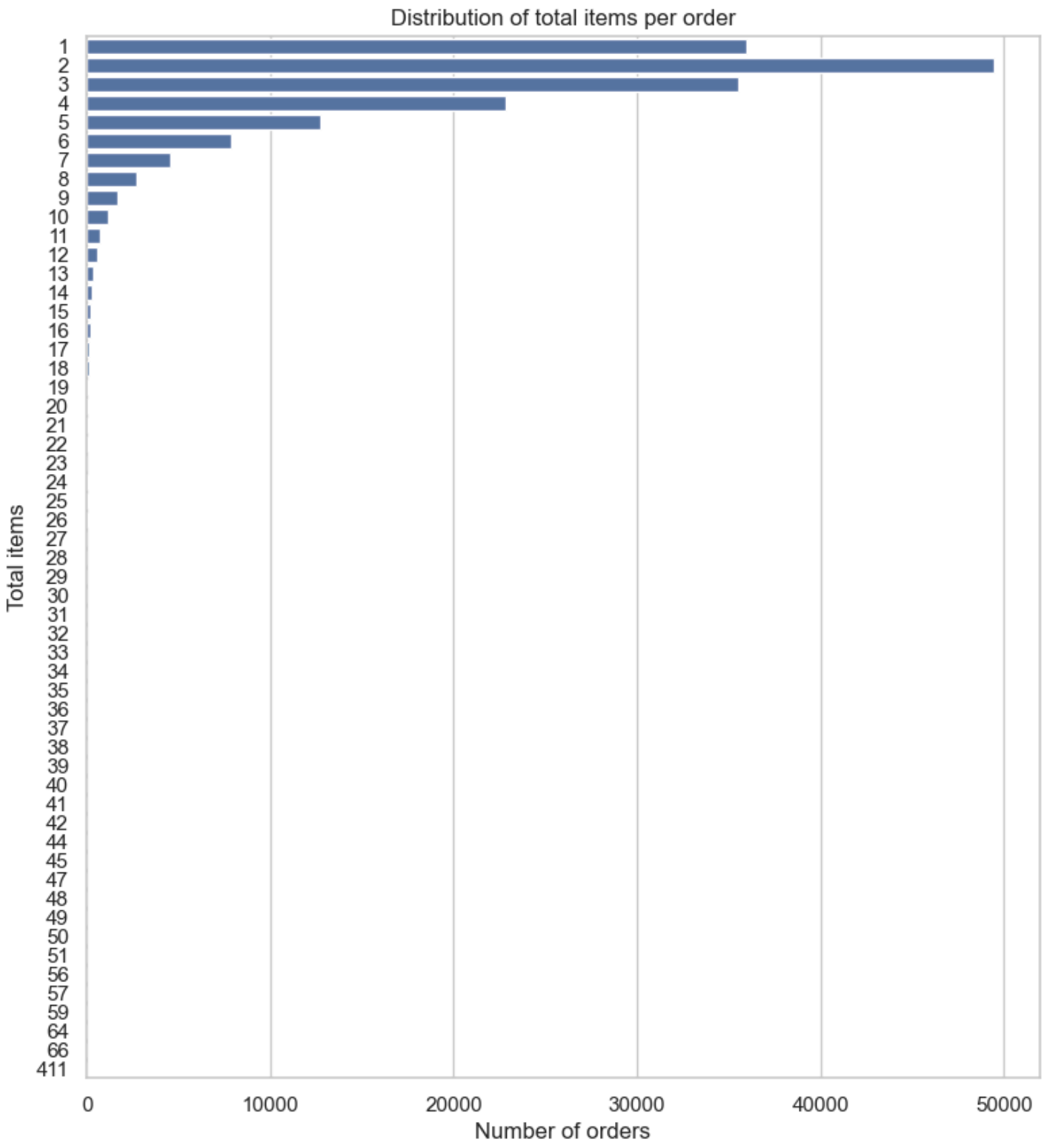
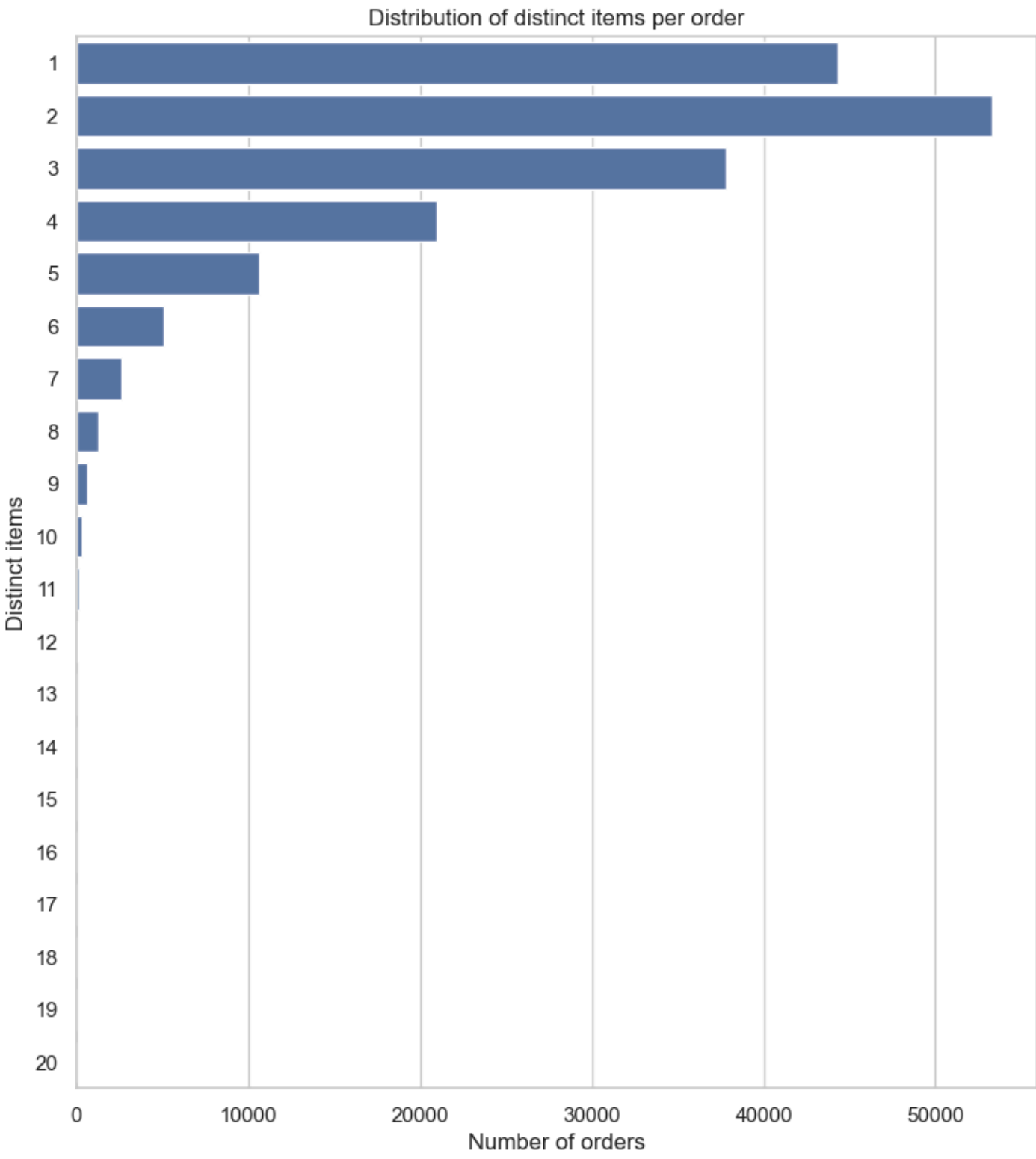


Observations

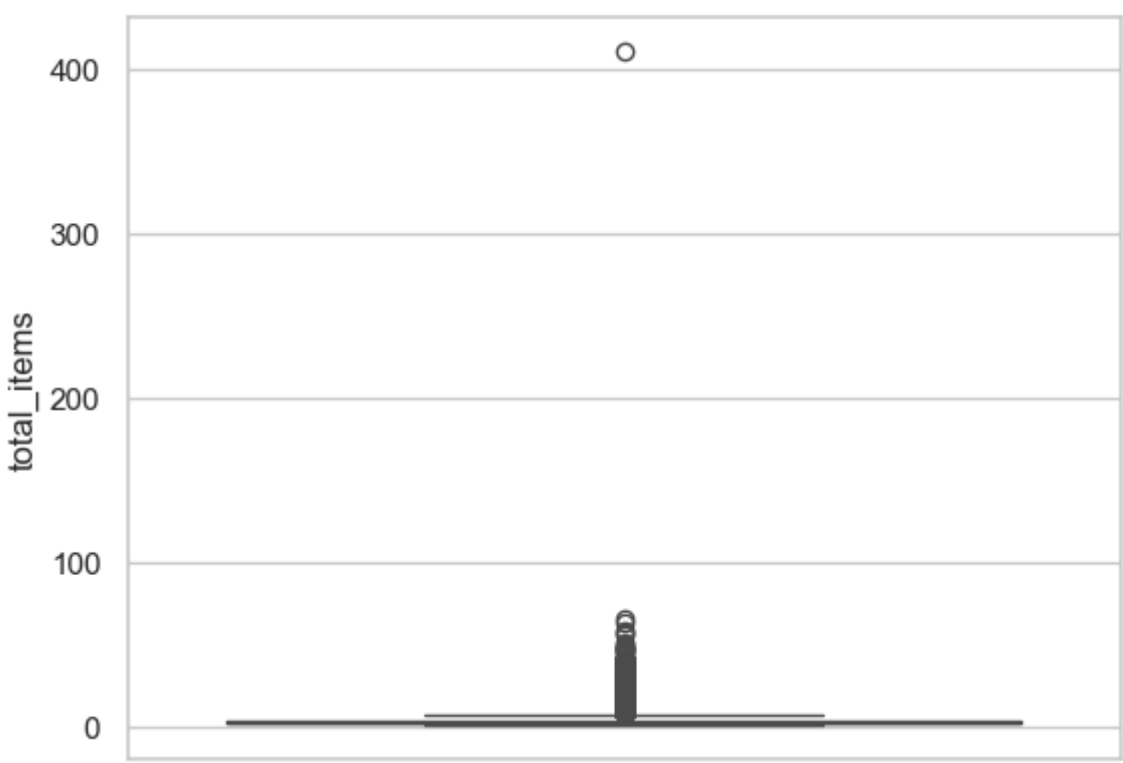
Distinct items

```
In [15]: fig, ax = plt.subplots(1, 2, figsize=(20, 10))
sns.countplot(data=df, y="num_distinct_items", ax=ax[0])
ax[0].set_title("Distribution of distinct items per order")
ax[0].set_ylabel("Distinct items")
ax[0].set_xlabel("Number of orders")

sns.countplot(data=df, y="total_items", ax=ax[1])
ax[1].set_title("Distribution of total items per order")
ax[1].set_ylabel("Total items")
ax[1].set_xlabel("Number of orders");
```



```
In [ ]:
```



Observations

Total OnShift Partners & Total Busy Partners

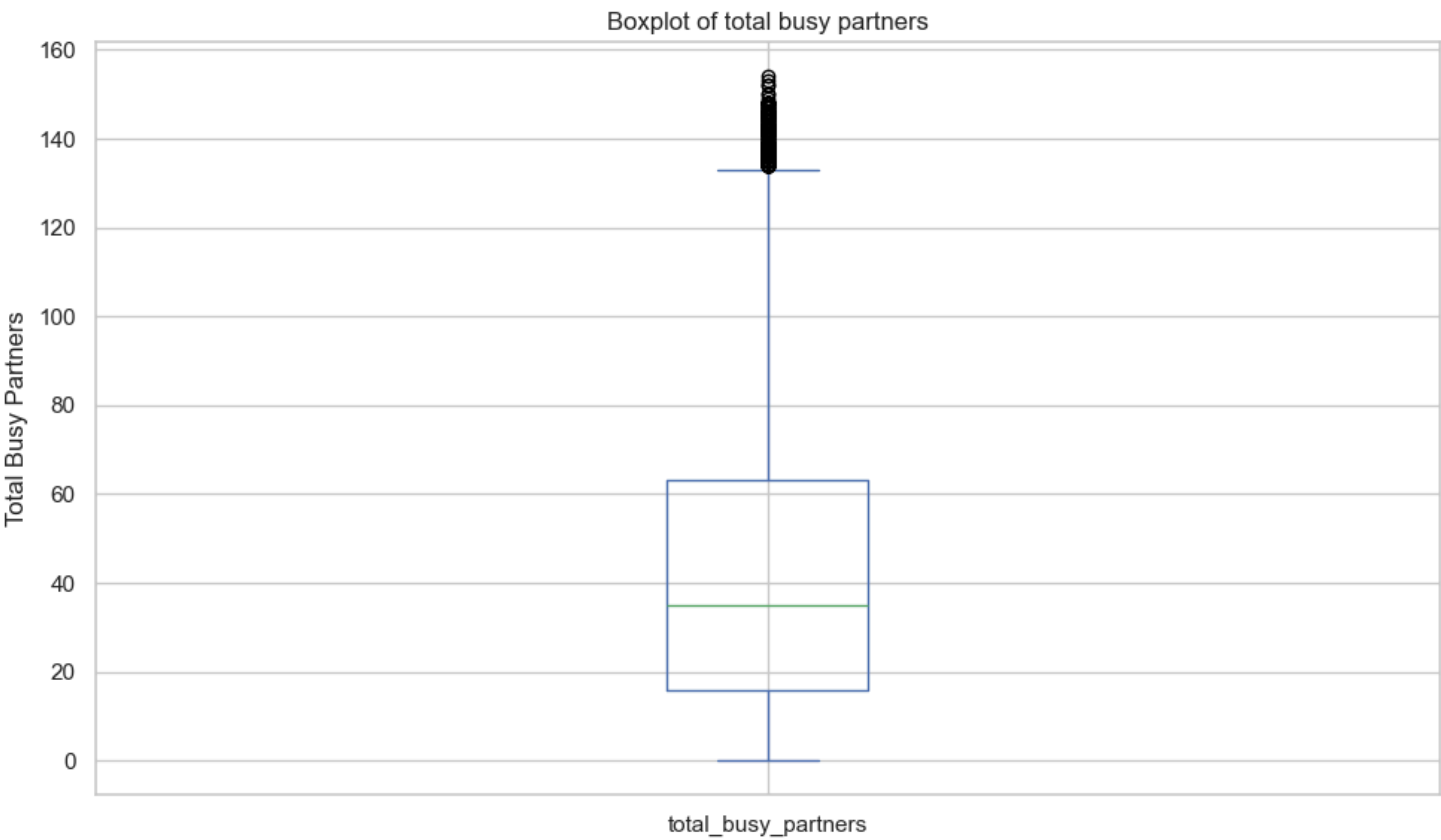
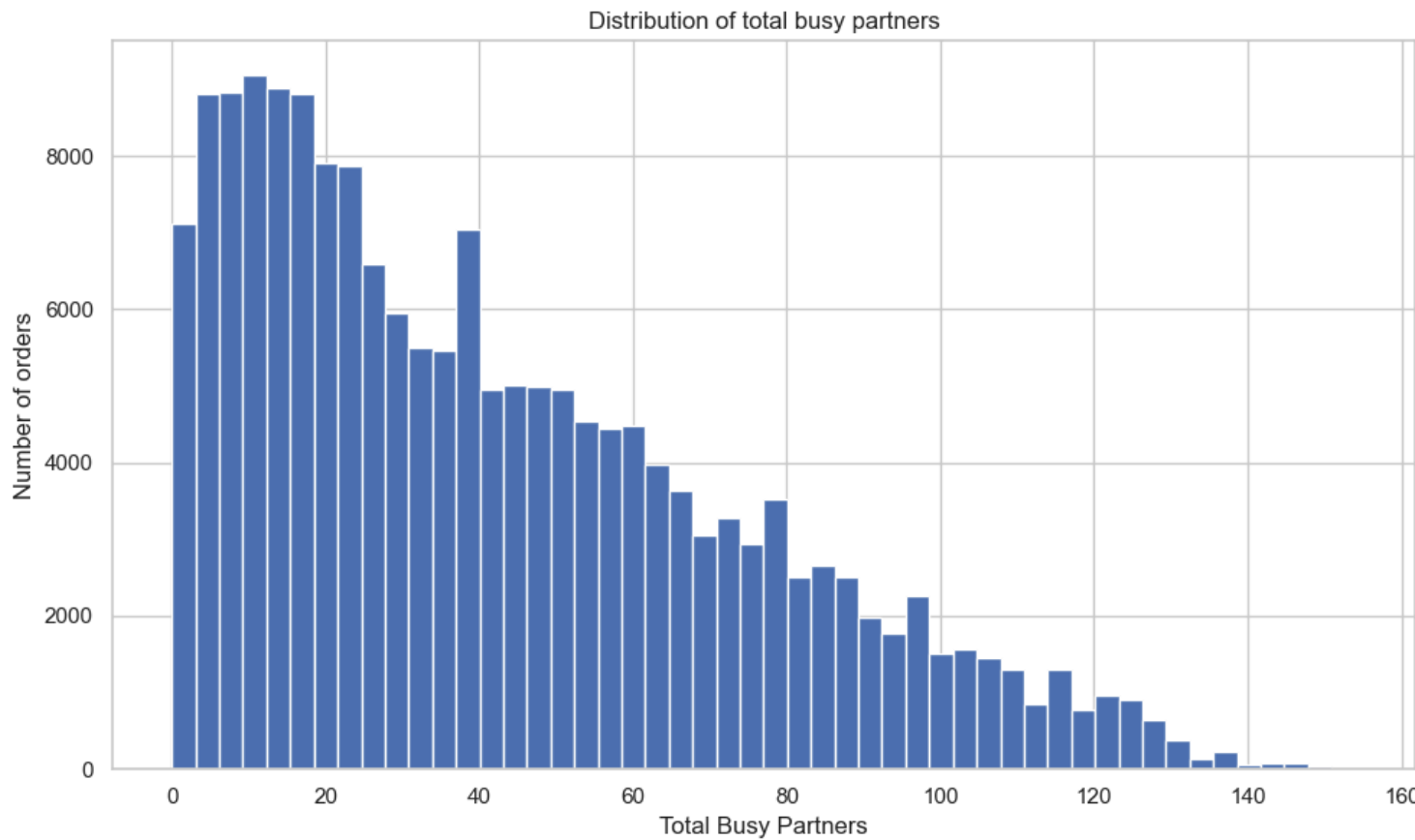
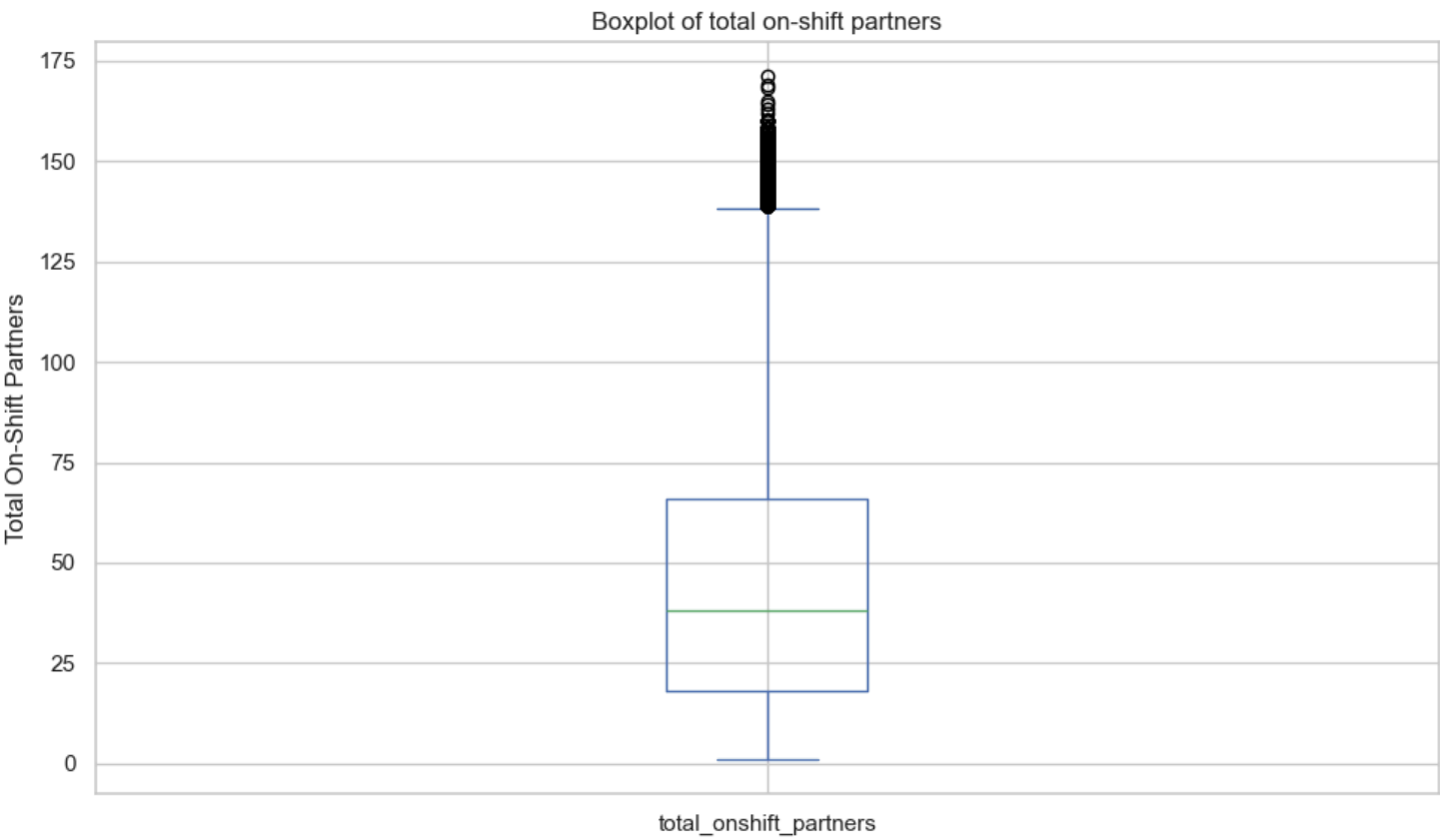
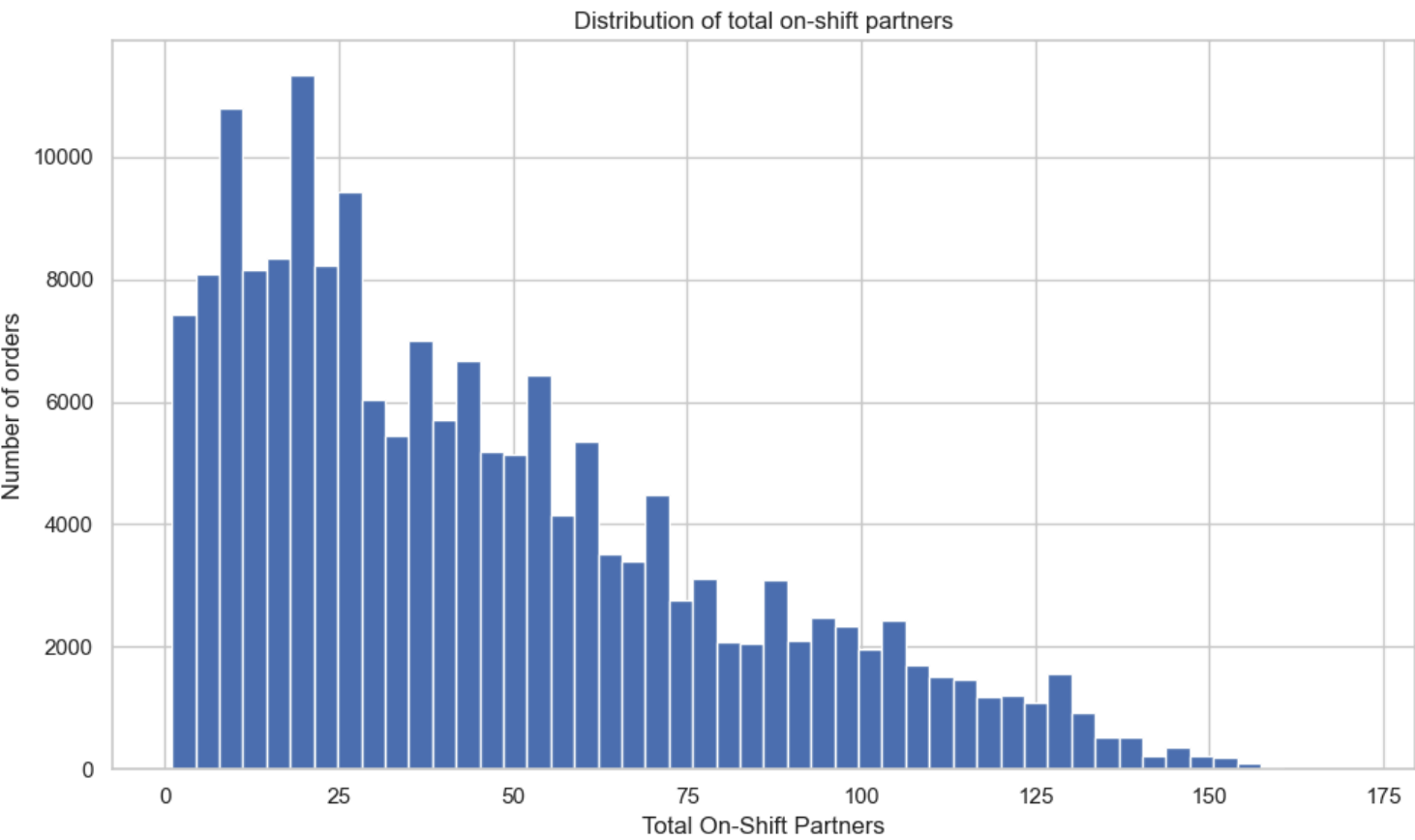
```
In [4]: fig, ax = plt.subplots(2,2, figsize=(20, 12))

df["total_onshift_partners"].hist(bins=50, ax=ax[0][0])
ax[0][0].set_title("Distribution of total on-shift partners")
ax[0][0].set_xlabel("Total On-Shift Partners")
ax[0][0].set_ylabel("Number of orders")

df["total_onshift_partners"].plot(kind="box", ax=ax[0][1])
ax[0][1].set_title("Boxplot of total on-shift partners")
ax[0][1].set_ylabel("Total On-Shift Partners")

# total_busy_partners
df["total_busy_partners"].hist(bins=50, ax=ax[1][0])
ax[1][0].set_title("Distribution of total busy partners")
ax[1][0].set_xlabel("Total Busy Partners")
ax[1][0].set_ylabel("Number of orders")
df["total_busy_partners"].plot(kind="box", ax=ax[1][1])
ax[1][1].set_title("Boxplot of total busy partners")
ax[1][1].set_ylabel("Total Busy Partners")

plt.tight_layout()
plt.show();
```

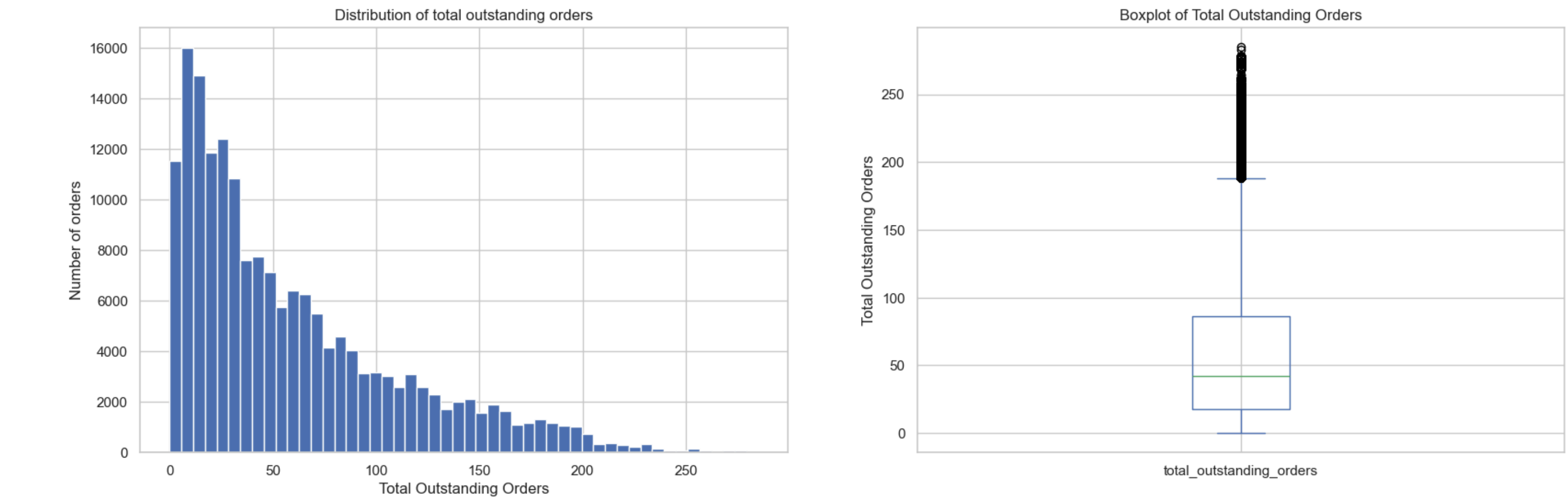


Observations

Total Outstanding Orders

```
In [9]: fig, ax = plt.subplots(1,2, figsize=(20, 6))

df["total_outstanding_orders"].hist(bins=50, ax=ax[0])
ax[0].set_title("Distribution of total outstanding orders")
ax[0].set_xlabel("Total Outstanding Orders")
ax[0].set_ylabel("Number of orders")
df["total_outstanding_orders"].plot(kind="box", ax=ax[1])
ax[1].set_title("Boxplot of Total Outstanding Orders")
ax[1].set_ylabel("Total Outstanding Orders")
plt.show();
```

Observations

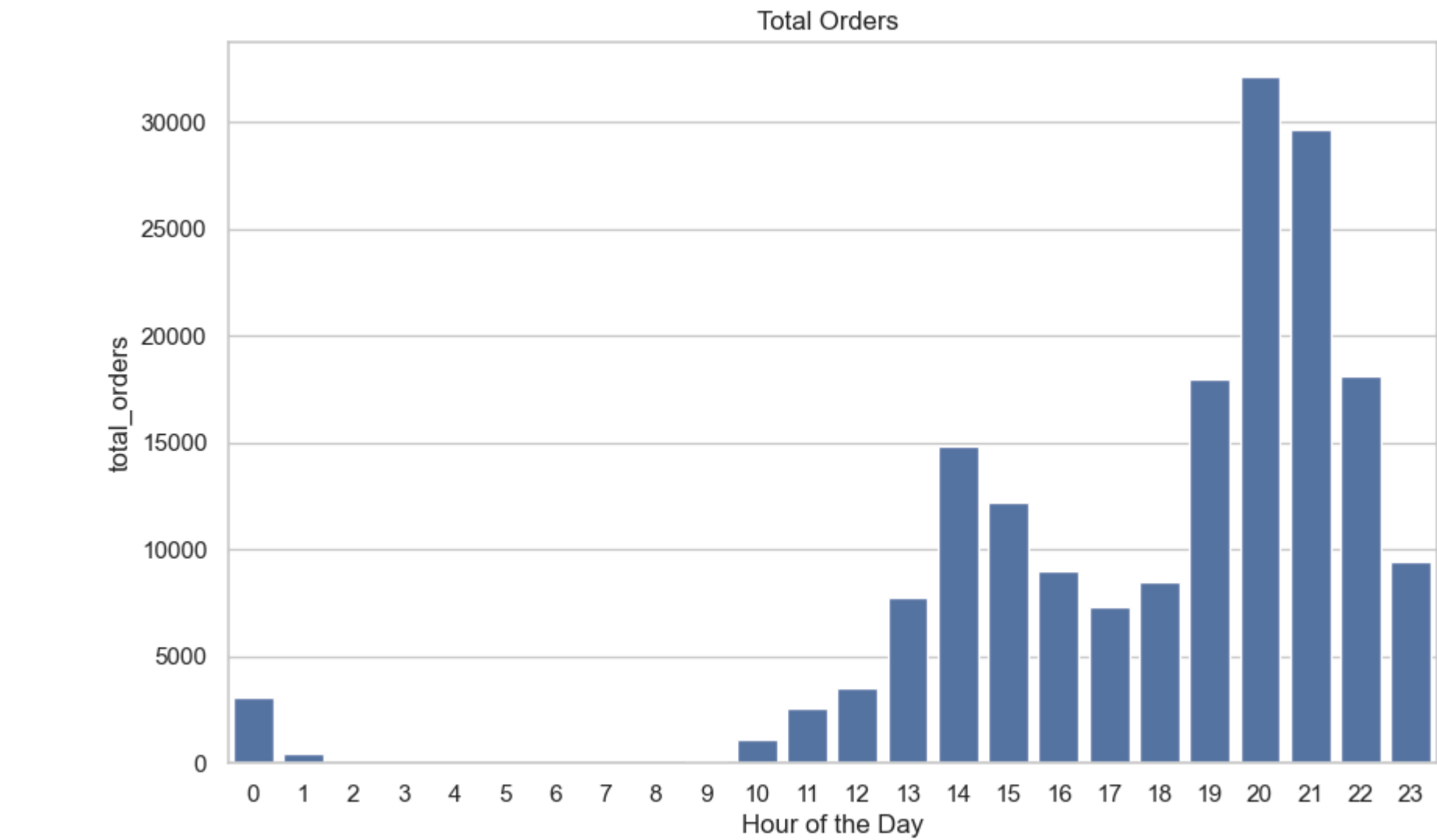
- At any given time there are around 30-80 outstanding orders

Order Created Hour

```
In [96]: full_hours = pd.DataFrame({'hour': range(0, 24)})

In [97]: full_hours=full_hours.copy().join(
df.groupby("created_at_hour").agg({"order_protocol": "size"}).rename(columns={"order_protocol": "total_orders"}),
on="hour",
how="left"
).fillna(0).set_index("hour")

In [98]: plt.figure(figsize=(10, 6))
sns.barplot(data=full_hours, x="hour", y="total_orders");
plt.title("Total Orders ")
plt.xlabel("Hour of the Day");
```



Observations

- We can see that during the lunch hours 1-2pm and dinner hours 7-10pm the number of orders is high

```
In [99]: orders_per_hour_df = db.sql("""
select created_at_hour as hour, round(count(*)/((date_diff('day', min(created_at), max(created_at))+1)*24),2) as orders_per_hour
from df
group by created_at_hour
order by created_at_hour
""").to_df()

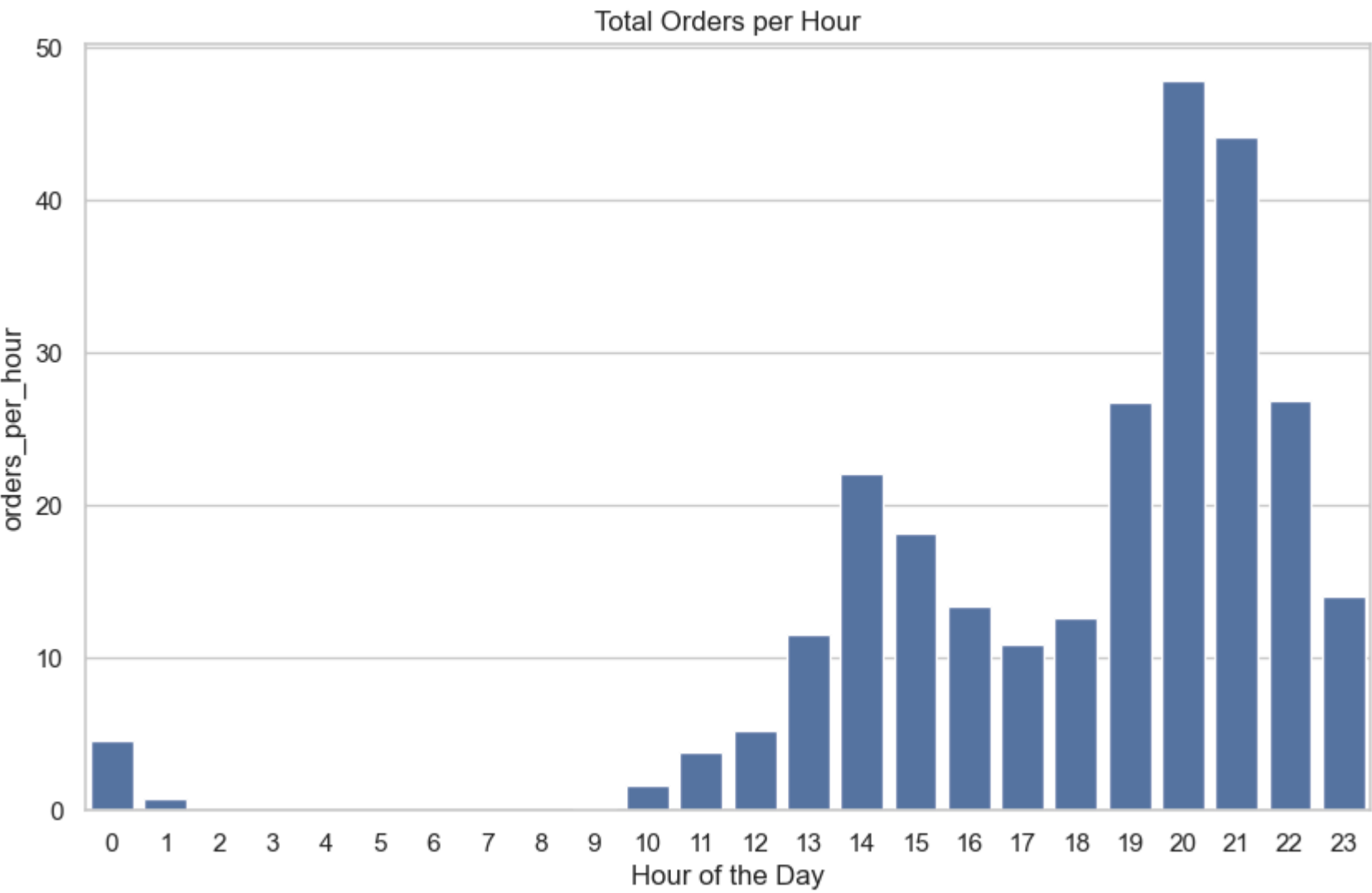
result_df = full_hours.copy().merge(orders_per_hour_df, on='hour', how='left').fillna(0)
result_df
```


Out [99]:

	hour	total_orders	orders_per_hour
0	0	3087.0	4.59
1	1	455.0	0.79
2	2	0.0	0.00
3	3	0.0	0.00
4	4	0.0	0.00
5	5	0.0	0.00
6	6	0.0	0.00
7	7	0.0	0.00
8	8	0.0	0.00
9	9	104.0	0.15
10	10	1075.0	1.60
11	11	2589.0	3.85
12	12	3484.0	5.18
13	13	7780.0	11.58
14	14	14810.0	22.04
15	15	12213.0	18.17
16	16	8993.0	13.38
17	17	7322.0	10.90
18	18	8459.0	12.59
19	19	17954.0	26.72
20	20	32117.0	47.79
21	21	29626.0	44.09
22	22	18080.0	26.90
23	23	9396.0	13.98

In [100]:

```
plt.figure(figsize=(10, 6))
sns.barplot(data=result_df, x="hour", y="orders_per_hour");
plt.title("Total Orders per Hour")
plt.xlabel("Hour of the Day");
```



Observations

- We can see that during the lunch hours 1-2pm and dinner hours 7-10pm the average number of orders is 25 and 45 respectively

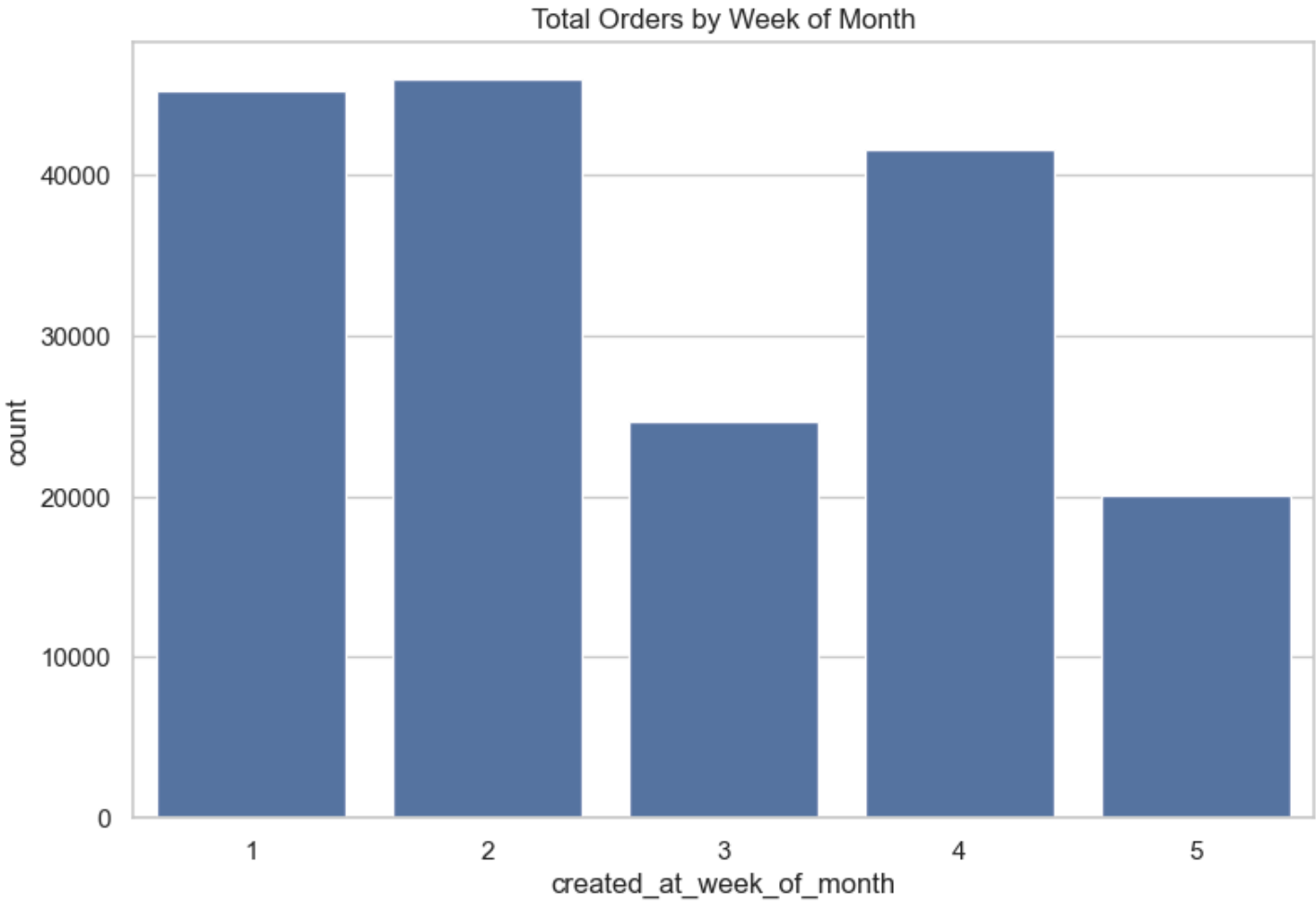
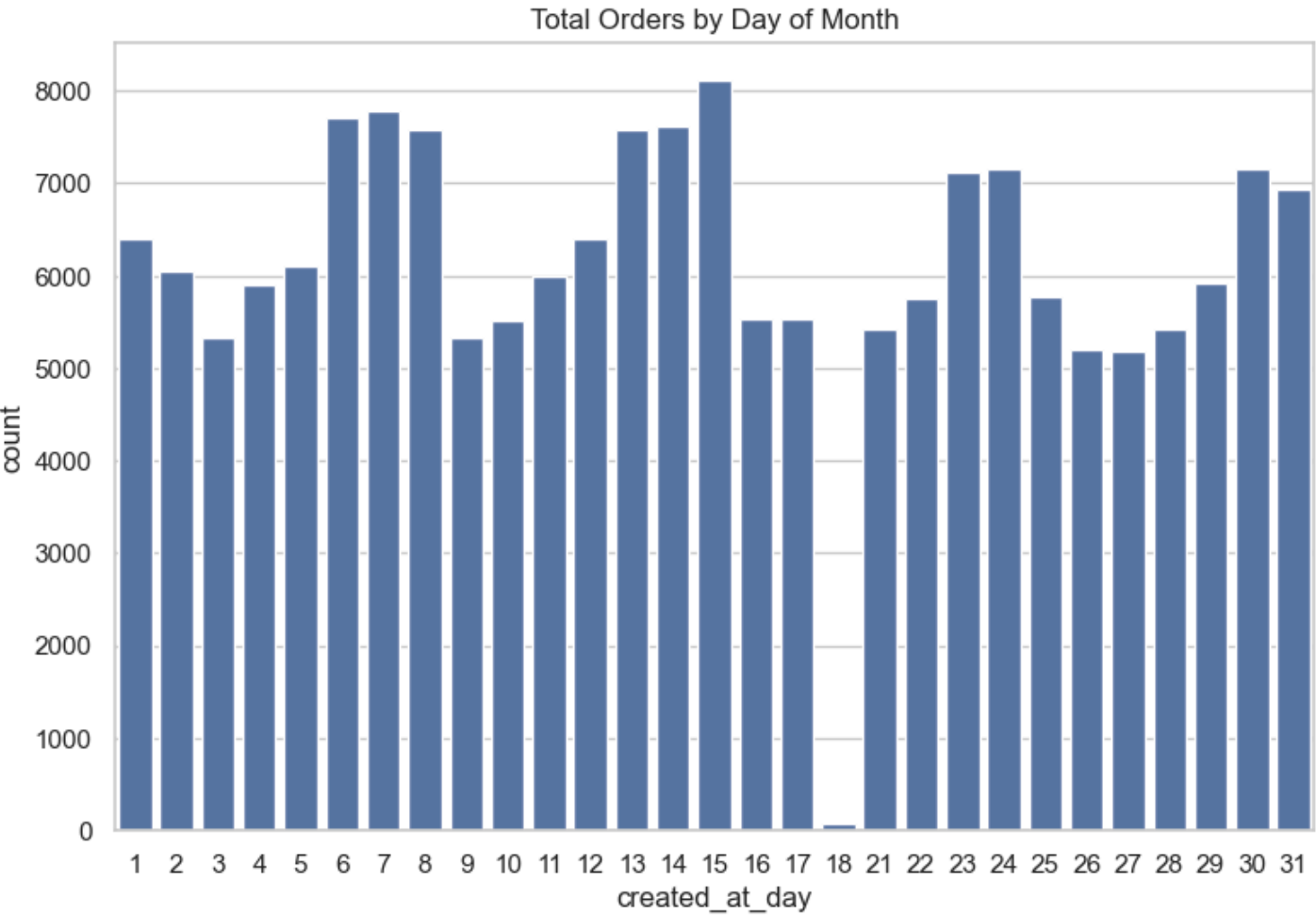
Order Created Week

In [9]:

```
fig, ax = plt.subplots(1,2, figsize=(20, 6))

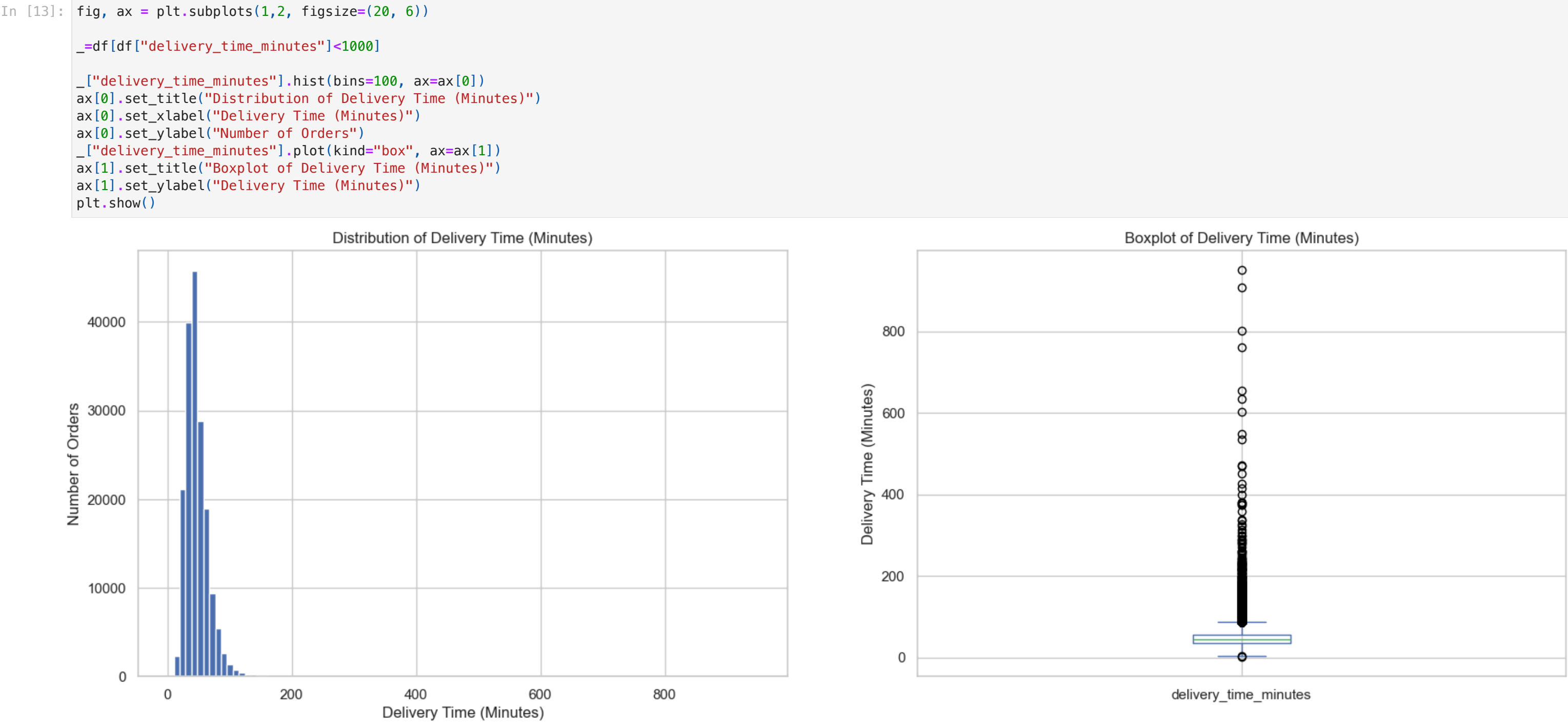
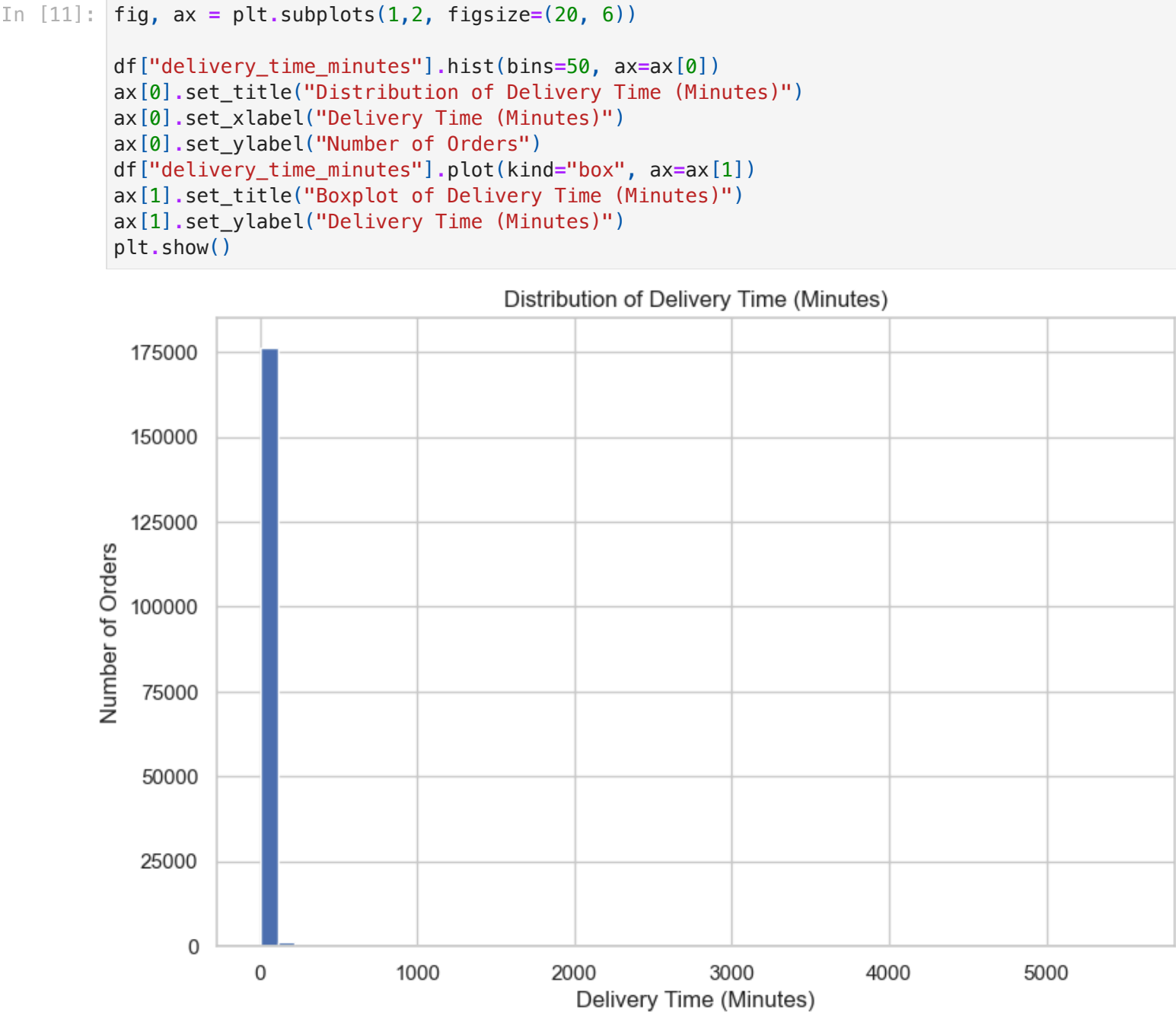
sns.countplot(data=df, x="created_at_day", ax=ax[0]);
ax[0].set_title("Total Orders by Day of Month");

sns.countplot(data=df, x="created_at_week_of_month", ax=ax[1]);
ax[1].set_title("Total Orders by Week of Month");
plt.show();
```



Observations

- We can see that there is a repeating pattern in the daily order data of a month
- We can see that orders are high in first 2 weeks and low in the last week of the month



Observations

- Most of the delivery times are under 100 minutes

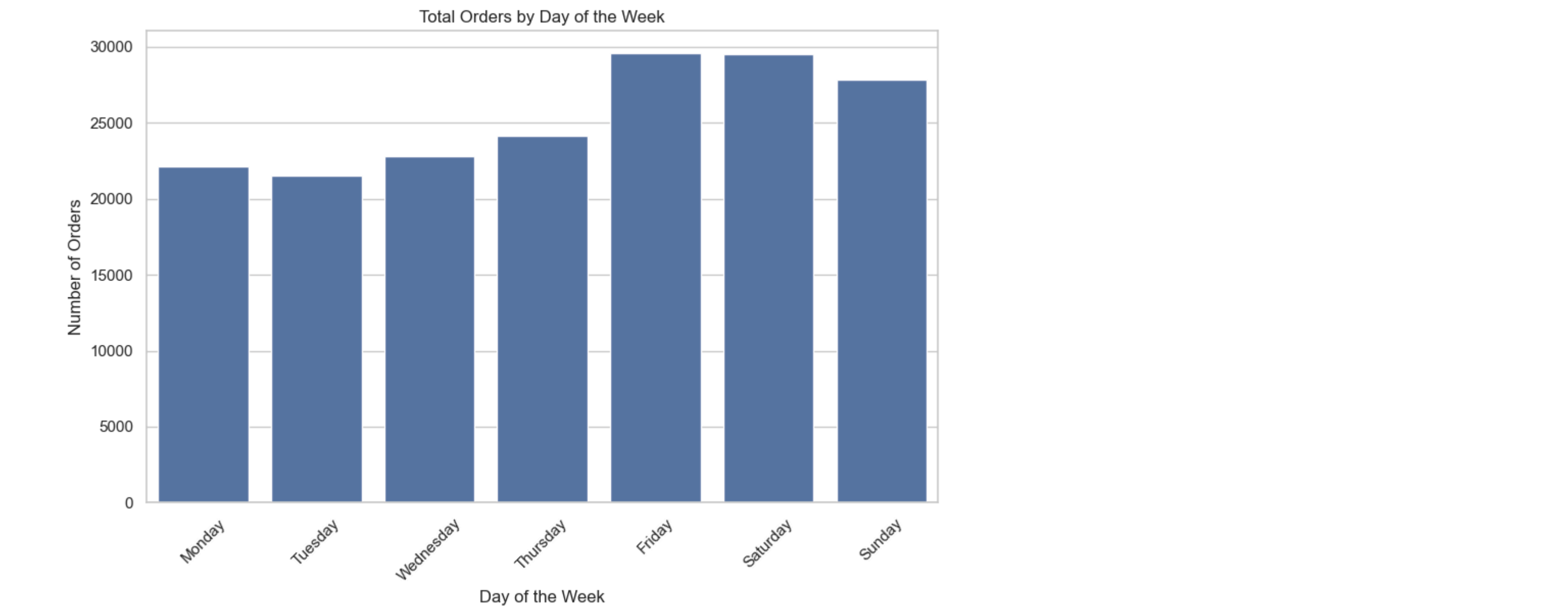
```
In [37]: temp=df["created_at_day_of_week"].value_counts().reset_index().sort_values(by="created_at_day_of_week")
temp
```

```
Out[37]:
```

	created_at_day_of_week	count
5	1	22105
6	2	21547
4	3	22796
3	4	24177
0	5	29562
1	6	29490
2	7	27867

```
In [42]: plt.figure(figsize=(10, 6))

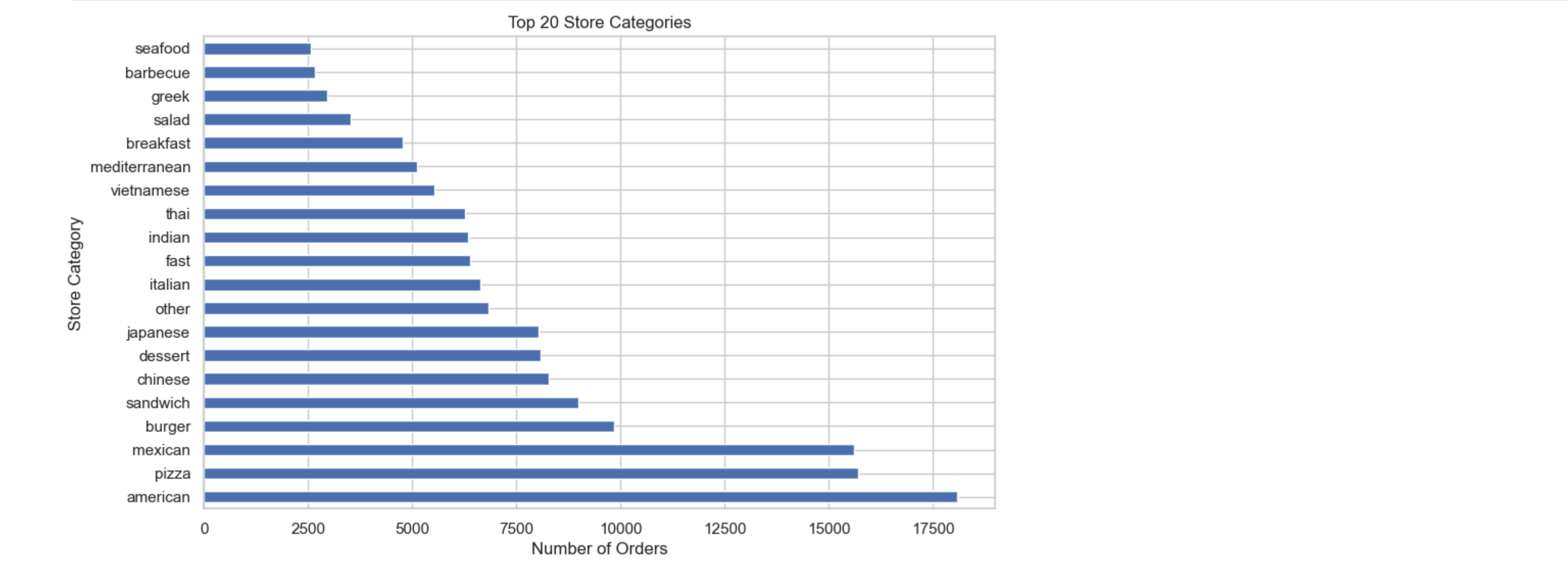
sns.barplot(data=temp, x="created_at_day_of_week", y="count")
day_names = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
tick_positions = plt.xticks()[0]
plt.xticks(tick_positions, day_names, rotation=45)
plt.xlabel("Day of the Week")
plt.ylabel("Number of Orders")
plt.title("Total Orders by Day of the Week");
```



Observations

- Most of the deliveries happen on Friday, Saturday and Sunday

```
In [71]: df["store_primary_category"].value_counts().head(20).plot(kind="barh", figsize=(10, 6));
plt.title("Top 20 Store Categories")
plt.xlabel("Number of Orders")
plt.ylabel("Store Category")
plt.show();
```



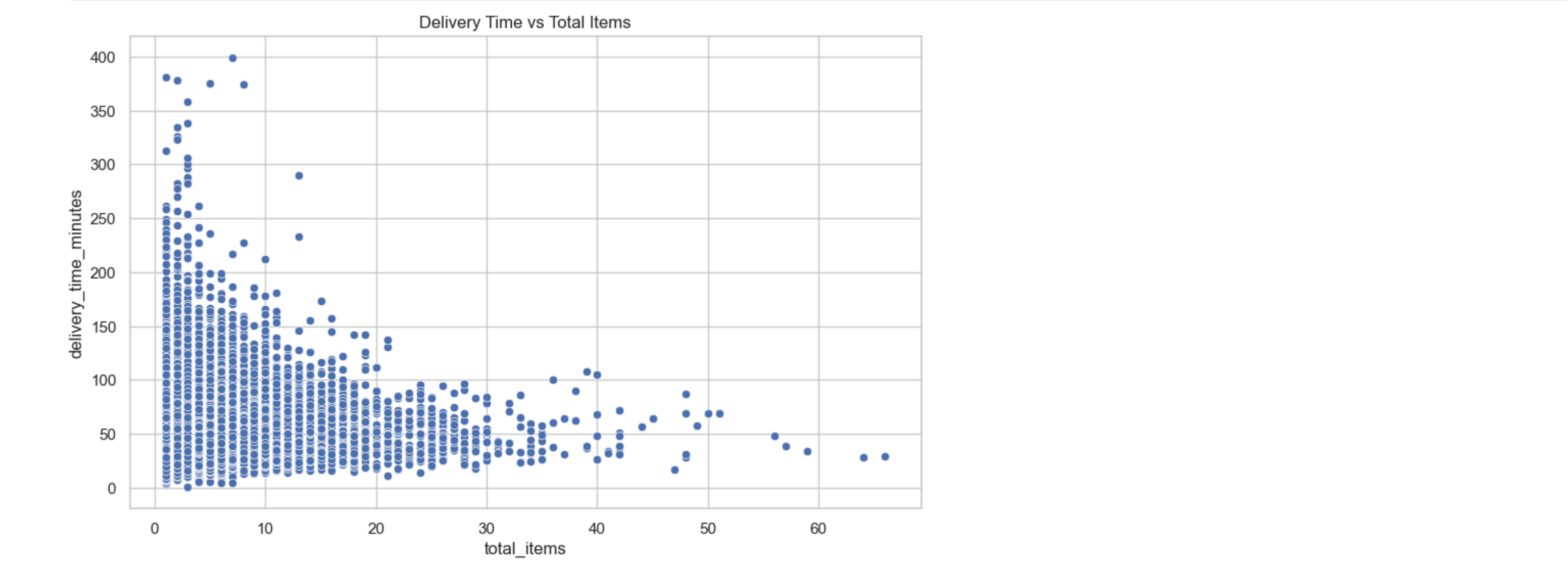
Observations

- We can see that American, Pizza, and Mexican are the most ordered categories

Bivariate Analysis

```
In [8]: t_df=df[(df["delivery_time_minutes"]<400) & (df["total_items"]<100) & (df["max_item_price"]<10000)]
```

```
In [114]: plt.figure(figsize=(10, 6))
sns.scatterplot(data=t_df, x='total_items', y='delivery_time_minutes');
plt.title("Delivery Time vs Total Items");
```

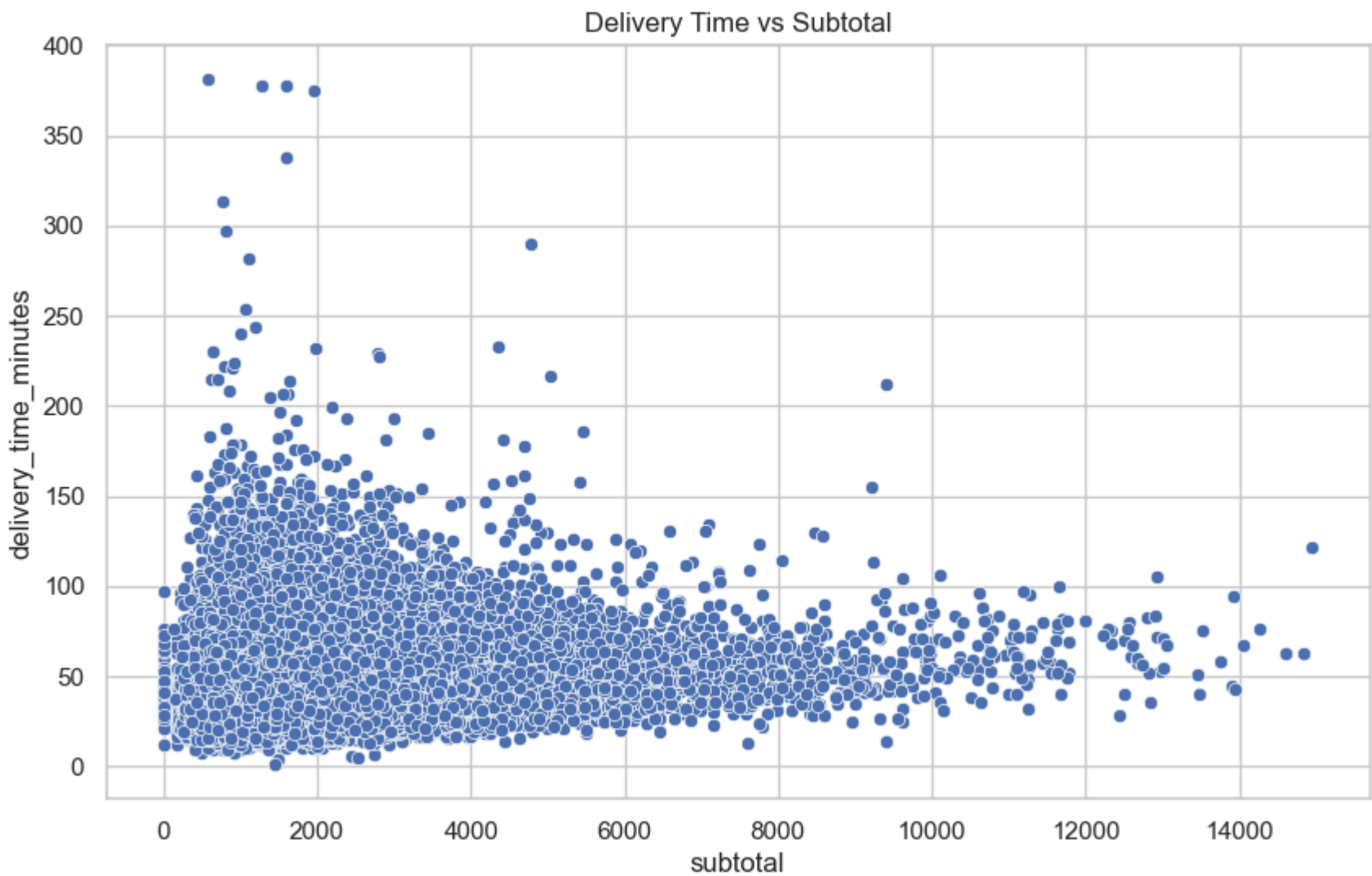


Observations

- We can see that delivery time decreases as the number of total items increases

Subtotal vs Total Items

```
In [99]: plt.figure(figsize=(10, 6))
sns.scatterplot(data=t_df, x='subtotal', y='delivery_time_minutes');
plt.title("Delivery Time vs Subtotal");
```

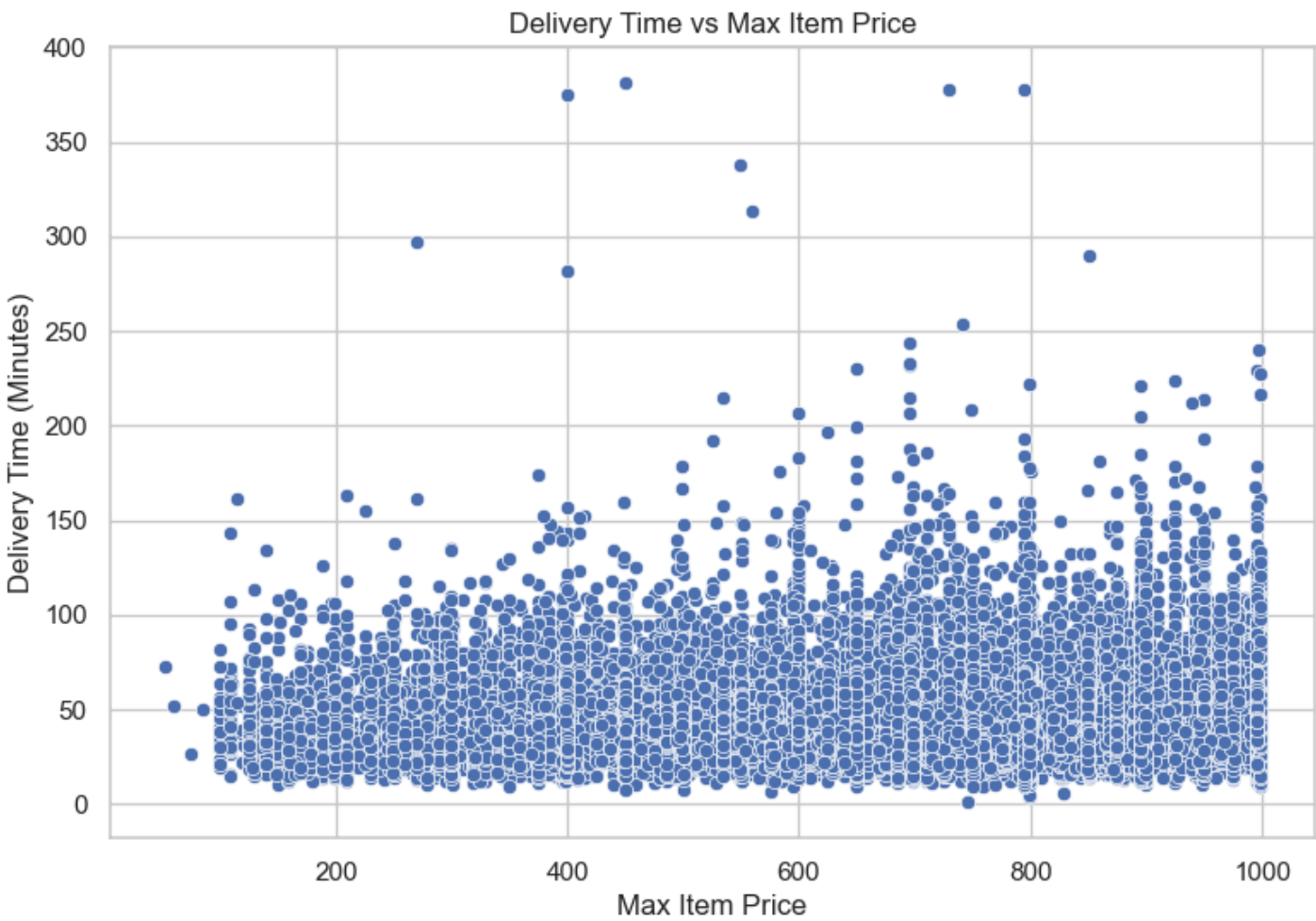


Observations

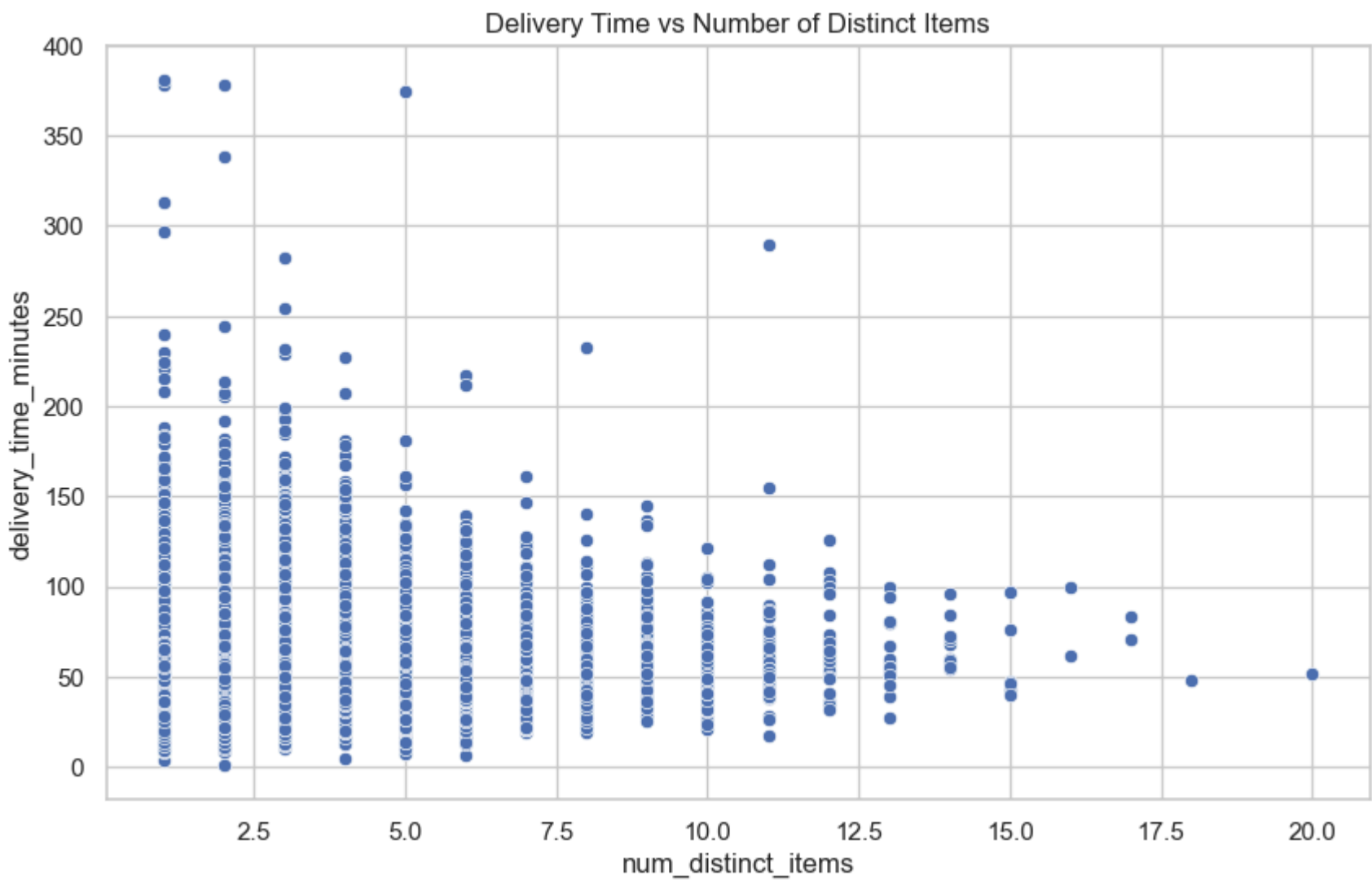
```
In [100]: fig, ax = plt.subplots(1,2, figsize=(20, 6))

sns.scatterplot(data=t_df, x='min_item_price', y='delivery_time_minutes', ax=ax[0]);
ax[0].set_title("Delivery Time vs Min Item Price");
ax[0].set_xlabel("Min Item Price")
ax[0].set_ylabel("Delivery Time (Minutes)")

sns.scatterplot(data=t_df, x='max_item_price', y='delivery_time_minutes', ax=ax[1]);
ax[1].set_title("Delivery Time vs Max Item Price");
ax[1].set_xlabel("Max Item Price")
ax[1].set_ylabel("Delivery Time (Minutes)");
```



```
In [101]: plt.figure(figsize=(10, 6))
sns.scatterplot(data=t_df, x='num_distinct_items', y='delivery_time_minutes');
plt.title("Delivery Time vs Number of Distinct Items");
```



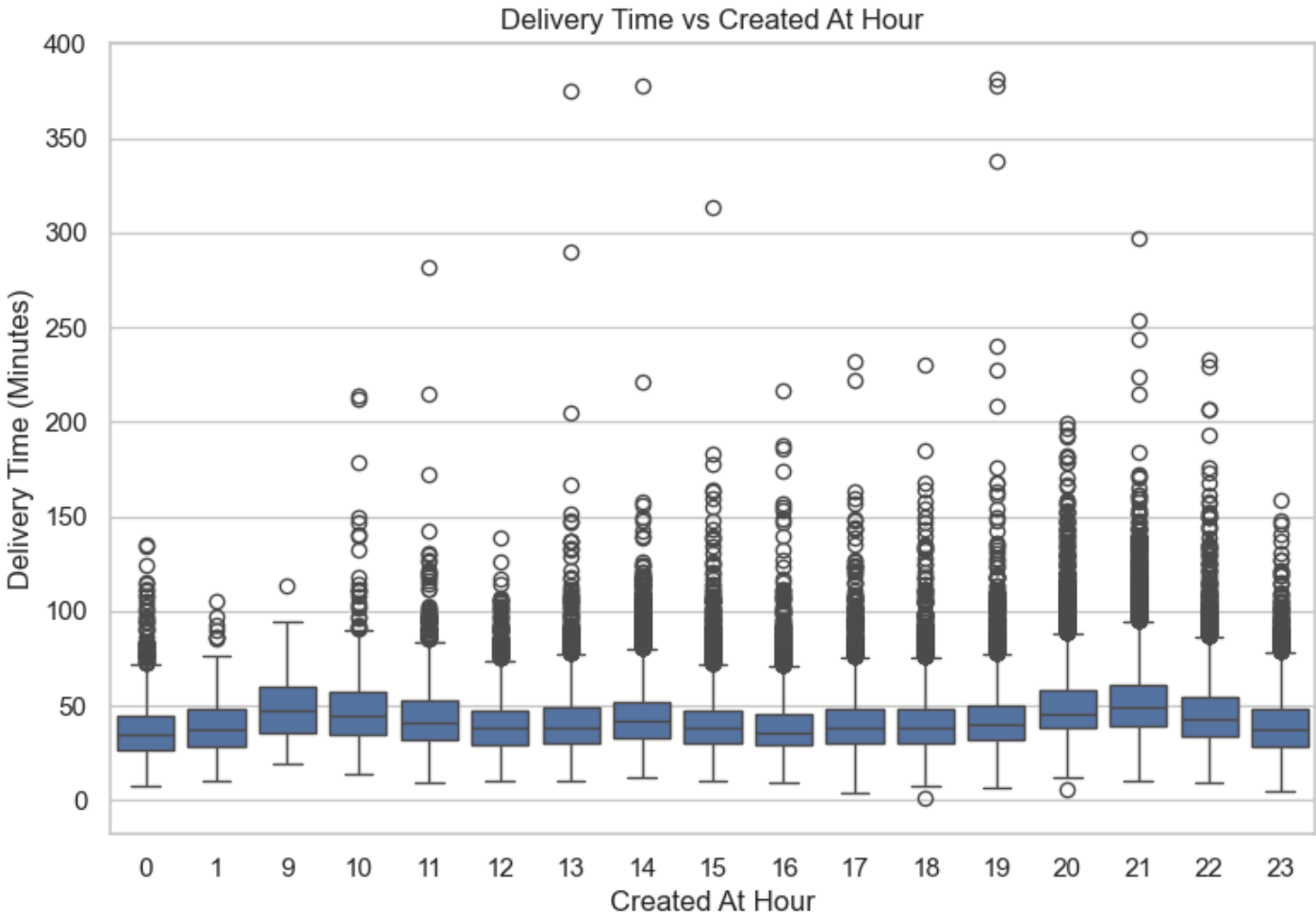
Observations

- We can see that the delivery time is less as the number of unique items increases


```
In [102... fig, ax = plt.subplots(1,2, figsize=(20, 6))

sns.scatterplot(data=t_df, x='created_at_hour', y='delivery_time_minutes', ax=ax[0]);
ax[0].set_title("Delivery Time vs Created At Hour");
ax[0].set_xlabel("Created At Hour")
ax[0].set_ylabel("Delivery Time (Minutes)")

sns.boxplot(data=t_df, x='created_at_hour', y='delivery_time_minutes', ax=ax[1]);
ax[1].set_title("Delivery Time vs Created At Hour");
ax[1].set_xlabel("Created At Hour");
ax[1].set_ylabel("Delivery Time (Minutes)");
```



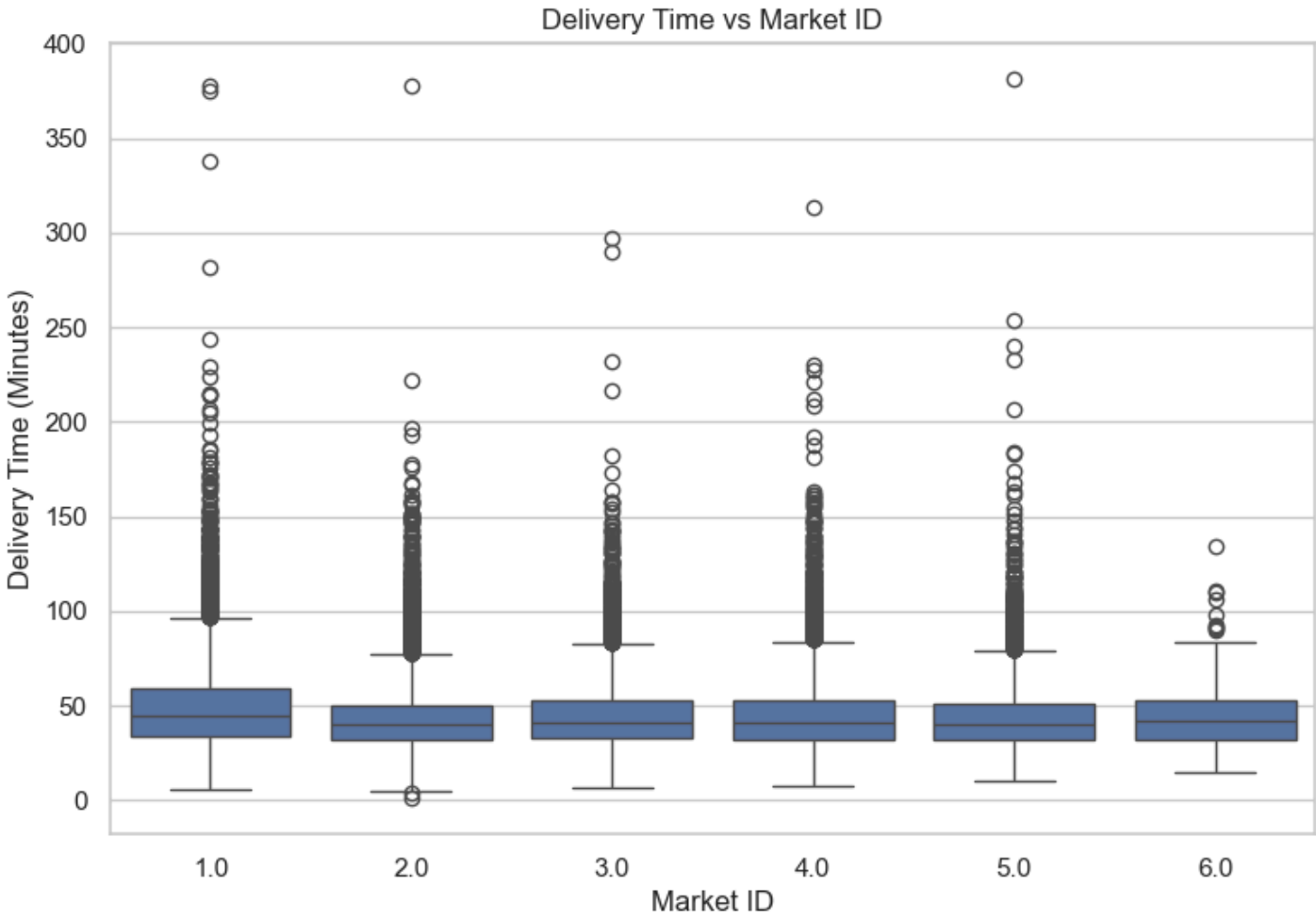
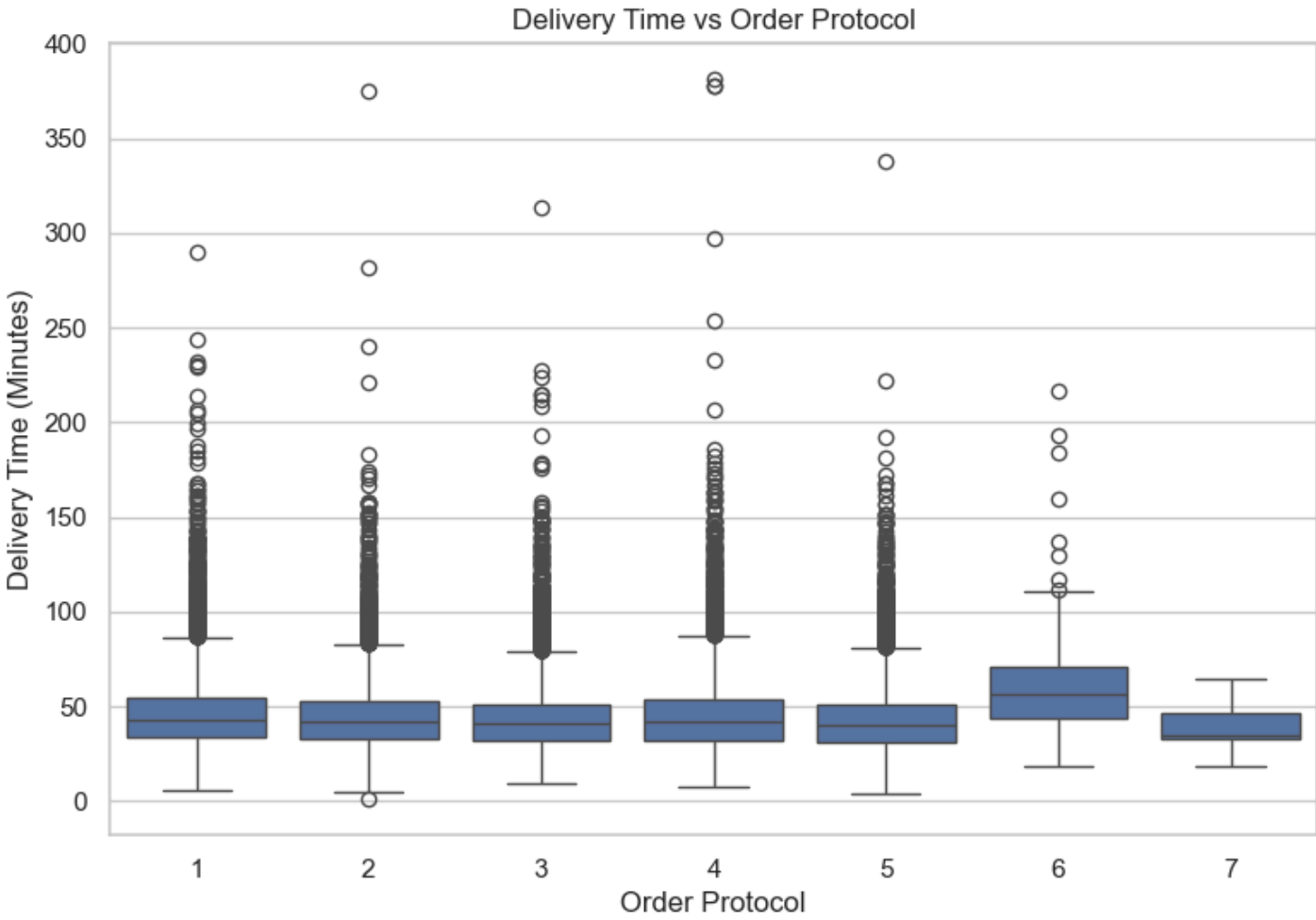
Observations

- We can see that delivery time increases during peak hours

```
In [103... fig, ax = plt.subplots(1,2, figsize=(20, 6))

sns.boxplot(data=t_df, x='order_protocol', y='delivery_time_minutes', ax=ax[0]);
ax[0].set_title("Delivery Time vs Order Protocol");
ax[0].set_xlabel("Order Protocol");
ax[0].set_ylabel("Delivery Time (Minutes)");

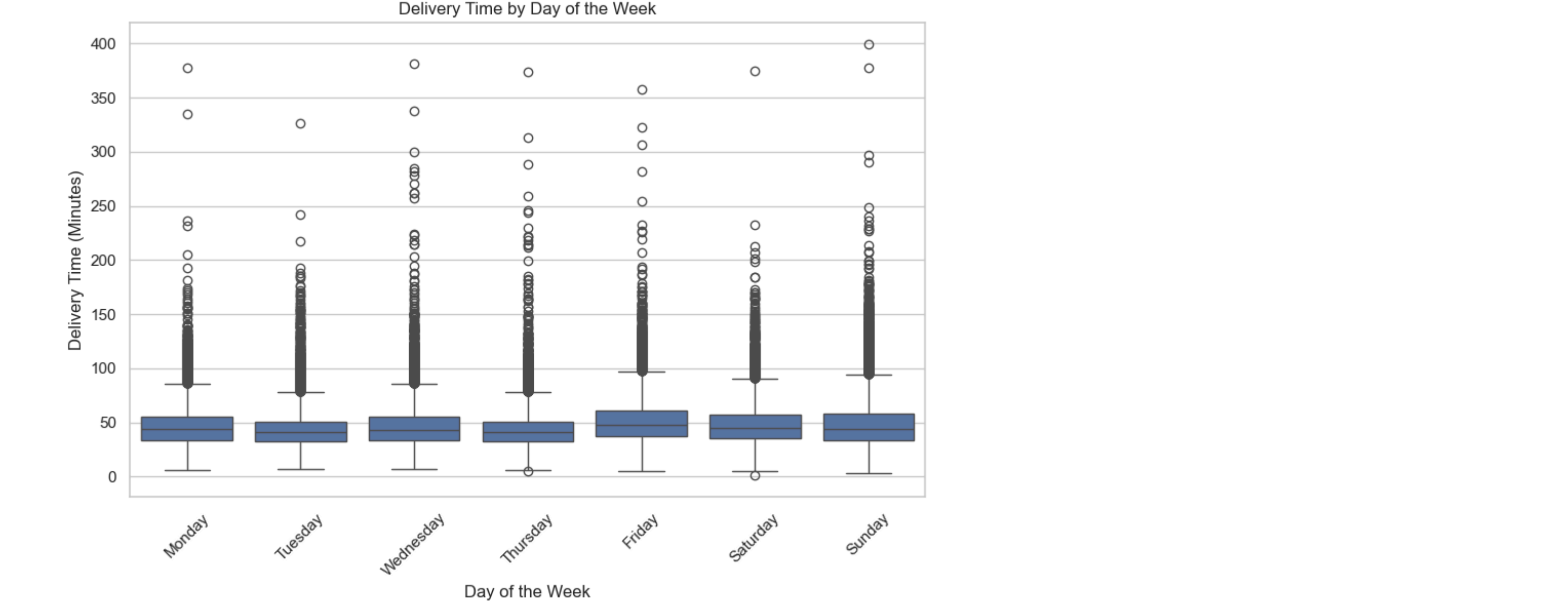
sns.boxplot(data=t_df, x='market_id', y='delivery_time_minutes', ax=ax[1]);
ax[1].set_title("Delivery Time vs Market ID");
ax[1].set_xlabel("Market ID");
ax[1].set_ylabel("Delivery Time (Minutes)");
```



Observations

- Order protocol 6 had a slightly higher delivery time

```
In [10]: plt.figure(figsize=(10, 6))
sns.boxplot(data=t_df, x="created_at_day_of_week", y="delivery_time_minutes")
plt.title("Delivery Time by Day of the Week")
plt.xlabel("Day of the Week")
plt.ylabel("Delivery Time (Minutes)")
day_names = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
tick_positions = plt.xticks()[0]
plt.xticks(tick_positions, day_names, rotation=45)
plt.show()
```

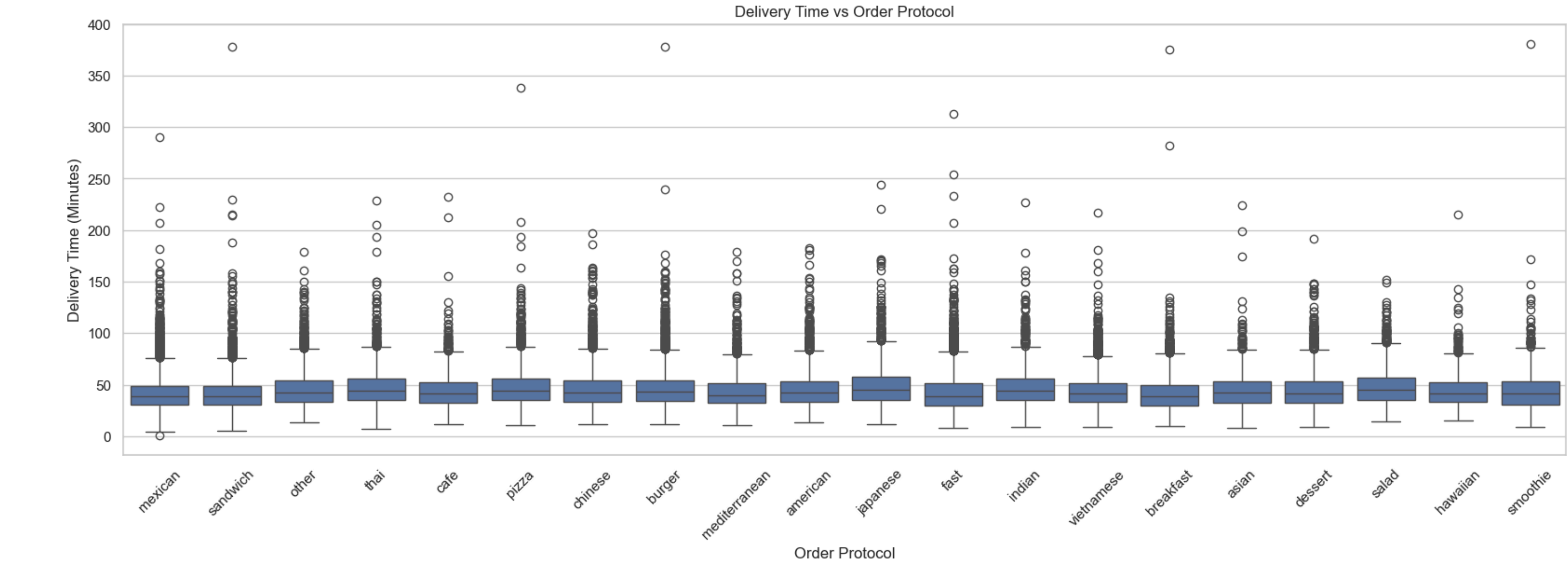


```
In [104... top_cat=t_df["store_primary_category"].value_counts().head(20).index
top_cat
t_df[t_df["store_primary_category"].isin(top_cat)]
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	...	actual_delivery_time
19	1.0	2015-01-30 20:51:23	2015-01-30 21:41:42	ce016f59ecc2366a43e1c96a4774d167	mexican	1	2	950	2	150.0	...	
20	1.0	2015-01-31 18:15:12	2015-01-31 18:44:05	ce016f59ecc2366a43e1c96a4774d167	mexican	1	5	1285	3	150.0	...	
31	3.0	2015-01-30 14:19:30	2015-01-30 14:58:45	50905d7b2216bfeccb5b41016357176b	sandwich	2	4	3490	3	175.0	...	
32	1.0	2015-01-28 15:03:04	2015-01-28 15:34:14	50905d7b2216bfeccb5b41016357176b	other	2	3	1765	3	275.0	...	
33	1.0	2015-02-12 14:57:42	2015-02-12 15:48:29	50905d7b2216bfeccb5b41016357176b	other	2	1	975	1	975.0	...	
...	...	...	...	...	...	...	...	...	...	...	...	
177539	1.0	2015-02-16 18:49:41	2015-02-16 19:54:48	a914ecef9c12ffdb9bede64bb703d877	fast	4	3	1389	3	345.0	...	
177540	1.0	2015-02-12 18:31:59	2015-02-12 19:28:22	a914ecef9c12ffdb9bede64bb703d877	fast	4	6	3010	4	405.0	...	
177541	1.0	2015-01-23 23:16:08	2015-01-24 00:06:16	a914ecef9c12ffdb9bede64bb703d877	fast	4	5	1836	3	300.0	...	
177542	1.0	2015-02-01 12:48:15	2015-02-01 13:53:22	c81e155d85dae5430a8cee6f2242e82c	sandwich	1	1	1175	1	535.0	...	
177543	1.0	2015-02-08 13:54:33	2015-02-08 14:31:41	c81e155d85dae5430a8cee6f2242e82c	sandwich	1	4	2605	4	425.0	...	

70721 rows x 39 columns

```
In [ ]: plt.figure(figsize=(20, 6))
sns.boxplot(data=t_df[t_df["store_primary_category"].isin(top_cat)], x='store_primary_category', y='delivery_time_minutes')
plt.title("Delivery Time vs Store primary category")
plt.xlabel("Store primary category")
plt.ylabel("Delivery Time (Minutes)")
plt.xticks(rotation=45);
```



Observations

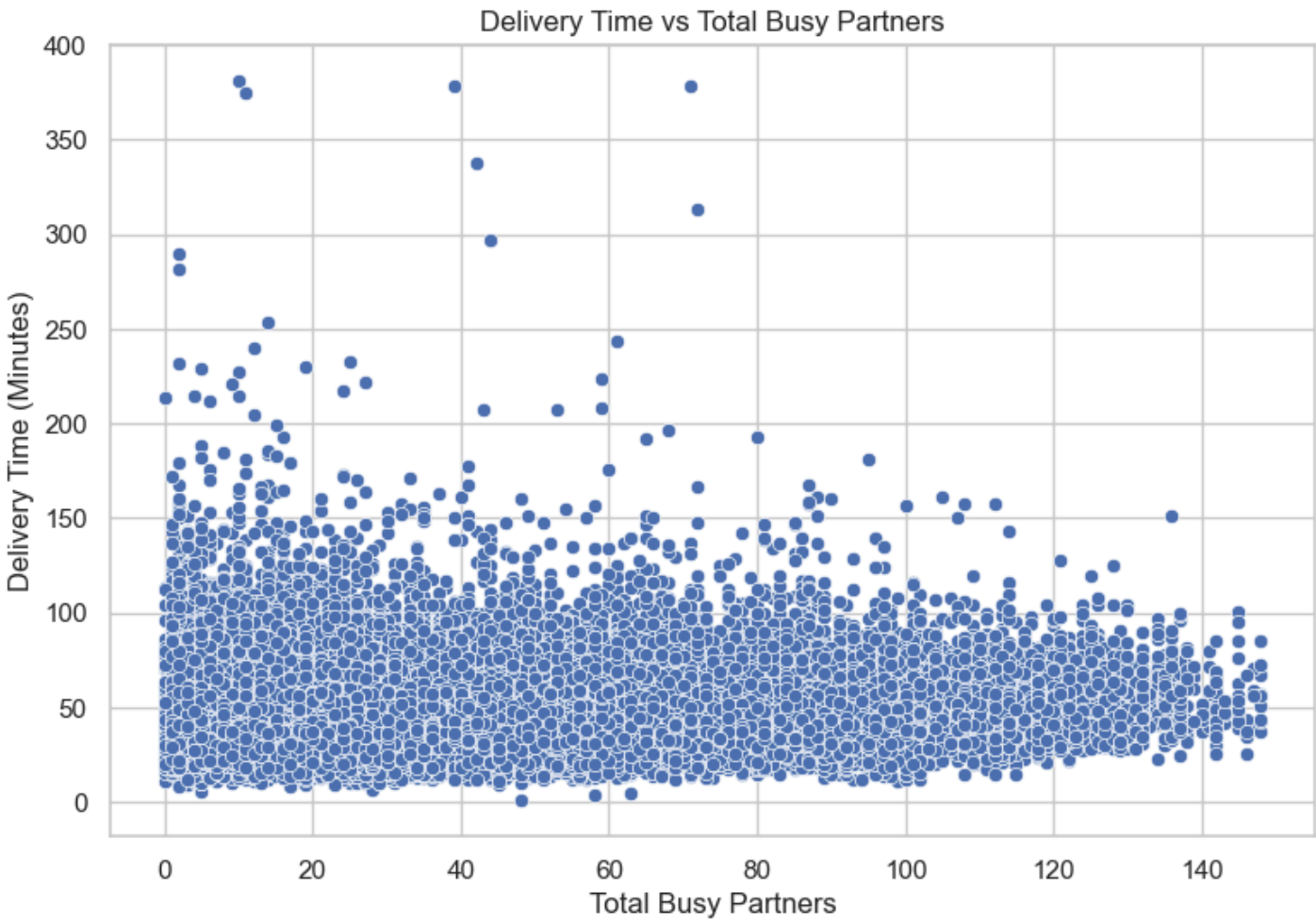
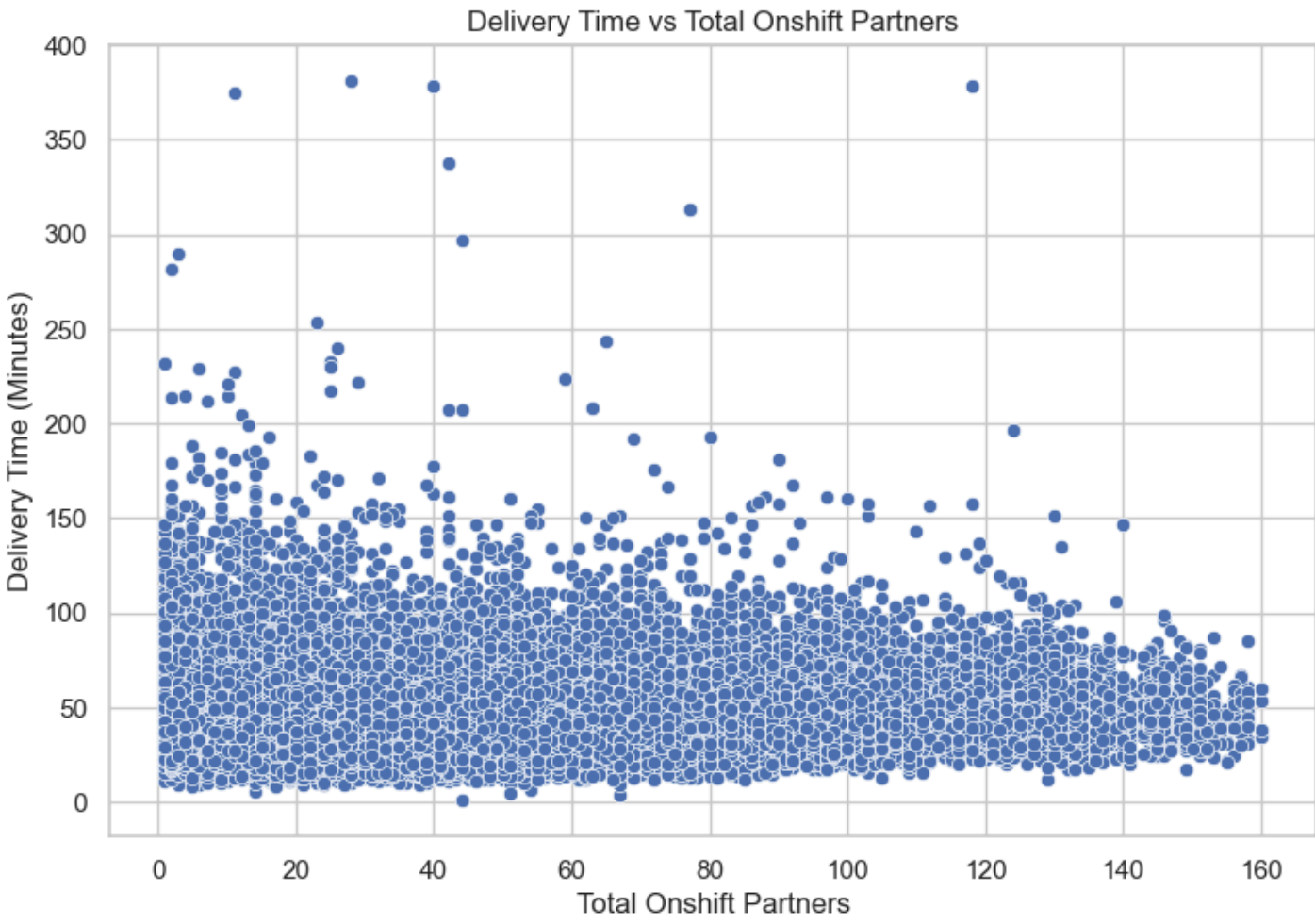
- We can see that the delivery time is not affected by the Store category

```
In [106... fig, ax = plt.subplots(1,2, figsize=(20, 6))

sns.scatterplot(data=t_df, x='total_onshift_partners', y='delivery_time_minutes', ax=ax[0]);
ax[0].set_title("Delivery Time vs Total Onshift Partners");
ax[0].set_xlabel("Total Onshift Partners")
ax[0].set_ylabel("Delivery Time (Minutes)");
```



```
sns.scatterplot(data=t_df, x='total_busy_partners', y='delivery_time_minutes', ax=ax[1]);
ax[1].set_title("Delivery Time vs Total Busy Partners");
ax[1].set_xlabel("Total Busy Partners");
ax[1].set_ylabel("Delivery Time (Minutes)");
```



Observations

- Delivery time appears to be influenced by the total number of busy partners and the total number of on-shift partners.
- Further analysis is required to understand the relationship and trends.

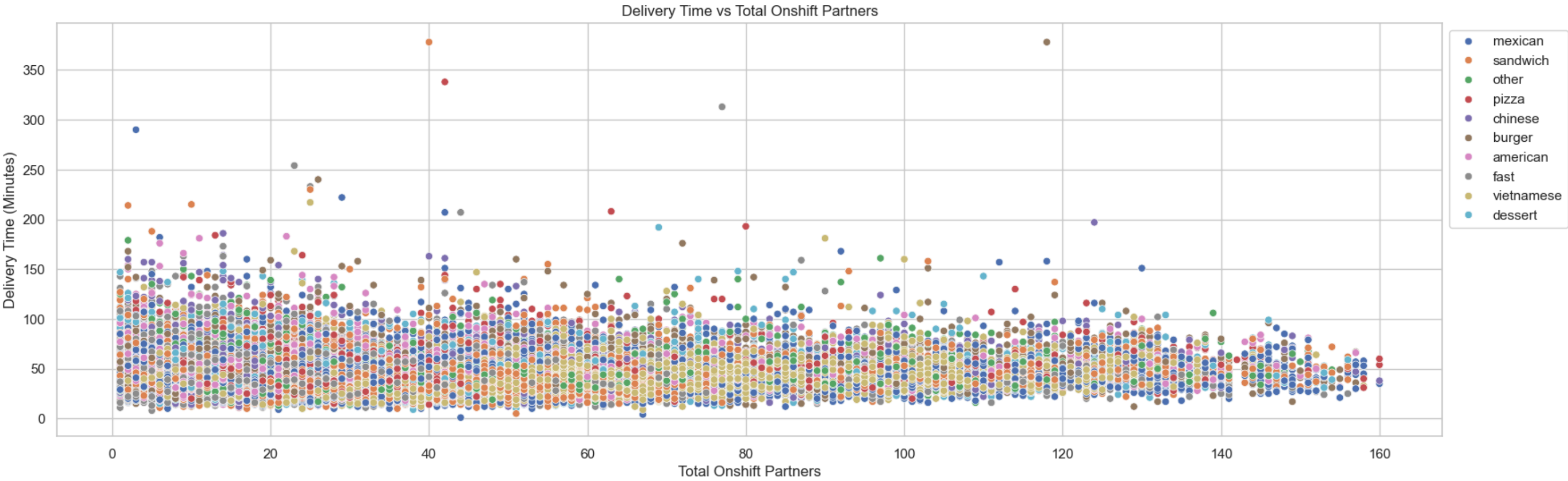
Multi-Variate Analysis

```
In [107...] top_cat=t_df["store_primary_category"].value_counts().head(10).index
top_cat
t_df[t_df["store_primary_category"].isin(top_cat)]
```

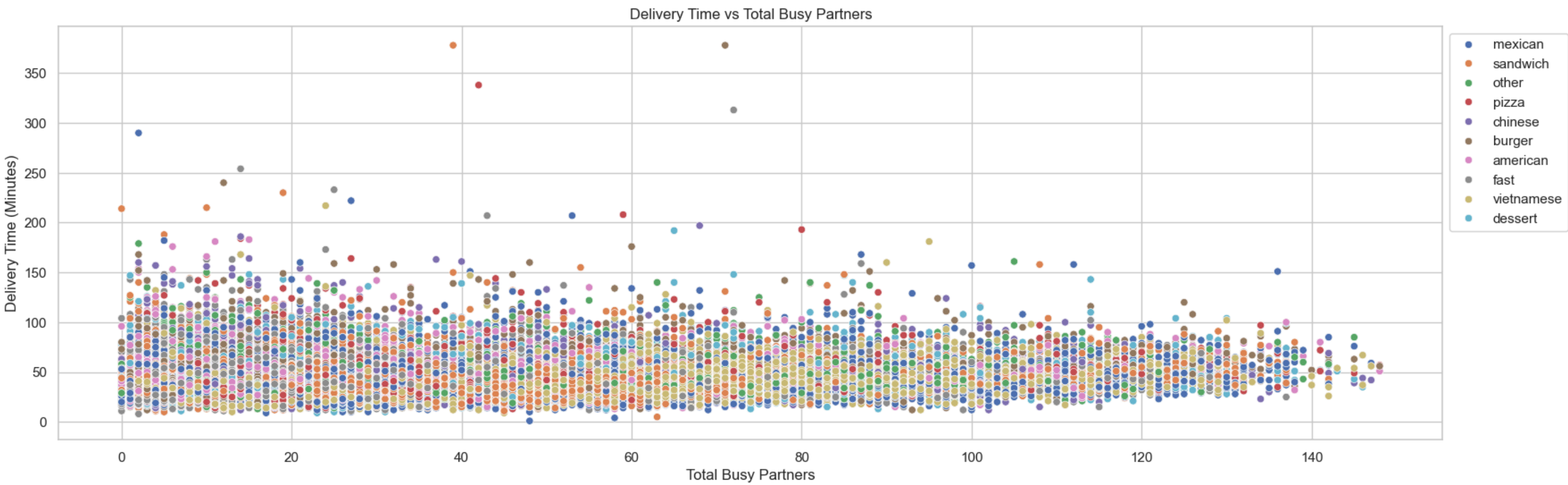
	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	...	actual_delivery_ti
	19	1.0	2015-01-30 20:51:23	2015-01-30 21:41:42	ce016f59ecc2366a43e1c96a4774d167	mexican	1	2	950	2	150.0	...
	20	1.0	2015-01-31 18:15:12	2015-01-31 18:44:05	ce016f59ecc2366a43e1c96a4774d167	mexican	1	5	1285	3	150.0	...
	31	3.0	2015-01-30 14:19:30	2015-01-30 14:58:45	50905d7b2216bfeccb5b41016357176b	sandwich	2	4	3490	3	175.0	...
	32	1.0	2015-01-28 15:03:04	2015-01-28 15:34:14	50905d7b2216bfeccb5b41016357176b	other	2	3	1765	3	275.0	...
	33	1.0	2015-02-12 14:57:42	2015-02-12 15:48:29	50905d7b2216bfeccb5b41016357176b	other	2	1	975	1	975.0	...
	...	...	...	...	...	...	...	...	...	...	...	...
	177539	1.0	2015-02-16 18:49:41	2015-02-16 19:54:48	a914ecef9c12ffdb9bede64bb703d877	fast	4	3	1389	3	345.0	...
	177540	1.0	2015-02-12 18:31:59	2015-02-12 19:28:22	a914ecef9c12ffdb9bede64bb703d877	fast	4	6	3010	4	405.0	...
	177541	1.0	2015-01-23 23:16:08	2015-01-24 00:06:16	a914ecef9c12ffdb9bede64bb703d877	fast	4	5	1836	3	300.0	...
	177542	1.0	2015-02-01 12:48:15	2015-02-01 13:53:22	c81e155d85dae5430a8cee6f2242e82c	sandwich	1	1	1175	1	535.0	...
	177543	1.0	2015-02-08 13:54:33	2015-02-08 14:31:41	c81e155d85dae5430a8cee6f2242e82c	sandwich	1	4	2605	4	425.0	...

52220 rows x 39 columns

```
In [108...] plt.figure(figsize=(20, 6))
sns.scatterplot(data=t_df[t_df["store_primary_category"].isin(top_cat)], x='total_onshift_partners', y='delivery_time_minutes', hue='store_primary_category');
plt.legend(bbox_to_anchor=(1, 1), loc='upper left')
plt.title("Delivery Time vs Total Onshift Partners");
plt.xlabel("Total Onshift Partners");
plt.ylabel("Delivery Time (Minutes)");
```



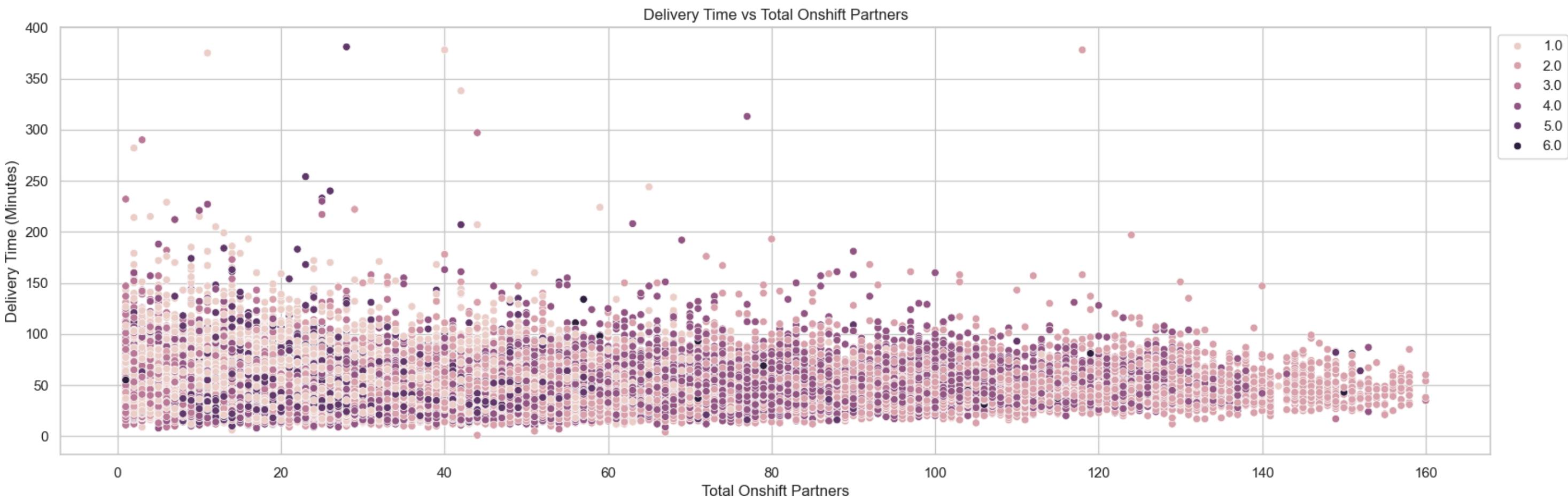
```
In [109...] plt.figure(figsize=(20, 6))
sns.scatterplot(data=t_df[t_df["store_primary_category"].isin(top_cat)], x='total_busy_partners', y='delivery_time_minutes', hue='store_primary_category');
plt.legend(bbox_to_anchor=(1, 1), loc='upper left')
plt.title("Delivery Time vs Total Busy Partners");
plt.xlabel("Total Busy Partners");
plt.ylabel("Delivery Time (Minutes)");
```

Observations

- From above plots we cannot see any correlation delivery time, busy delivery partners, and restaurant category

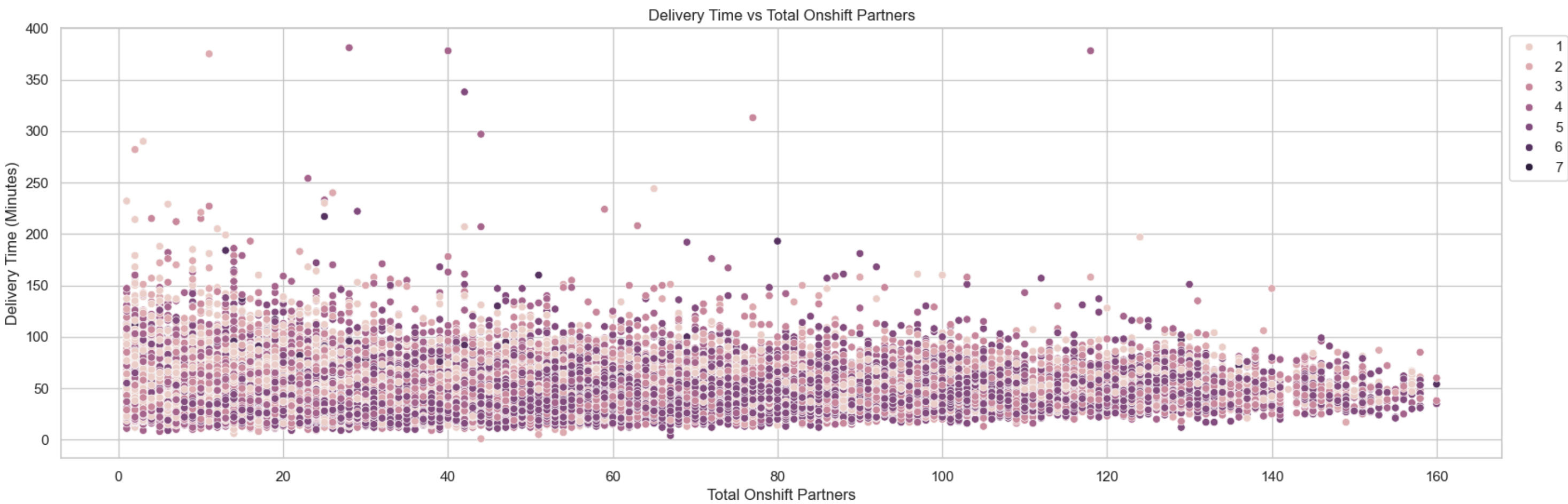
```
In [110]: plt.figure(figsize=(20, 6))
sns.scatterplot(data=t_df, x='total_onshift_partners', y='delivery_time_minutes', hue='market_id');
plt.legend(bbox_to_anchor=(1, 1), loc='upper left')
plt.title("Delivery Time vs Total Onshift Partners");
plt.xlabel("Total Onshift Partners")
plt.ylabel("Delivery Time (Minutes)");
```



Observations

- From above plots we cannot see any correlation delivery time, onshift delivery partners, and market id

```
In [111]: plt.figure(figsize=(20, 6))
sns.scatterplot(data=t_df, x='total_onshift_partners', y='delivery_time_minutes', hue='order_protocol');
plt.legend(bbox_to_anchor=(1, 1), loc='upper left')
plt.title("Delivery Time vs Total Onshift Partners");
plt.xlabel("Total Onshift Partners")
plt.ylabel("Delivery Time (Minutes)");
```



Observations

- From above plots we cannot see any correlation delivery time, available delivery partners, and order protocol

Data Preparation for Machine Learning and Deep Learning

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
import duckdb as db
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler, TargetEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
```

```
In [14]: df=pd.read_parquet("../data/cleaned/dataset.parquet")
```

```
In [15]: df.columns
```

```
Out[15]: Index(['market_id', 'created_at', 'actual_delivery_time', 'store_id',
               'store_primary_category', 'order_protocol', 'total_items', 'subtotal',
               'num_distinct_items', 'min_item_price', 'max_item_price',
               'total_onshift_partners', 'total_busy_partners',
               'total_outstanding_orders', 'created_at_year', 'created_at_month',
               'created_at_day', 'created_at_hour', 'created_at_minute',
               'created_at_second', 'actual_delivery_time_year',
               'actual_delivery_time_month', 'actual_delivery_time_day',
               'actual_delivery_time_hour', 'actual_delivery_time_minute',
               'actual_delivery_time_second', 'created_at_month_name',
               'actual_delivery_time_month_name', 'created_at_day_of_week',
               'actual_delivery_time_day_of_week', 'created_at_day_name',
               'actual_delivery_time_day_name', 'created_at_week_number',
               'actual_delivery_time_week_number', 'created_at_week_of_month',
               'actual_delivery_time_week_of_month', 'delivery_time_seconds',
               'delivery_time_minutes'],
              dtype='object')
```

```
In [16]: to_drop = [
    'created_at', 'actual_delivery_time', 'store_id',
    'created_at_year', 'created_at_month',
    'created_at_second', 'actual_delivery_time_year',
    'actual_delivery_time_month', 'actual_delivery_time_day',
    'actual_delivery_time_hour', 'actual_delivery_time_minute',
    'actual_delivery_time_second', 'created_at_month_name',
    'actual_delivery_time_month_name',
    'actual_delivery_time_day_of_week', 'created_at_day_name',
    'actual_delivery_time_day_name', 'created_at_week_number',
    'actual_delivery_time_week_number',
    'actual_delivery_time_week_of_month', 'delivery_time_seconds',
]
```

```
In [17]: df = df.drop(columns=to_drop)
df.columns
```

```
Out[17]: Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
               'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
               'total_onshift_partners', 'total_busy_partners',
               'total_outstanding_orders', 'created_at_day', 'created_at_hour',
               'created_at_minute', 'created_at_day_of_week',
               'created_at_week_of_month', 'delivery_time_minutes'],
              dtype='object')
```

```
In [18]: df=df.drop_duplicates()
```

```
In [19]: df.shape
```

```
Out[19]: (177544, 17)
```

```
In [20]: df=df[df["total_items"] < 100]
df=df[df["delivery_time_minutes"] < 200]
df=df[df["max_item_price"] < 10000]
df=df[df["min_item_price"] < 10000]
df=df[(df["subtotal"] < 15000) & (df["subtotal"] > 0)]
```

```
In [21]: df["is_weekend"] = np.where(df["created_at_day_of_week"] >= 5, 1, 0)
df["is_peak_hour"] = np.where(df["created_at_hour"].isin([14,15,20,21]), 1, 0)
```

```
In [22]: df.shape
```

```
Out[22]: (177285, 19)
```

```
In [23]: poly = PolynomialFeatures(degree=3, include_bias=False)
poly_features = poly.fit_transform(df[["total_onshift_partners", "total_busy_partners", "total_outstanding_orders"]])
poly_feature_names = poly.get_feature_names_out(["total_onshift_partners", "total_busy_partners", "total_outstanding_orders"])
poly_df = pd.DataFrame(poly_features, columns=poly_feature_names)
poly_df = poly_df.drop(columns=["total_onshift_partners", "total_busy_partners", "total_outstanding_orders"])

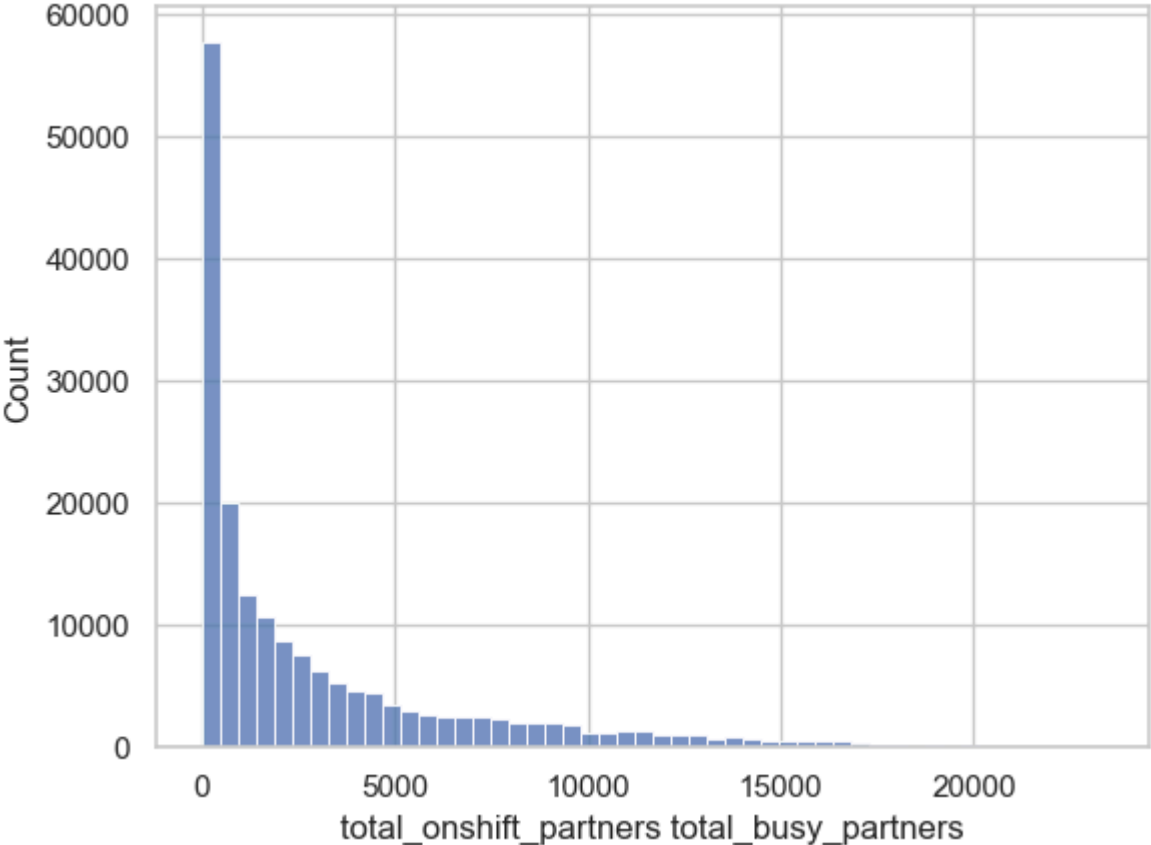
df = pd.concat([df.reset_index(drop=True), poly_df.reset_index(drop=True)], axis=1)
```

```
In [24]: df.shape, poly_df.shape
```

```
Out[24]: ((177285, 35), (177285, 16))
```

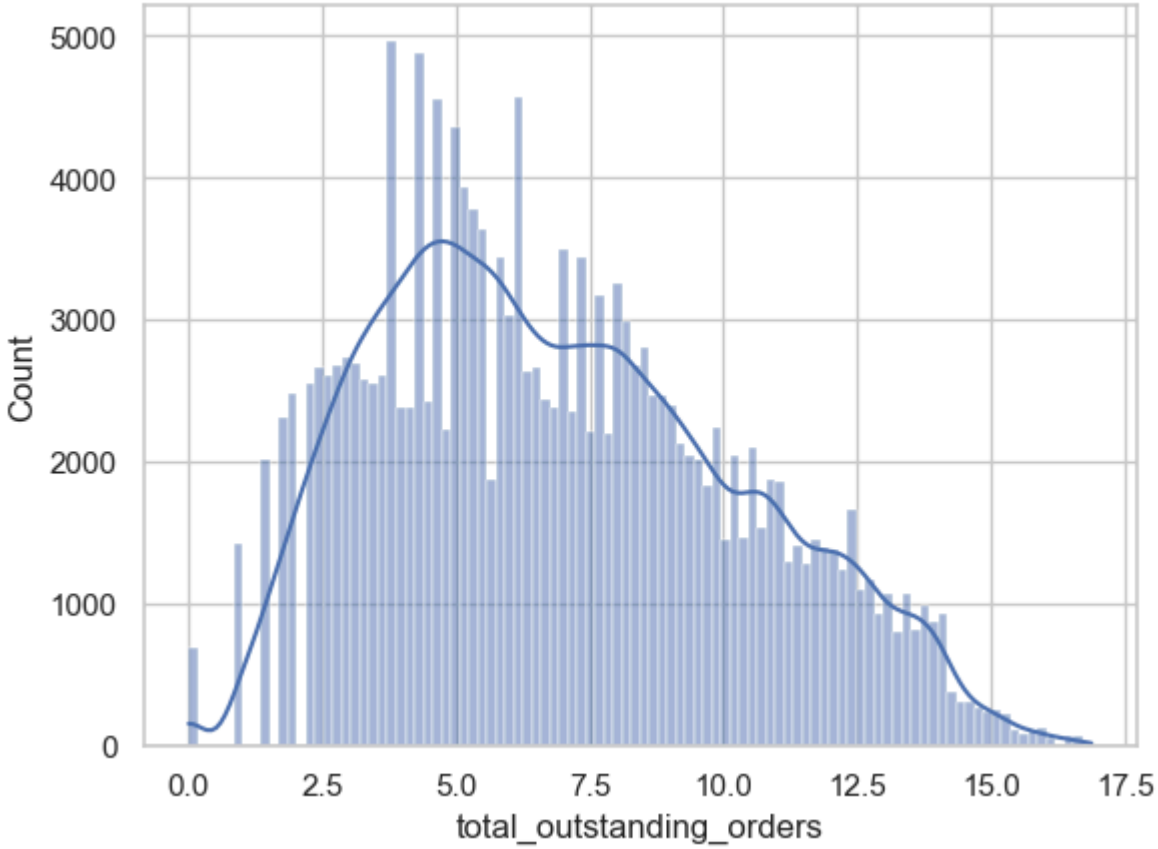
```
In [25]: sns.histplot(data=poly_df, x="total_onshift_partners total_busy_partners", bins=50)
```

```
Out[25]: <Axes: xlabel='total_onshift_partners total_busy_partners', ylabel='Count'>
```



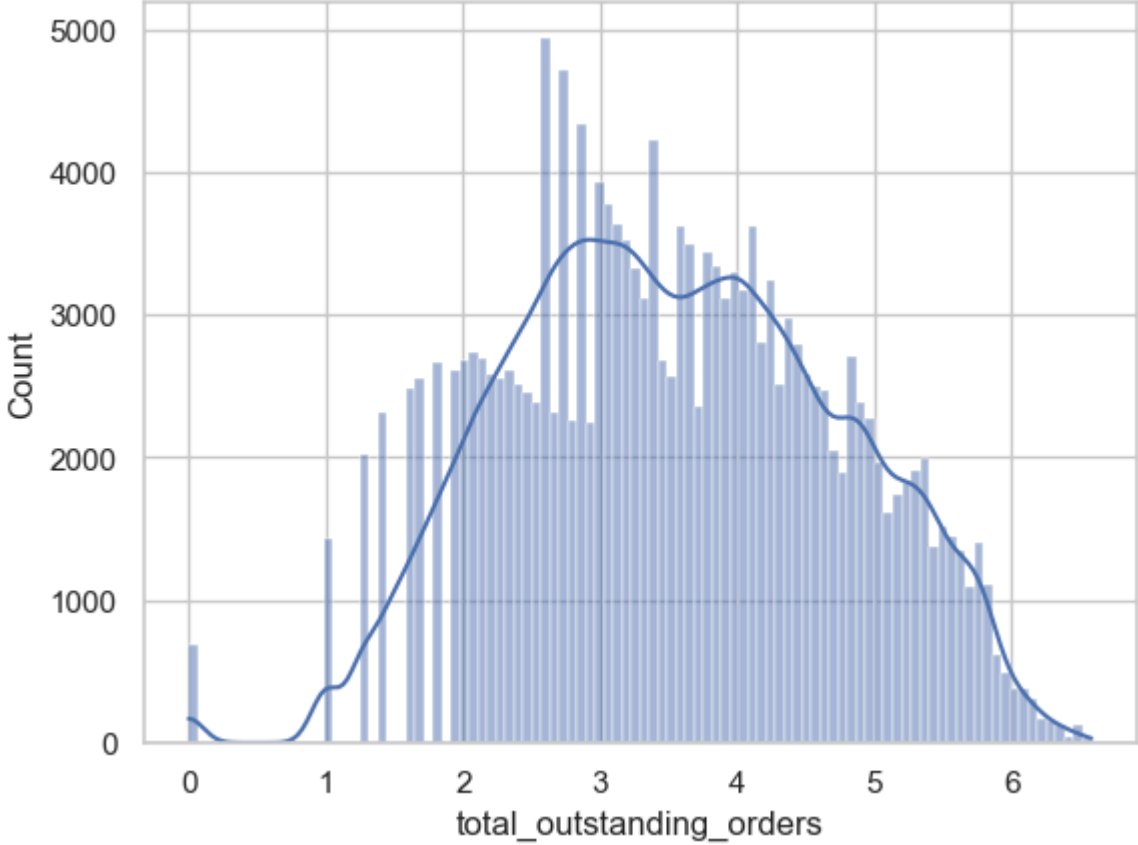
```
In [26]: # sns.histplot(np.log1p(df["delivery_time_minutes"]), bins=100, kde=True)
# cube root
sns.histplot(np.sqrt(df["total_outstanding_orders"]), bins=100, kde=True)
```

Out[26]: <Axes: xlabel='total_outstanding_orders', ylabel='Count'>



In [27]: `sns.histplot(np.cbrt(df["total_outstanding_orders"]), bins=100, kde=True)`

Out[27]: <Axes: xlabel='total_outstanding_orders', ylabel='Count'>

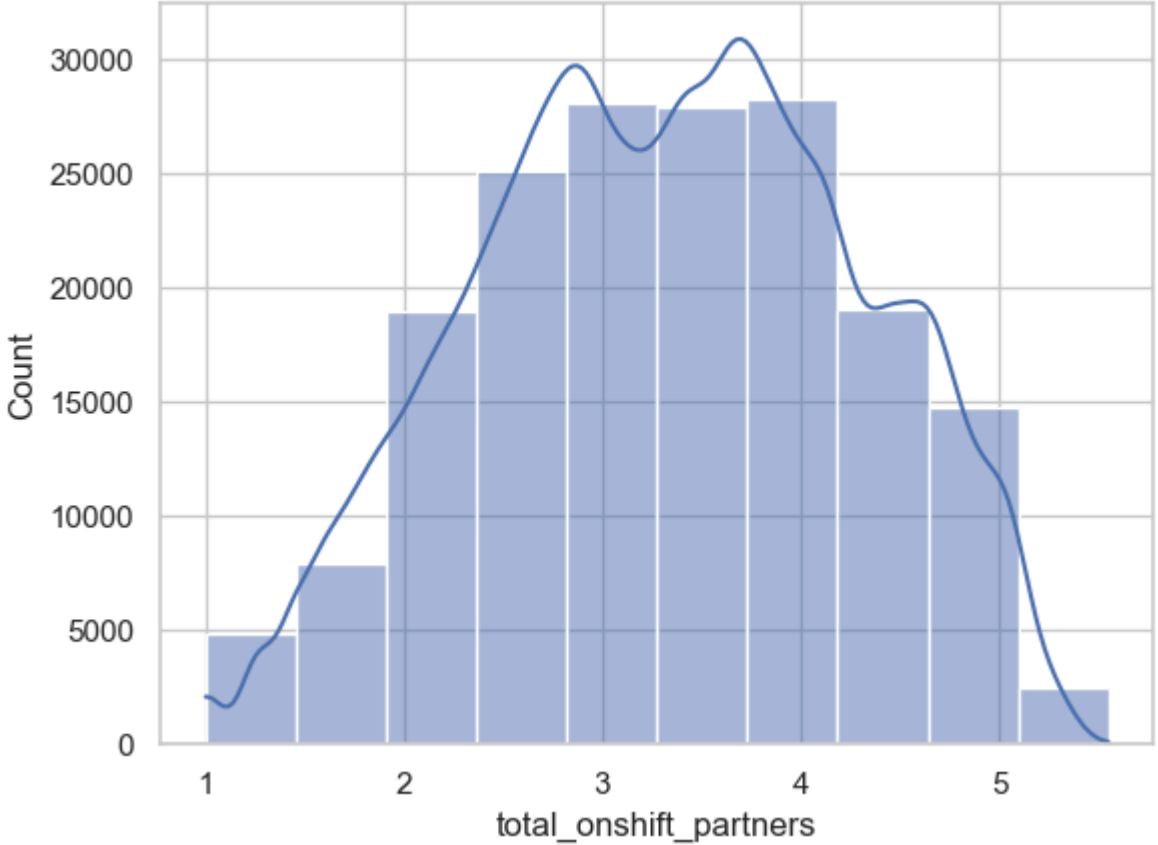


In [28]: `df["num_distinct_items"].min(), df["subtotal"].max()`

Out[28]: (1, 14955)

In [29]: `sns.histplot(x=np.cbrt(df["total_onshift_partners"]), bins=10, kde=True)`

Out[29]: <Axes: xlabel='total_onshift_partners', ylabel='Count'>



In [30]: `# df["total_items"] = np.log(df["total_items"])
df["max_item_price"] = np.cbrt(df["max_item_price"])
df["min_item_price"] = np.cbrt(df["min_item_price"])
df["subtotal"] = np.cbrt(df["subtotal"])
df["total_onshift_partners"] = np.cbrt(df["total_onshift_partners"])
df["total_busy_partners"] = np.cbrt(df["total_busy_partners"])
df["total_outstanding_orders"] = np.cbrt(df["total_outstanding_orders"])`

In [31]: `df.shape`

Out[31]: (177285, 35)

In [32]: `df`

Out[32]:

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	...	total_onshift_partners^3	1
0	1.0	american	1	4	3441	4	557.0	1239.0	33.0	14.0	...	35937.0	
1	2.0	mexican	2	1	1900	1	1400.0	1400.0	1.0	2.0	...	1.0	
2	3.0	other	1	1	1900	1	1900.0	1900.0	1.0	0.0	...	1.0	
3	3.0	other	1	6	6900	5	600.0	1800.0	1.0	1.0	...	1.0	
4	3.0	other	1	3	3900	3	1100.0	1600.0	6.0	6.0	...	216.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
177280	1.0	fast	4	3	1389	3	345.0	649.0	17.0	17.0	...	4913.0	
177281	1.0	fast	4	6	3010	4	405.0	825.0	12.0	11.0	...	1728.0	
177282	1.0	fast	4	5	1836	3	300.0	399.0	39.0	41.0	...	59319.0	
177283	1.0	sandwich	1	1	1175	1	535.0	535.0	7.0	7.0	...	343.0	
177284	1.0	sandwich	1	4	2605	4	425.0	750.0	20.0	20.0	...	8000.0	

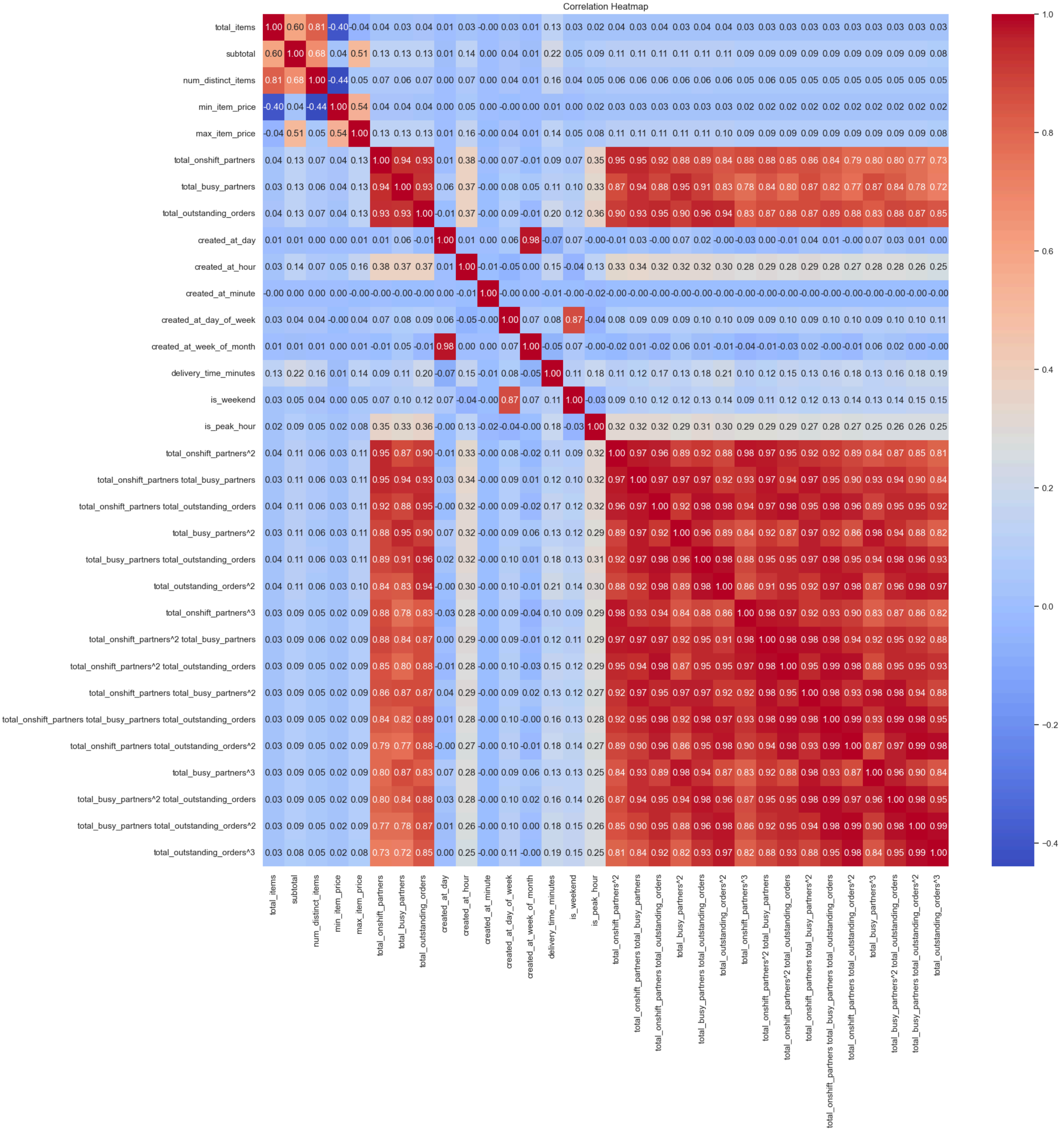
177285 rows × 35 columns

In [33]:

```
plt.figure(figsize=(20, 20))
sns.heatmap(df.drop(['market_id', 'store_primary_category', 'order_protocol'], axis=1).corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap")
```

Out[33]:

Text(0.5, 1.0, 'Correlation Heatmap')



In [34]:

```
df.columns
```



```
Out[34]: Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
              'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
              'total_onshift_partners', 'total_busy_partners',
              'total_outstanding_orders', 'created_at_day', 'created_at_hour',
              'created_at_minute', 'created_at_day_of_week',
              'created_at_week_of_month', 'delivery_time_minutes', 'is_weekend',
              'is_peak_hour', 'total_onshift_partners^2',
              'total_onshift_partners total_busy_partners',
              'total_onshift_partners total_outstanding_orders',
              'total_busy_partners^2', 'total_busy_partners total_outstanding_orders',
              'total_outstanding_orders^2', 'total_onshift_partners^3',
              'total_onshift_partners^2 total_busy_partners',
              'total_onshift_partners^2 total_outstanding_orders',
              'total_onshift_partners total_busy_partners^2',
              'total_onshift_partners total_busy_partners total_outstanding_orders',
              'total_onshift_partners total_outstanding_orders^2',
              'total_busy_partners^3',
              'total_busy_partners^2 total_outstanding_orders',
              'total_busy_partners total_outstanding_orders^2',
              'total_outstanding_orders^3'],
              dtype='object')
```

```
In [35]: df[["created_at_day", "created_at_week_of_month"]].sample(20)
```

Out[35]:

	created_at_day	created_at_week_of_month
65721	25	4
90122	12	2
146653	17	3
139955	25	4
160201	5	1
54085	15	3
73879	1	1
46481	2	1
175668	8	2
16242	22	4
49207	5	1
144666	28	4
71066	11	2
66843	13	2
173801	22	4
70359	30	5
173948	11	2
87911	8	2
126283	5	1
106092	17	3

```
In [36]: cat_df = df["store_primary_category"].value_counts().reset_index().rename(columns={ 'store_primary_category': 'category'})
cat_df
```

Out[36]:

	category	count
0	american	18076
1	pizza	15684
2	mexican	15599
3	burger	9833
4	sandwich	8984
...	...	...
68	african	10
69	lebanese	9
70	belgian	2
71	chocolate	1
72	alcohol-plus-food	1

73 rows × 2 columns

```
In [37]: df=df.merge(cat_df, left_on="store_primary_category", right_on="category", how="left")
```

```
In [38]: df["store_primary_category"] = np.where(df["count"] < 20, "other", df["store_primary_category"])
```

```
In [39]: df=df.drop(columns=["category", "count"])
```

```
In [40]: df.columns
```

Out[40]: Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
 'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
 'total_onshift_partners', 'total_busy_partners',
 'total_outstanding_orders', 'created_at_day', 'created_at_hour',
 'created_at_minute', 'created_at_day_of_week',
 'created_at_week_of_month', 'delivery_time_minutes', 'is_weekend',
 'is_peak_hour', 'total_onshift_partners^2',
 'total_onshift_partners total_busy_partners',
 'total_onshift_partners total_outstanding_orders',
 'total_busy_partners^2', 'total_busy_partners total_outstanding_orders',
 'total_outstanding_orders^2', 'total_onshift_partners^3',
 'total_onshift_partners^2 total_busy_partners',
 'total_onshift_partners^2 total_outstanding_orders',
 'total_onshift_partners total_busy_partners^2',
 'total_onshift_partners total_busy_partners total_outstanding_orders',
 'total_onshift_partners total_outstanding_orders^2',
 'total_busy_partners^3',
 'total_busy_partners^2 total_outstanding_orders',
 'total_busy_partners total_outstanding_orders^2',
 'total_outstanding_orders^3'],
 dtype='object')

```
In [41]: all_cols=['market_id', 'store_primary_category', 'order_protocol', 'total_items',
                  'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
                  'total_onshift_partners', 'total_busy_partners',
                  'total_outstanding_orders', 'created_at_day', 'created_at_hour',
                  'created_at_minute', 'created_at_day_of_week',
                  'created_at_week_of_month', 'delivery_time_minutes', 'is_weekend',
                  'is_peak_hour', 'total_onshift_partners^2',
                  'total_onshift_partners total_busy_partners',
                  'total_onshift_partners total_outstanding_orders',
                  'total_busy_partners^2', 'total_busy_partners total_outstanding_orders',
                  'total_outstanding_orders^2', 'total_onshift_partners^3',
                  'total_onshift_partners^2 total_busy_partners',
```

```

    'total_onshift_partners^2 total_outstanding_orders',
    'total_onshift_partners total_busy_partners^2',
    'total_onshift_partners total_busy_partners total_outstanding_orders',
    'total_onshift_partners total_outstanding_orders^2',
    'total_busy_partners^3',
    'total_busy_partners^2 total_outstanding_orders',
    'total_busy_partners total_outstanding_orders^2',
    'total_outstanding_orders^3']

date_time_cols = [ 'created_at_hour', 'created_at_minute',
                    'created_at_day_of_week'
                  ]

def convert_cyclical(df, col, max_val):
    df[col + '_sin'] = np.sin(2 * np.pi * df[col] / max_val).round(8)
    df[col + '_cos'] = np.cos(2 * np.pi * df[col] / max_val).round(8)
    return df
```

```
In [42]: for col in date_time_cols:
        df = convert_cyclical(df, col, df[col].max())

df = df.drop(columns=date_time_cols)
```

```
In [43]: category_label_encoder = LabelEncoder()
df['store_primary_category_le'] = category_label_encoder.fit_transform(df['store_primary_category'])
```

```
In [44]: order_protocol_onehot_encoder = OneHotEncoder(sparse_output=False)
order_protocol_onehot = order_protocol_onehot_encoder.fit_transform(df[['order_protocol']])
order_protocol_onehot_df = pd.DataFrame(order_protocol_onehot, columns=order_protocol_onehot_encoder.get_feature_names_out(['order_protocol']))
df = pd.concat([df, order_protocol_onehot_df], axis=1)
df = df.drop(columns=['order_protocol'])

market_id_onehot_encoder = OneHotEncoder(sparse_output=False)
market_id_onehot = market_id_onehot_encoder.fit_transform(df[['market_id']])
market_id_onehot_df = pd.DataFrame(market_id_onehot, columns=market_id_onehot_encoder.get_feature_names_out(['market_id']))
df = pd.concat([df, market_id_onehot_df], axis=1)
df = df.drop(columns=['market_id'])
```

```
In [45]: df
```

Out[45]:

	store_primary_category	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	...	order_prot
0	american	4	3441	4	557.0	1239.0	33.0	14.0	21.0	6	...	
1	mexican	1	1900	1	1400.0	1400.0	1.0	2.0	2.0	10	...	
2	other	1	1900	1	1900.0	1900.0	1.0	0.0	0.0	22	...	
3	other	6	6900	5	600.0	1800.0	1.0	1.0	2.0	3	...	
4	other	3	3900	3	1100.0	1600.0	6.0	6.0	9.0	14	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
177280	fast	3	1389	3	345.0	649.0	17.0	17.0	23.0	16	...	
177281	fast	6	3010	4	405.0	825.0	12.0	11.0	14.0	12	...	
177282	fast	5	1836	3	300.0	399.0	39.0	41.0	40.0	23	...	
177283	sandwich	1	1175	1	535.0	535.0	7.0	7.0	12.0	1	...	
177284	sandwich	4	2605	4	425.0	750.0	20.0	20.0	23.0	8	...	

177285 rows × 50 columns

```
In [46]: X=df.drop(columns=['delivery_time_minutes'])
y=np.cbrt(df["delivery_time_minutes"])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5, random_state=42)
X_train.shape, X_val.shape, X_test.shape, y_train.shape, y_val.shape, y_test.shape
```

```
Out[46]: ((124099, 49), (26593, 49), (26593, 49), (124099,), (26593,), (26593,))
```

```
In [47]: y_train.to_frame().isna().sum()
```

```
Out[47]: delivery_time_minutes    0
dtype: int64
```

```
In [48]: category_target_encoder = TargetEncoder(target_type="continuous")
X_train["store_primary_category"]=category_target_encoder.fit_transform(X_train[['store_primary_category']], y_train)
X_val["store_primary_category"]=category_target_encoder.transform(X_val[['store_primary_category']])
X_test["store_primary_category"]=category_target_encoder.transform(X_test[['store_primary_category']])
```

```
In [49]: X_train_store_category_le = X_train.pop('store_primary_category_le')
X_val_store_category_le = X_val.pop('store_primary_category_le')
X_test_store_category_le = X_test.pop('store_primary_category_le')
```

```
In [50]: X_train.shape
```

```
Out[50]: (124099, 48)
```

```
In [51]: y.shape
```

```
Out[51]: (177285,)
```

```
In [52]: cols=X_train.columns
```

```
In [53]: sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_val = sc.transform(X_val)
X_test = sc.transform(X_test)
```

```
In [54]: sc_y = StandardScaler()
y_train = sc_y.fit_transform(y_train.values.reshape(-1, 1))
y_val = sc_y.transform(y_val.values.reshape(-1, 1))
y_test = sc_y.transform(y_test.values.reshape(-1, 1))
```

```
In [55]: X_train.shape
```

```
Out[55]: (124099, 48)
```

```
In [56]: pd.Series(y_train.reshape(-1))
```

```
Out[56]: 0      -0.723537
1      -0.510422
2      -1.108517
3       0.410726
4       1.026274
...
124094  -1.028169
124095   0.830366
124096   1.568953
124097  -1.449784
124098  -0.723537
Length: 124099, dtype: float64
```

```
In [57]: X_train = pd.DataFrame(X_train, columns=cols)
X_val = pd.DataFrame(X_val, columns=cols)
X_test = pd.DataFrame(X_test, columns=cols)
```

```
y_train = pd.Series(y_train.reshape(-1), name="delivery_time_minutes")
y_val = pd.Series(y_val.reshape(-1), name="delivery_time_minutes")
y_test = pd.Series(y_test.reshape(-1), name="delivery_time_minutes")
```

In [58]: `import pickle`

```
with open('../data/processed/dataset.pkl', 'wb') as f:
    pickle.dump({'X_train': X_train,
                'y_train': y_train,
                'X_val': X_val,
                'y_val': y_val,
                'X_test': X_test,
                'y_test': y_test,
                'X_train_store_category_le': X_train_store_category_le,
                'X_val_store_category_le': X_val_store_category_le,
                'X_test_store_category_le': X_test_store_category_le,
                "y_scaler": sc_y,
                "X_scaler": sc,
                },
              f)
```

In [59]: `with open('../data/processed/dataset.pkl', 'rb') as f:`
`dataset = pickle.load(f)`

In [60]: `dataset.keys()`

Out[60]: dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])

In [61]: `df.to_parquet("../data/processed/dataset.parquet", index=False)`

In [62]: `X_train`

	store_primary_category	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	...	order_prc
0	0.139958	-0.887847	-0.774580	-1.034172	1.175252	0.233244	0.356917	-0.239323	-0.063258	-1.134736	...	-0
1	-0.294285	0.323576	0.936110	0.203869	-0.179647	0.466436	-0.898395	-0.866322	-0.691474	0.050511	...	-0
2	0.082312	0.727384	0.600475	0.822890	-0.664373	0.053865	-0.985975	-1.023072	-0.938953	0.912508	...	-0
3	-1.741454	-0.080232	-1.258467	0.203869	-1.151047	-1.817052	-0.927588	-1.023072	-0.938953	0.697009	...	3
4	0.719151	1.938807	1.653122	2.060932	-1.053712	-0.295923	1.115942	0.857927	1.078952	1.666756	...	-0
...	...	...	...	...	...	...	...	...	...	...	...	...
124094	-1.667240	-0.080232	-0.288488	0.203869	-0.179647	-0.835852	-0.256143	-0.803622	-0.653400	1.451257	...	-0
124095	0.304998	-0.484040	-0.667111	-0.415151	0.007236	-0.663649	1.641422	1.798427	2.620935	1.666756	...	-0
124096	0.346322	0.727384	1.802477	1.441911	-0.664373	1.348978	0.619656	0.701177	0.679178	1.235757	...	-0
124097	-1.359593	-0.080232	-0.443905	0.203869	-0.576772	-0.609835	-0.139369	-0.490123	-0.577253	0.589259	...	-0
124098	-1.285614	-0.484040	-0.309982	-0.415151	0.735299	-0.172152	-1.190328	-1.148472	-0.977026	1.235757	...	-0

124099 rows × 48 columns

In [63]: `y_train`

Out[63]:

0	-0.723537
1	-0.510422
2	-1.108517
3	0.410726
4	1.026274
...	
124094	-1.028169
124095	0.830366
124096	1.568953
124097	-1.449784
124098	-0.723537

Name: delivery_time_minutes, Length: 124099, dtype: float64

Baseline ML Model

```
In [1]: import pickle
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

from sklearn.linear_model import LinearRegression, ElasticNet, LassoCV, RidgeCV
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from lightgbm import LGBMRegressor

import optuna

In [2]: with open('../data/processed/dataset.pkl', 'rb') as f:
dataset = pickle.load(f)

In [3]: dataset.keys()

Out[3]: dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])

In [4]: X_train = dataset['X_train']
X_test = dataset['X_test']
X_val = dataset['X_val']
y_train = dataset['y_train']
y_test = dataset['y_test']
y_val = dataset['y_val']

In [5]: lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_val)
print("Linear Regression")
print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
print("R2 Score:", r2_score(y_val, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))

Linear Regression
Mean Squared Error: 0.7238494476388447
R2 Score: 0.2747745696494136
Mean Absolute Error: 0.6608437771781829

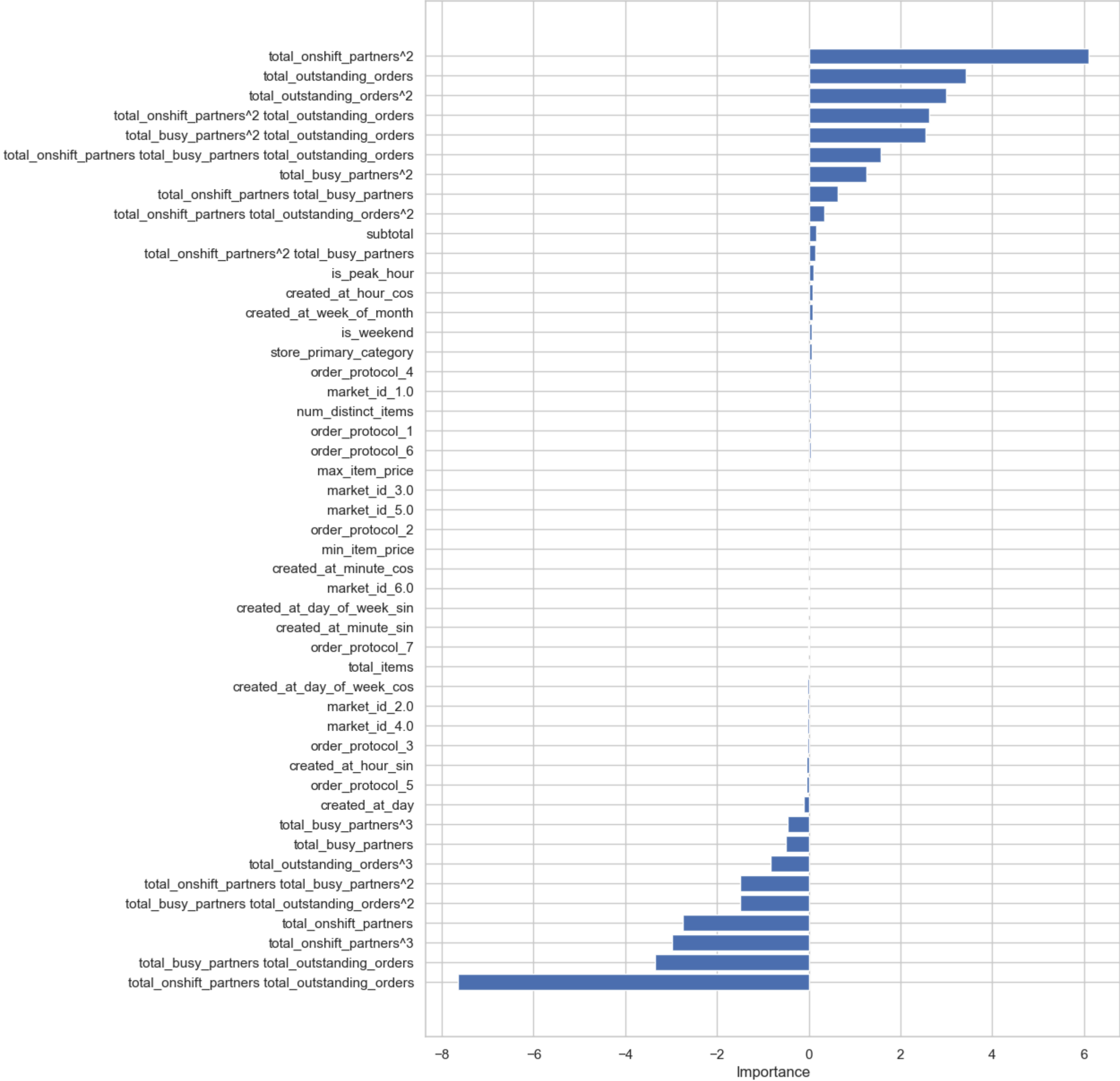
In [6]: C=[0.1, 1, 10]

for c in C:
    lasso = LassoCV(alphas=[c], cv=5)
    lasso.fit(X_train, y_train)
    y_pred = lasso.predict(X_val)
    print(f"Lasso Regression (alpha={c})")
    print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
    print("R2 Score:", r2_score(y_val, y_pred))
    print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))
    ridge = RidgeCV(alphas=C)
    ridge.fit(X_train, y_train)
    y_pred = ridge.predict(X_val)
    print("Ridge Regression")
    print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
    print("R2 Score:", r2_score(y_val, y_pred))
    print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))

Lasso Regression (alpha=0.1)
Mean Squared Error: 0.9070163455659324
R2 Score: 0.0912594853891967
Mean Absolute Error: 0.7415297818944618
Ridge Regression
Mean Squared Error: 0.7238708808917491
R2 Score: 0.27475309565352657
Mean Absolute Error: 0.660850102072167
Lasso Regression (alpha=1)
Mean Squared Error: 0.9981092966643812
R2 Score: -6.626476704241213e-06
Mean Absolute Error: 0.7850793418708032
Ridge Regression
Mean Squared Error: 0.7238708808917491
R2 Score: 0.27475309565352657
Mean Absolute Error: 0.660850102072167
Lasso Regression (alpha=10)
Mean Squared Error: 0.9981092966643812
R2 Score: -6.626476704241213e-06
Mean Absolute Error: 0.7850793418708032
Ridge Regression
Mean Squared Error: 0.7238708808917491
R2 Score: 0.27475309565352657
Mean Absolute Error: 0.660850102072167

In [9]: importance = lr.coef_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 15))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
plt.show()
```

Feature Importance

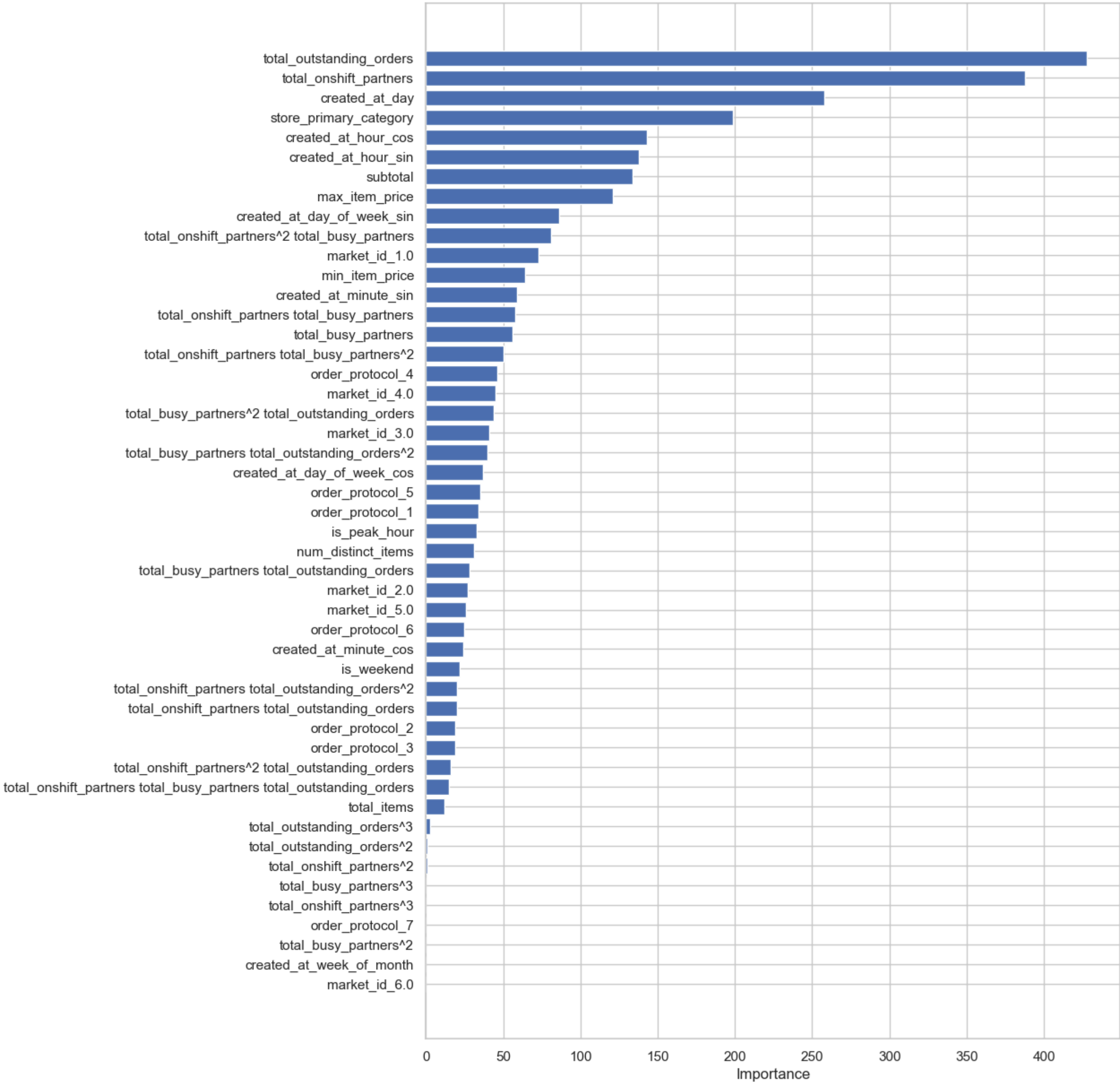


```
In [10]: lgbm = LGBMRegressor()
lgbm.fit(X_train, y_train)
y_pred = lgbm.predict(X_val)
print("LightGBM")
print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
print("R2 Score:", r2_score(y_val, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))

[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.006536 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 5507
[LightGBM] [Info] Number of data points in the train set: 124099, number of used features: 47
[LightGBM] [Info] Start training from score -0.000000
LightGBM
Mean Squared Error: 0.6615931175671896
R2 Score: 0.33714924426654636
Mean Absolute Error: 0.6306848036268882
```

```
In [12]: importance = lgbm.feature_importances_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 15))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
plt.show()
```

Feature Importance



```
In [ ]: # set up optuna study
study = optuna.create_study(direction="minimize")
def objective(trial):
    # hyperparameter tuning
    params = {
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1),
        'num_leaves': trial.suggest_int('num_leaves', 20, 100),
        'max_depth': trial.suggest_int('max_depth', 3, 12),
        'n_estimators': trial.suggest_int('n_estimators', 50, 200),
        'min_child_samples': trial.suggest_int('min_child_samples', 1, 20)
    }
    lgbm = LGBMRegressor(**params)
    lgbm.fit(X_train, y_train)
    y_pred = lgbm.predict(X_val)
    mse = mean_squared_error(y_val, y_pred)
    return mse
# optimize hyperparameters
study.optimize(objective, n_trials=100)
# print best hyperparameters
best_params = study.best_params
print("Best Hyperparameters:", best_params)
# train final model with best hyperparameters
```

Best Hyperparameters: {'learning_rate': 0.07752425203437846, 'num_leaves': 97, 'max_depth': 11, 'n_estimators': 153, 'min_child_samples': 19}

```
In [11]: lgbm = LGBMRegressor(**best_params)
lgbm.fit(X_train, y_train)
y_pred = lgbm.predict(X_val)
print("Validation Mean Squared Error:", mean_squared_error(y_val, y_pred))
print("Validation R2 Score:", r2_score(y_val, y_pred))
importance = lgbm.feature_importances_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 10))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.002314 seconds. You can set `force\_row\_wise=true` to remove the overhead. And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 1823

[LightGBM] [Info] Number of data points in the train set: 124099, number of used features: 31

[LightGBM] [Info] Start training from score -0.000000

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

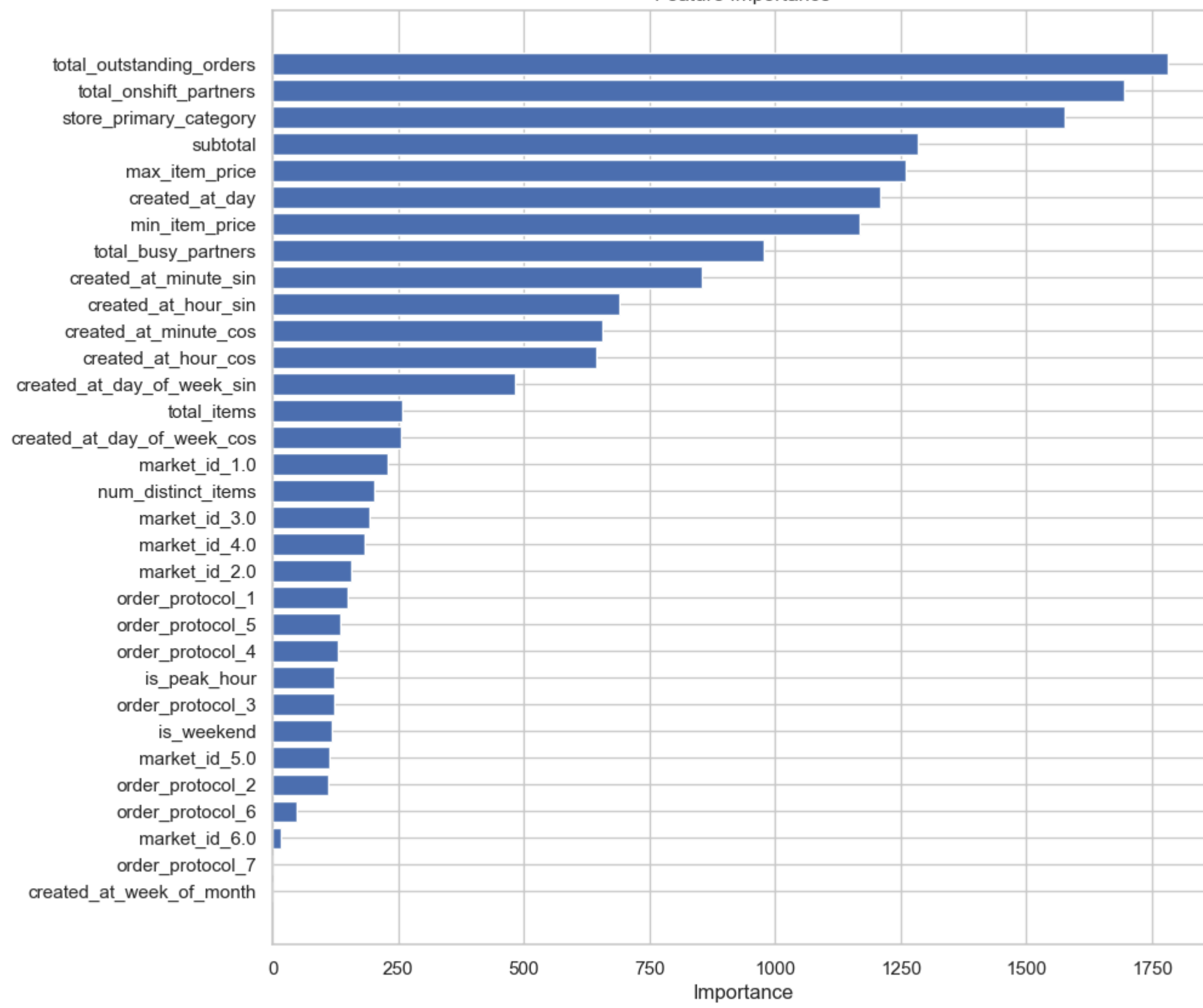
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

Validation Mean Squared Error: 0.6448919586547537

Validation R2 Score: 0.35388215081104113

Feature Importance



Deep Learning

```
In [1]: import pandas as pd
import numpy as np
from torch import nn
from torch.utils.data import DataLoader, Dataset, TensorDataset
from torchvision import datasets
from torchvision.transforms import ToTensor
import torch
import torch.nn as nn
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
from torchinfo import summary
from torch.utils.tensorboard import SummaryWriter
from torch.utils.data import WeightedRandomSampler
import os
from datetime import datetime
# counter
from collections import Counter

from notifier import send_notification, send_image, send_email
import time
from typing import Dict, Optional, Any

import ray
from ray import tune
from ray.tune.search import ConcurrencyLimiter
from ray.tune.search.optuna import OptunaSearch
from ray.tune.schedulers import ASHAScheduler
```

```
In [2]: writer = SummaryWriter()
```

```
In [3]: with open('../data/processed/dataset.pkl', 'rb') as f:
dataset = pickle.load(f)
```

```
In [4]: device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
device
```

Out[4]: 'mps'

```
In [5]: dataset.keys()
```

Out[5]: dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])

```
In [6]: X_train = dataset['X_train'].drop(columns=["store_primary_category"], axis=1)
y_train = dataset['y_train']
X_test = dataset['X_test'].drop(columns=["store_primary_category"], axis=1)
y_test = dataset['y_test']
X_val = dataset['X_val'].drop(columns=["store_primary_category"], axis=1)
y_val = dataset['y_val']
X_train_store_category_le=dataset['X_train_store_category_le']
X_val_store_category_le=dataset['X_val_store_category_le']
X_test_store_category_le=dataset['X_test_store_category_le']
y_scaler = dataset['y_scaler']
```

```
In [7]: len(X_train_store_category_le.unique())
```

Out[7]: 66

```
In [8]: X_train
```

Out[8]:

	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	created_at_week_of_month	...	order
0	-0.887847	-0.774580	-1.034172	1.175252	0.233244	0.356917	-0.239323	-0.063258	-1.134736	-1.239225	...	
1	0.323576	0.936110	0.203869	-0.179647	0.466436	-0.898395	-0.866322	-0.691474	0.050511	0.223960	...	
2	0.727384	0.600475	0.822890	-0.664373	0.053865	-0.985975	-1.023072	-0.938953	0.912508	0.955552	...	
3	-0.080232	-1.258467	0.203869	-1.151047	-1.817052	-0.927588	-1.023072	-0.938953	0.697009	0.955552	...	
4	1.938807	1.653122	2.060932	-1.053712	-0.295923	1.115942	0.857927	1.078952	1.666756	1.687144	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
124094	-0.080232	-0.288488	0.203869	-0.179647	-0.835852	-0.256143	-0.803622	-0.653400	1.451257	1.687144	...	
124095	-0.484040	-0.667111	-0.415151	0.007236	-0.663649	1.641422	1.798427	2.620935	1.666756	1.687144	...	
124096	0.727384	1.802477	1.441911	-0.664373	1.348978	0.619656	0.701177	0.679178	1.235757	0.955552	...	
124097	-0.080232	-0.443905	0.203869	-0.576772	-0.609835	-0.139369	-0.490123	-0.577253	0.589259	0.223960	...	
124098	-0.484040	-0.309982	-0.415151	0.735299	-0.172152	-1.190328	-1.148472	-0.977026	1.235757	0.955552	...	

124099 rows x 47 columns

```
In [9]: class CustomDataset(Dataset):
def __init__(self, X, y, category):
    self.X = torch.tensor(X.values, dtype=torch.float32)
    self.y = torch.tensor(y.values, dtype=torch.long).unsqueeze(1)
    self.category = torch.tensor(category.values, dtype=torch.long)

def __len__(self):
    return len(self.X)

def __getitem__(self, idx):
    return self.X[idx], self.y[idx], self.category[idx]

train_dataset = CustomDataset(X_train, y_train, X_train_store_category_le)
val_dataset = CustomDataset(X_val, y_val, X_val_store_category_le)
test_dataset = CustomDataset(X_test, y_test, X_test_store_category_le)

batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
In [10]: for X_batch, y_batch, category_batch in val_loader:
print(f'X_batch shape: {X_batch.shape}, y_batch shape: {y_batch.shape}, category_batch shape: {category_batch.shape}')
break
```

X_batch shape: torch.Size([64, 47]), y_batch shape: torch.Size([64, 1]), category_batch shape: torch.Size([64])

```
In [11]: class Trainer:
def __init__(self, model, criterion, optimizer, log_dir=None):
    self.model = model.to(device)
    self.criterion = criterion
    self.optimizer = optimizer
    self.plotting_dict = {
        "train_loss": [],
        "val_loss": [],
    }
    self.early_stop_counter = 0
    self.early_stop_patience = 5
```

```
self.best_val_loss = float('inf')

# TensorBoard setup
if log_dir is None:
    current_time = datetime.now().strftime('%Y%m%d-%H%M%S')
    log_dir = os.path.join('runs', current_time)

self.writer = SummaryWriter(log_dir)

def train(self, train_dataloader, val_dataloader, epochs=5):
    self.model.train()
    dataset_size = len(train_dataloader)
    for epoch in range(epochs):
        total_loss = 0
        for i, (inputs, targets, category) in enumerate(train_dataloader):
            inputs, targets, category = inputs.to(device), targets.to(device), category.to(device)

            outputs = self.model(inputs, category)
            loss = self.criterion(outputs, targets.float())
            loss.backward()
            self.optimizer.step()
            self.optimizer.zero_grad()

            total_loss += loss.item()

        if i % 10 == 0:
            step = epoch * len(train_dataloader) + i
            self.writer.add_scalar('Batch/Loss', loss.item(), step)

    individual_loss = total_loss / dataset_size
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {individual_loss:.4f}")

    val_loss = self.evaluate(val_dataloader, "Validation")
    if val_loss < self.best_val_loss:
        self.best_val_loss = val_loss
        self.early_stop_counter = 0
    else:
        self.early_stop_counter += 1
        if self.early_stop_counter >= self.early_stop_patience:
            print(f"Early stopping at epoch {epoch}")
            break

    self.plotting_dict["train_loss"].append(individual_loss)
    self.plotting_dict["val_loss"].append(val_loss)

    self.writer.add_scalar('Loss/Train', individual_loss, epoch)
    self.writer.add_scalar('Loss/Validation', val_loss, epoch)

    for name, param in self.model.named_parameters():
        self.writer.add_histogram(f'Parameters/{name}', param, epoch)
    return val_loss

def evaluate(self, dataloader, mode="Evaluation"):
    self.model.eval()
    loss = 0
    with torch.no_grad():
        for inputs, targets, category in dataloader:
            inputs, targets, category = inputs.to(device), targets.to(device), category.to(device)
            outputs = self.model(inputs, category)
            loss += self.criterion(outputs, targets.float()).item()
    loss /= len(dataloader)
    print(f"{mode} Loss: {loss:.4f}")
    print("-----")

    return loss

def plot(self):
    fig = plt.figure(figsize=(10, 6))
    plt.plot(self.plotting_dict["train_loss"], label="Train Loss")
    plt.plot(self.plotting_dict["val_loss"], label="Validation Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()
    plt.tight_layout()

# self.writer.add_figure('Loss Curves', fig)

return fig

def close(self):
    """Close the TensorBoard writer when done"""
    self.writer.close()
```

Simple Neural Network

```
In [17]: class SimpleNN(nn.Module):
def __init__(self, input_size):
    super().__init__()
    self.fc1 = nn.Linear(input_size, 256)
    self.fc2 = nn.Linear(256, 128)
    self.fc3 = nn.Linear(128, 64)
    self.fc4 = nn.Linear(64, 32)
    self.fc5 = nn.Linear(32, 1)
    self.dropout = nn.Dropout(0.2)

def forward(self, x, category):
    x = torch.tanh(self.fc1(x))
    # x = self.dropout(x)
    x = torch.tanh(self.fc2(x))
    x = self.dropout(x)
    x = torch.tanh(self.fc3(x))
    x = self.dropout(x)
    x = torch.tanh(self.fc4(x))
    x = self.fc5(x)
    return x
```

```
In [18]: model = SimpleNN(X_train.shape[1]).to(device)
print(summary(model))
```

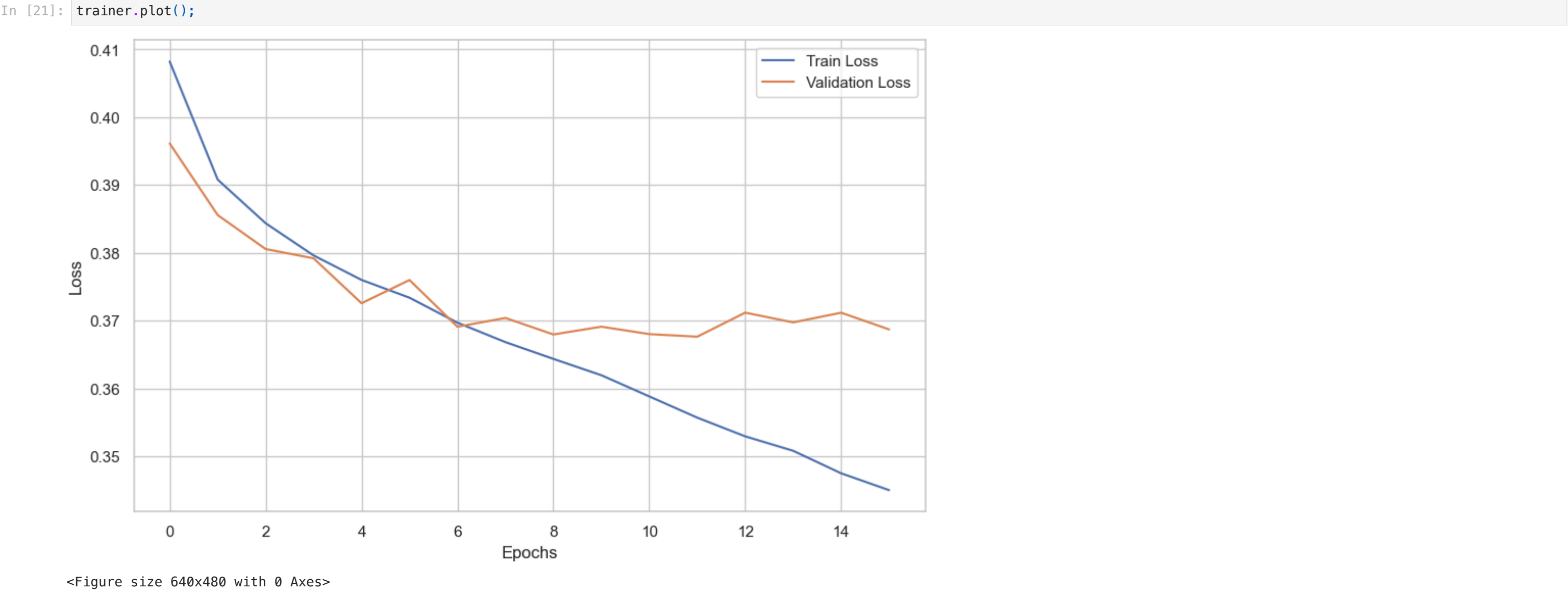
```
=====
Layer (type:depth-idx)              Param #
=====
SimpleNN                             --
├─Linear: 1-1                        12,288
├─Linear: 1-2                        32,896
├─Linear: 1-3                        8,256
├─Linear: 1-4                        2,080
├─Linear: 1-5                        33
├─Dropout: 1-6                       --
=====
Total params: 55,553
Trainable params: 55,553
Non-trainable params: 0
=====
```

```
In [19]: loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```



```
In [20]: num_epochs = 20
         trainer = Trainer(model, loss_fn, optimizer)
         trainer.train(train_loader, val_loader, epochs=num_epochs)
         # trainer.evaluate(test_loader)
         send_notification("Training completed")

Epoch [1/20], Loss: 0.4082
Validation Loss: 0.3961
-----
Epoch [2/20], Loss: 0.3908
Validation Loss: 0.3856
-----
Epoch [3/20], Loss: 0.3844
Validation Loss: 0.3806
-----
Epoch [4/20], Loss: 0.3796
Validation Loss: 0.3792
-----
Epoch [5/20], Loss: 0.3760
Validation Loss: 0.3726
-----
Epoch [6/20], Loss: 0.3734
Validation Loss: 0.3760
-----
Epoch [7/20], Loss: 0.3697
Validation Loss: 0.3691
-----
Epoch [8/20], Loss: 0.3668
Validation Loss: 0.3704
-----
Epoch [9/20], Loss: 0.3644
Validation Loss: 0.3680
-----
Epoch [10/20], Loss: 0.3620
Validation Loss: 0.3691
-----
Epoch [11/20], Loss: 0.3588
Validation Loss: 0.3680
-----
Epoch [12/20], Loss: 0.3557
Validation Loss: 0.3676
-----
Epoch [13/20], Loss: 0.3529
Validation Loss: 0.3712
-----
Epoch [14/20], Loss: 0.3508
Validation Loss: 0.3697
-----
Epoch [15/20], Loss: 0.3475
Validation Loss: 0.3712
-----
Epoch [16/20], Loss: 0.3450
Validation Loss: 0.3687
-----
Epoch [17/20], Loss: 0.3424
Validation Loss: 0.3733
-----
Early stopping at epoch 16
```



Neural Network with Category Embedding Layer

```
In [26]: class NetworkWithCategoryEmbedding(nn.Module):
         def __init__(self, input_size, num_categories, embedding_dim):
             super().__init__()
             self.category_embedding = nn.Embedding(num_categories, embedding_dim)
             self.fc1 = nn.Linear(input_size, 256)
             self.fc2 = nn.Linear(256 + embedding_dim, 256)
             self.batchnorm_emb = nn.BatchNorm1d(embedding_dim)
             self.fc3 = nn.Linear(256, 128)
             self.fc4 = nn.Linear(128, 64)
             self.fc5 = nn.Linear(64, 32)
             self.fc6 = nn.Linear(32, 1)
             self.dropout = nn.Dropout(0.1)
             self.batchnorm = nn.BatchNorm1d(256)

         def forward(self, x, categories):
             emb = self.batchnorm_emb(self.category_embedding(categories))
             x = torch.relu(self.fc1(x))
             x = torch.cat((x, emb), dim=1)
             x = torch.relu(self.fc2(x))
             x = self.batchnorm(x)
             x = torch.relu(self.fc3(x))
             x = self.dropout(x)
             x = torch.relu(self.fc4(x))
             x = torch.relu(self.fc5(x))
             x = self.fc6(x)
             return x

In [28]: model_with_embedding = NetworkWithCategoryEmbedding(X_train.shape[1], len(X_train_store_category_le.unique()), 35).to(device)
         print(summary(model_with_embedding))
         optimizer = torch.optim.Adam(model_with_embedding.parameters(), lr=0.001)
         loss_fn = nn.MSELoss()
         # x_sample = torch.randn(1, X_train.shape[1])
```

```
# category_sample = torch.randint(0, len(X_train_store_category_le.unique()), (1,))
# writer.add_graph(model_with_embedding, (x_sample, category_sample))
# writer.close()
# model_with_embedding = model_with_embedding.to(device)
```

Layer (type:depth-idx)	Param #
NetworkWithCategoryEmbedding	--
├─Embedding: 1-1	2,310
├─Linear: 1-2	12,288
├─Linear: 1-3	74,752
├─BatchNorm1d: 1-4	70
├─Linear: 1-5	32,896
├─Linear: 1-6	8,256
├─Linear: 1-7	2,080
├─Linear: 1-8	33
├─Dropout: 1-9	--
├─BatchNorm1d: 1-10	512
Total params: 133,197	
Trainable params: 133,197	
Non-trainable params: 0	

```
In [29]: num_epochs = 15
optimizer = torch.optim.Adam(model_with_embedding.parameters(), lr=0.001)
e_trainer = Trainer(model_with_embedding, loss_fn, optimizer)
e_trainer.train(train_loader, val_loader, epochs=num_epochs)
# e_trainer.evaluate(test_loader) # Uncommented to evaluate the model
send_notification("Training completed") # Uncommented to send notification
e_trainer.close() # Close the TensorBoard writer
```

Epoch [1/15], Loss: 0.3964
Validation Loss: 0.3801

Epoch [2/15], Loss: 0.3804
Validation Loss: 0.3759

Epoch [3/15], Loss: 0.3738
Validation Loss: 0.3691

Epoch [4/15], Loss: 0.3681
Validation Loss: 0.3648

Epoch [5/15], Loss: 0.3637
Validation Loss: 0.3648

Epoch [6/15], Loss: 0.3592
Validation Loss: 0.3645

Epoch [7/15], Loss: 0.3553
Validation Loss: 0.3620

Epoch [8/15], Loss: 0.3512
Validation Loss: 0.3662

Epoch [9/15], Loss: 0.3485
Validation Loss: 0.3651

Epoch [10/15], Loss: 0.3455
Validation Loss: 0.3686

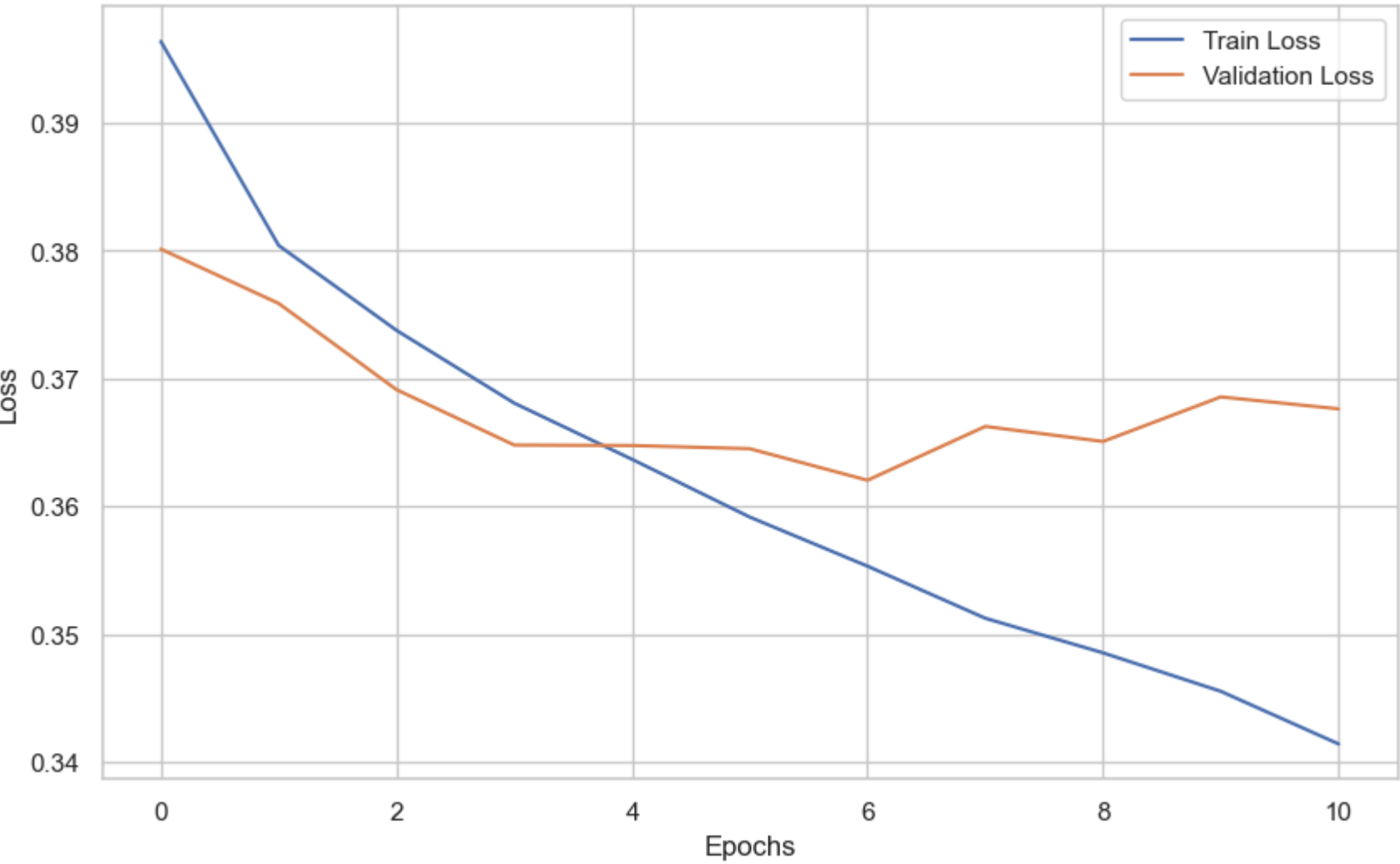
Epoch [11/15], Loss: 0.3414
Validation Loss: 0.3676

Epoch [12/15], Loss: 0.3376
Validation Loss: 0.3784

Early stopping at epoch 11

objective_cba40b48_19_activation=relu,layer1_neurons=128,layer2_neurons=256,layer3_neurons=128,layer4_neurons=16_2025-03-14_13-02-46'

```
In [30]: e_trainer.plot();
```



<Figure size 640x480 with 0 Axes>

Observations

- From above plot we can see that the model is overfitting the training data after epoch 6

Hyperparameter Tuning Using Ray Tune & Optuna

```
In [ ]: import pandas as pd
import numpy as np
from torch import nn
from torch.utils.data import DataLoader, Dataset, TensorDataset
from torchvision import datasets
from torchvision.transforms import ToTensor
import torch
import torch.nn as nn
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
from torchinfo import summary
from torch.utils.tensorboard import SummaryWriter
import os
from datetime import datetime
from collections import Counter

from notifier import send_notification, send_image, send_email
import time
from typing import Dict, Optional, Any

import ray
from ray import tune
from ray.tune.search import ConcurrencyLimiter
from ray.tune.search.optuna import OptunaSearch
from ray.tune.schedulers import ASHAScheduler
from ray.train import Checkpoint
```

```
In [2]: writer = SummaryWriter()
```

```
In [3]: with open('../data/processed/dataset.pkl', 'rb') as f:
dataset = pickle.load(f)
```

```
In [4]: device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
device
```

Out[4]: 'mps'

```
In [5]: dataset.keys()
```

Out[5]: dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])

```
In [6]: X_train = dataset['X_train'].drop(columns=["store_primary_category"], axis=1)
y_train = dataset['y_train']
X_test = dataset['X_test'].drop(columns=["store_primary_category"], axis=1)
y_test = dataset['y_test']
X_val = dataset['X_val'].drop(columns=["store_primary_category"], axis=1)
y_val = dataset['y_val']
X_train_store_category_le=dataset['X_train_store_category_le']
X_val_store_category_le=dataset['X_val_store_category_le']
X_test_store_category_le=dataset['X_test_store_category_le']
y_scaler = dataset['y_scaler']
```

```
In [7]: len(X_train_store_category_le.unique())
```

Out[7]: 66

```
In [8]: X_train
```

Out[8]:

	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	created_at_week_of_month	...	order
0	-0.887847	-0.774580	-1.034172	1.175252	0.233244	0.356917	-0.239323	-0.063258	-1.134736	-1.239225	...	
1	0.323576	0.936110	0.203869	-0.179647	0.466436	-0.898395	-0.866322	-0.691474	0.050511	0.223960	...	
2	0.727384	0.600475	0.822890	-0.664373	0.053865	-0.985975	-1.023072	-0.938953	0.912508	0.955552	...	
3	-0.080232	-1.258467	0.203869	-1.151047	-1.817052	-0.927588	-1.023072	-0.938953	0.697009	0.955552	...	
4	1.938807	1.653122	2.060932	-1.053712	-0.295923	1.115942	0.857927	1.078952	1.666756	1.687144	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
124094	-0.080232	-0.288488	0.203869	-0.179647	-0.835852	-0.256143	-0.803622	-0.653400	1.451257	1.687144	...	
124095	-0.484040	-0.667111	-0.415151	0.007236	-0.663649	1.641422	1.798427	2.620935	1.666756	1.687144	...	
124096	0.727384	1.802477	1.441911	-0.664373	1.348978	0.619656	0.701177	0.679178	1.235757	0.955552	...	
124097	-0.080232	-0.443905	0.203869	-0.576772	-0.609835	-0.139369	-0.490123	-0.577253	0.589259	0.223960	...	
124098	-0.484040	-0.309982	-0.415151	0.735299	-0.172152	-1.190328	-1.148472	-0.977026	1.235757	0.955552	...	

124099 rows × 47 columns

```
In [9]: class CustomDataset(Dataset):
def __init__(self, X, y, category):
self.X = torch.tensor(X.values, dtype=torch.float32)
self.y = torch.tensor(y.values, dtype=torch.long).unsqueeze(1)
self.category = torch.tensor(category.values, dtype=torch.long)

def __len__(self):
return len(self.X)

def __getitem__(self, idx):
return self.X[idx], self.y[idx], self.category[idx]

train_dataset = CustomDataset(X_train, y_train, X_train_store_category_le)
val_dataset = CustomDataset(X_val, y_val, X_val_store_category_le)
test_dataset = CustomDataset(X_test, y_test, X_test_store_category_le)

batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
In [10]: for X_batch, y_batch, category_batch in val_loader:
print(f'X_batch shape: {X_batch.shape}, y_batch shape: {y_batch.shape}, category_batch shape: {category_batch.shape}')
break
```

X_batch shape: torch.Size([64, 47]), y_batch shape: torch.Size([64, 1]), category_batch shape: torch.Size([64])

```
In [ ]: class Trainer:
def __init__(self, model, criterion, optimizer, log_dir=None, checkpoint_dir=None):
self.model = model.to(device)
self.criterion = criterion
self.optimizer = optimizer
self.plotting_dict = {
"train_loss": [],
"val_loss": [],
}
self.early_stop_counter = 0
self.early_stop_patience = 5
self.best_val_loss = float('inf')
```



```

self.checkpoint_dir = checkpoint_dir

if log_dir is None:
    current_time = datetime.now().strftime('%Y%m%d-%H%M%S')
    log_dir = os.path.join('runs', current_time)

self.writer = SummaryWriter(log_dir)

def train(self, train_dataloader, val_dataloader, epochs=5, checkpoint_path=None):
    self.model.train()
    dataset_size = len(train_dataloader)

    temp_checkpoint_dir = self.checkpoint_dir
    if temp_checkpoint_dir is None:
        temp_checkpoint_dir = os.path.abspath(os.path.join(os.getcwd(), 'ray_results'))
        os.makedirs(temp_checkpoint_dir, exist_ok=True)

    checkpoint = ray.train.get_checkpoint()
    if checkpoint:
        with checkpoint.as_directory() as checkpoint_dir:
            print(f"Loading checkpoint from {checkpoint_dir}")
            checkpoint_dict = torch.load(os.path.join(checkpoint_dir, "checkpoint.pt"))
            start = checkpoint_dict["epoch"] + 1
            self.model.load_state_dict(checkpoint_dict["model_state"])

    for epoch in range(epochs):
        total_loss = 0
        for i, (inputs, targets, category) in enumerate(train_dataloader):
            inputs, targets, category = inputs.to(device), targets.to(device), category.to(device)

            outputs = self.model(inputs, category)
            loss = self.criterion(outputs, targets.float())
            loss.backward()
            self.optimizer.step()
            self.optimizer.zero_grad()

            total_loss += loss.item()

            if i % 10 == 0:
                step = epoch * len(train_dataloader) + i
                self.writer.add_scalar('Batch/Loss', loss.item(), step)

        individual_loss = total_loss / dataset_size
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {individual_loss:.4f}")

        val_loss = self.evaluate(val_dataloader, "Validation")
        if val_loss < self.best_val_loss:
            self.best_val_loss = val_loss
            self.early_stop_counter = 0

            print(f"Validation loss improved to {val_loss:.4f}. Saving model...")
            checkpoint_path = os.path.join(temp_checkpoint_dir, "checkpoint.pt")
            torch.save({
                "epoch": epoch,
                "model_state": self.model.state_dict(),
                "optimizer_state": self.optimizer.state_dict(),
                "val_loss": val_loss
            }, checkpoint_path)
        else:
            self.early_stop_counter += 1
            if self.early_stop_counter >= self.early_stop_patience:
                print(f"Early stopping at epoch {epoch}")
                break

        self.plotting_dict["train_loss"].append(individual_loss)
        self.plotting_dict["val_loss"].append(val_loss)

        self.writer.add_scalar('Loss/Train', individual_loss, epoch)
        self.writer.add_scalar('Loss/Validation', val_loss, epoch)

        for name, param in self.model.named_parameters():
            self.writer.add_histogram(f'Parameters/{name}', param, epoch)

    try:
        print("Reporting metrics and checkpoint...")
        checkpoint = ray.train.Checkpoint.from_directory(temp_checkpoint_dir)
        ray.train.report({"val_loss": val_loss}, checkpoint=checkpoint)
    except Exception as e:
        print(f"Warning: Failed to report checkpoint: {e}")
        try:
            ray.train.report({"val_loss": val_loss})
        except Exception as e:
            print(f"Warning: Failed to report metrics: {e}")

    return val_loss

def evaluate(self, dataloader, mode="Evaluation"):
    self.model.eval()
    loss = 0
    with torch.no_grad():
        for inputs, targets, category in dataloader:
            inputs, targets, category = inputs.to(device), targets.to(device), category.to(device)
            outputs = self.model(inputs, category)
            loss += self.criterion(outputs, targets.float()).item()
    loss /= len(dataloader)
    print(f"{mode} Loss: {loss:.4f}")
    print("-----")

    return loss

def plot(self):
    fig = plt.figure(figsize=(10, 6))
    plt.plot(self.plotting_dict["train_loss"], label="Train Loss")
    plt.plot(self.plotting_dict["val_loss"], label="Validation Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()
    plt.tight_layout()

    # self.writer.add_figure('Loss Curves', fig)

    return fig

def close(self):
    """Close the TensorBoard writer when done"""
    self.writer.close()

def save_checkpoint(self, epoch, val_loss):
    if not self.checkpoint_dir:
        return

    checkpoint_path = os.path.join(self.checkpoint_dir, f"checkpoint_epoch_{epoch}.pt")
    checkpoint = {
        "epoch": epoch,
        "model_state_dict": self.model.state_dict(),
        "optimizer_state_dict": self.optimizer.state_dict(),
        "val_loss": val_loss,
    }
    torch.save(checkpoint, checkpoint_path)
    print(f"Checkpoint saved: {checkpoint_path}")

```

```
def load_checkpoint(self, checkpoint_path):
    if not os.path.exists(checkpoint_path):
        print(f"No checkpoint found at {checkpoint_path}")
        return

    checkpoint = torch.load(checkpoint_path, map_location=device)
    self.model.load_state_dict(checkpoint["model_state_dict"])
    self.optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
    self.best_val_loss = checkpoint["val_loss"]
    print(f"Checkpoint loaded from {checkpoint_path}")
```

```
In [17]: class NetworkWithCategoryEmbedding(nn.Module):
def __init__(self, input_size, num_categories, embedding_dim,
            neuron_counts=[256,128, 128, 64, 32],
            activation_fn=torch.tanh):
    super().__init__()
    self.category_embedding = nn.Embedding(num_categories, embedding_dim)
    self.batchnorm_emb = nn.BatchNorm1d(embedding_dim)

    self.fc1 = nn.Linear(input_size, neuron_counts[0])
    self.fc2 = nn.Linear(neuron_counts[0] + embedding_dim, neuron_counts[1])
    self.fc3 = nn.Linear(neuron_counts[1], neuron_counts[2])
    self.fc4 = nn.Linear(neuron_counts[2], neuron_counts[3])
    self.fc5 = nn.Linear(neuron_counts[3], neuron_counts[4])
    self.fc6 = nn.Linear(neuron_counts[4], 1)
    self.dropout = nn.Dropout(0.2)
    self.batchnorm = nn.BatchNorm1d(neuron_counts[1])

    self.activation = activation_fn

def forward(self, x, categories):
    emb = self.batchnorm_emb(self.category_embedding(categories))
    x = self.activation(self.fc1(x))
    x = torch.cat((x, emb), dim=1)
    x = self.activation(self.fc2(x))
    x = self.batchnorm(x)
    x = self.activation(self.fc3(x))
    x = self.dropout(x)
    x = self.activation(self.fc4(x))
    x = self.activation(self.fc5(x))
    x = self.fc6(x)
    return x
```

```
In [18]: def objective(config):

    activation_map = {
        "tanh": torch.tanh,
        "relu": torch.relu,
        "leaky_relu": nn.functional.leaky_relu,
        "elu": nn.functional.elu
    }

    activation_fn = activation_map[config["activation"]]

    neuron_counts = [
        config["layer1_neurons"],
        config["layer2_neurons"],
        config["layer3_neurons"],
        config["layer4_neurons"],
        config["layer5_neurons"],
    ]

    model = NetworkWithCategoryEmbedding(
        input_size=X_train.shape[1],
        num_categories=len(X_train.store_category_le.unique()),
        embedding_dim=35,
        neuron_counts=neuron_counts,
        activation_fn=activation_fn,
    )

    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    loss_fn = nn.MSELoss()

    tunerTrainer = Trainer(
        model,
        loss_fn,
        optimizer,
        # checkpoint_path=tune.get_checkpoint_dir(),
    )

    val_loss = tunerTrainer.train(
        train_loader,
        val_loader,
        epochs=10,
        # checkpoint_path=tune.get_checkpoint_dir(),
    )

    # ray.train.report(dict(val_loss=val_loss))
```

```
In [ ]: def run_optuna_optimization():
    search_space = {
        "layer1_neurons": tune.choice([128, 256]),
        "layer2_neurons": tune.choice([64, 128, 256]),
        "layer3_neurons": tune.choice([32, 64, 128]),
        "layer4_neurons": tune.choice([16, 32, 64]),
        "layer5_neurons": tune.choice([8, 16, 32]),

        "activation": tune.choice(["tanh", "relu", "leaky_relu", "elu"])
    }

    optuna_search = OptunaSearch(
        metric="val_loss",
        mode="min"
    )

    scheduler = ASHAScheduler(
        max_t=10,
        grace_period=5,
        reduction_factor=2,
        metric="val_loss",
        mode="min"
    )

    tuner = tune.Tuner(
        tune.with_resources(
            objective,
            resources={"cpu": 2}
        ),
        tune_config=tune.TuneConfig(
            search_alg=optuna_search,
            scheduler=scheduler,
            num_samples=20,
        ),
        param_space=search_space,
        run_config=ray.train.RunConfig(
            storage_path=os.path.abspath(os.path.join(os.getcwd(), 'ray_results')),
            name="optuna_regression_asha_mps",
            checkpoint_config=ray.train.CheckpointConfig(
                num_to_keep=1,
                checkpoint_score_attribute="val_loss",
```

```
        ),
        ),
    ),

    results = tuner.fit()
    return results
```

```
In [ ]: def get_best_model_from_tuning(results):
        best_config = results.get_best_config(metric="val_loss", mode="min")
        best_trial = results.get_best_trial(metric="val_loss", mode="min")

        activation_map = {
            "relu": nn.ReLU(),
            "tanh": nn.Tanh(),
            "leaky_relu": nn.LeakyReLU(),
            "elu": nn.ELU()
        }

        best_model = NetworkWithCategoryEmbedding(
            input_size=X_train.shape[1],
            num_categories=len(X_train_store_category_le.unique()),
            embedding_dim=35,
            neuron_counts=[
                best_config["layer1_neurons"],
                best_config["layer2_neurons"],
                best_config["layer3_neurons"],
                best_config["layer4_neurons"],
                best_config["layer5_neurons"],
            ],
            activation_fn=activation_map[best_config["activation"]],
        ).to(device)

        checkpoint_path = results.get_best_checkpoint(best_trial, metric="val_loss", mode="min").to_directory()
        checkpoint = torch.load(os.path.join(checkpoint_path, "checkpoint.pt"), map_location=device)
        best_model.load_state_dict(checkpoint["model_state"])

        return best_model, best_config
```

```
In [21]: results = run_optuna_optimization()
        send_notification("Tuning finished")
```

Tune Status

Current time:	2025-03-20 10:58:08
Running for:	00:05:35.44
Memory:	19.1/64.0 GiB

System Info

Using AsyncHyperBand: num_stopped=0
Bracket: Iter 10.000: None | Iter 5.000: None
Logical resource usage: 2.0/16 CPUs, 0/0 GPUs

Trial Status

Trial name	status	loc	activation	layer1_neurons	layer2_neurons	layer3_neurons	layer4_neurons	layer5_neurons	iter	total time (s)	val_loss
objective_914a9451	TERMINATED	127.0.0.1:3670	elu	256	256	32	16	32	1	112.03	0.36339
objective_711ad93e	TERMINATED	127.0.0.1:3678	elu	128	256	128	32	32	1	111.026	0.362535
objective_b37b8005	TERMINATED	127.0.0.1:3680	tanh	128	64	64	32	32	1	106.851	0.374905
objective_85302274	TERMINATED	127.0.0.1:3681	relu	128	64	128	16	32	1	110.468	0.365131
objective_03bcd069	TERMINATED	127.0.0.1:3682	leaky_relu	128	64	128	64	8	1	111.996	0.370334
objective_4489d434	TERMINATED	127.0.0.1:3683	tanh	256	256	64	32	16	1	110.231	0.37018
objective_09b41655	TERMINATED	127.0.0.1:3696	elu	256	128	128	64	32	1	111.456	0.365192
objective_074d9f07	TERMINATED	127.0.0.1:3699	tanh	128	128	128	64	8	1	109.341	0.372363
objective_b8128406	TERMINATED	127.0.0.1:3744	relu	256	256	64	32	32	1	110.836	0.36815
objective_4fb3f49c	TERMINATED	127.0.0.1:3746	tanh	128	256	128	16	32	1	107.765	0.366969
objective_727e8290	TERMINATED	127.0.0.1:3751	leaky_relu	128	128	64	16	32	1	110.984	0.365205
objective_e8390d67	TERMINATED	127.0.0.1:3753	relu	128	64	64	64	16	1	107.52	0.366692
objective_3712fe93	TERMINATED	127.0.0.1:3760	relu	128	128	64	64	8	1	99.1381	0.366201
objective_5868280b	TERMINATED	127.0.0.1:3763	leaky_relu	256	256	128	64	16	1	111.613	0.368404
objective_59366d73	TERMINATED	127.0.0.1:3765	tanh	128	128	64	32	16	1	107.828	0.371683
objective_64bedd3d	TERMINATED	127.0.0.1:3770	leaky_relu	256	64	64	16	8	1	110.213	0.370317
objective_b8eb627a	TERMINATED	127.0.0.1:3876	elu	256	64	64	32	16	1	101.17	0.364631
objective_f277c222	TERMINATED	127.0.0.1:3877	elu	256	256	32	16	16	1	102.851	0.365021
objective_e6e69742	TERMINATED	127.0.0.1:3879	elu	256	256	32	16	16	1	102.742	0.365527
objective_63c309bd	TERMINATED	127.0.0.1:3893	elu	256	256	32	32	32	1	102.842	0.36803


```
type(results)

ray.tune.result_grid.ResultGrid

best_result = results.get_best_result(metric="val_loss", mode="min")
best_config = best_result.config
best_config

{'layer1_neurons': 128,
 'layer2_neurons': 256,
 'layer3_neurons': 128,
 'layer4_neurons': 32,
 'layer5_neurons': 32,
 'activation': 'elu'}

results.get_best_result(metric="val_loss", mode="min").metrics

{'val_loss': 0.35998701601504135,
 'timestamp': 1742419152,
 'checkpoint_dir_name': 'checkpoint_000000',
 'should_checkpoint': True,
 'done': True,
 'training_iteration': 1,
 'trial_id': '577065fa',
 'date': '2025-03-20_02-49-12',
 'time_this_iter_s': 82.945809841156,
 'time_total_s': 82.945809841156,
 'pid': 13085,
 'hostname': 'Gautams-MacBook-Pro.local',
 'node_ip': '127.0.0.1',
 'config': {'layer1_neurons': 256,
 'layer2_neurons': 128,
 'layer3_neurons': 32,
 'layer4_neurons': 64,
 'activation': 'elu'},
 'time_since_restore': 82.945809841156,
 'iterations_since_restore': 1,
 'experiment_tag': '19_activation=elu,layer1_neurons=256,layer2_neurons=128,layer3_neurons=32,layer4_neurons=64'}
```

```
In [ ]: def load_previous_tuning_results():
        from ray.tune import ExperimentAnalysis
```

```
results_path = os.path.abspath(os.path.join(os.getcwd(), 'ray_results', 'optuna_regression_asha_mps'))

if not os.path.exists(results_path):
    print(f"No results found at {results_path}")
    return None

analysis = ExperimentAnalysis(results_path)
print(f"Loaded results from {results_path}")
print(f"Number of trials: {len(analysis.trials)}")

return analysis

results = load_previous_tuning_results()

if results is not None:
    best_config = results.get_best_config(metric="val_loss", mode="min")
    best_trial = results.get_best_trial(metric="val_loss", mode="min")
    # best_model = get_best_model_from_tuning(results)

    print("Best hyperparameters found:")
    print(best_config)
    print(f"Best validation loss: {best_trial.last_result['val_loss']}")
```

Loaded results from /Users/gautamnaik/Developer/ai/PorterDeepLearningCaseStudy/notebooks/ray_results/optuna_regression_asha_mps
Number of trials: 20
Best hyperparameters found:
{'layer1_neurons': 128, 'layer2_neurons': 256, 'layer3_neurons': 128, 'layer4_neurons': 32, 'layer5_neurons': 32, 'activation': 'elu'}
Best validation loss: 0.36253498032545817

```
In [25]: best_model, best_config = get_best_model_from_tuning(results)
```

```
In [26]: # You can now use the best model for inference
best_model.eval()
predictions = []
with torch.no_grad():
    for X_batch, y_batch, category_batch in test_loader:
        X_batch, y_batch, category_batch = X_batch.to(device), y_batch.to(device), category_batch.to(device)
        preds = best_model(X_batch, category_batch)
        predictions.append(preds)
predictions = torch.cat(predictions).cpu().numpy()
```

```
In [27]: y_test.shape, predictions.shape
```

Out[27]: ((26593,), (26593, 1))

```
In [28]: # inverse the scaling
y_test_o = np.power(y_scaler.inverse_transform(y_test.to_frame()), 3)
predictions_o = np.power(y_scaler.inverse_transform(predictions), 3)
```

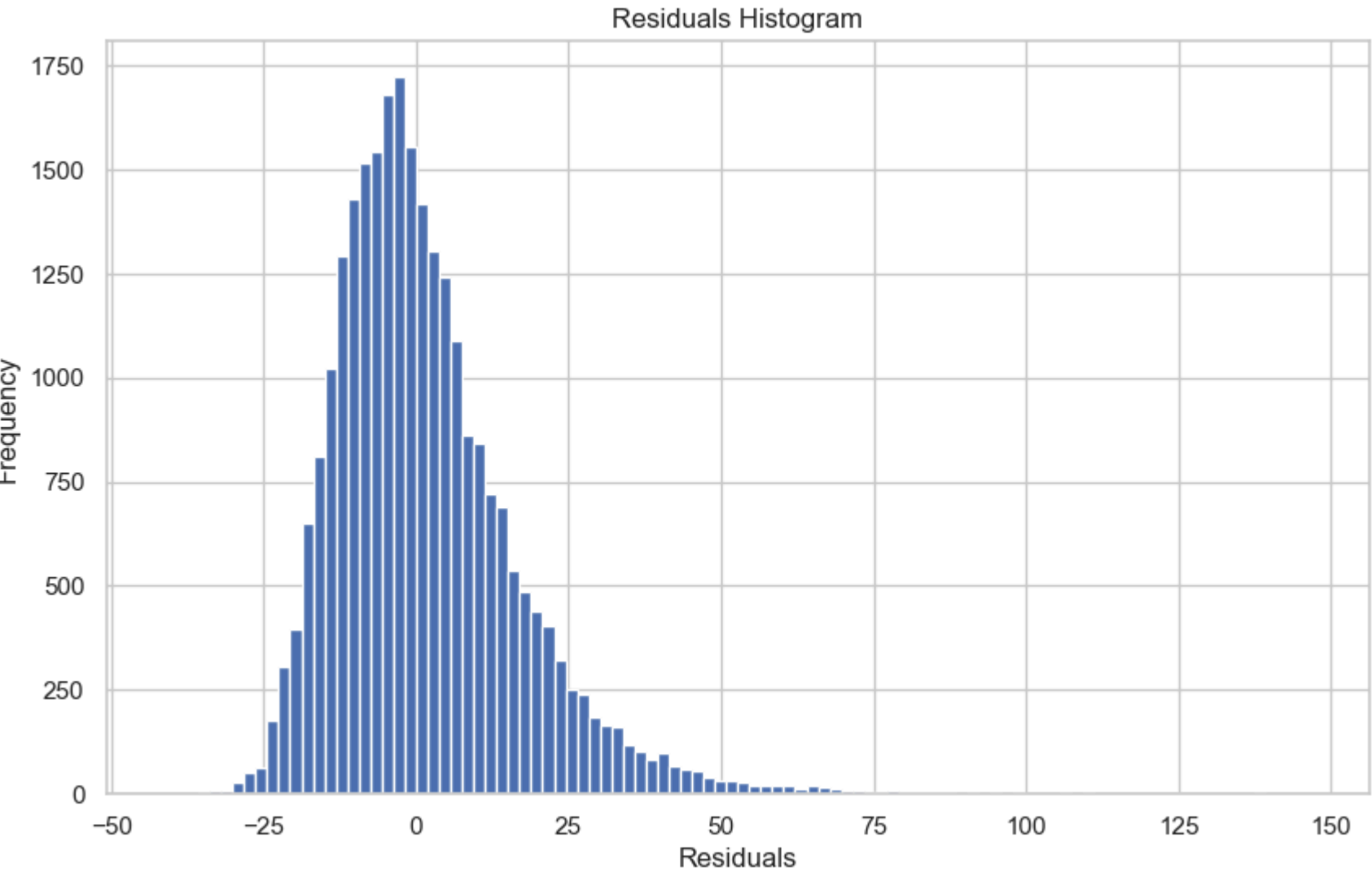
```
In [29]: # mean squared error
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(y_test_o, predictions_o)
mae = mean_absolute_error(y_test_o, predictions_o)
r2 = r2_score(y_test_o, predictions_o)

print(f"MSE: {mse:.4f}")
print(f"MAE: {mae:.4f}")
print(f"R2: {r2:.4f}")

MSE: 245.9411
MAE: 11.3271
R2: 0.2397
```

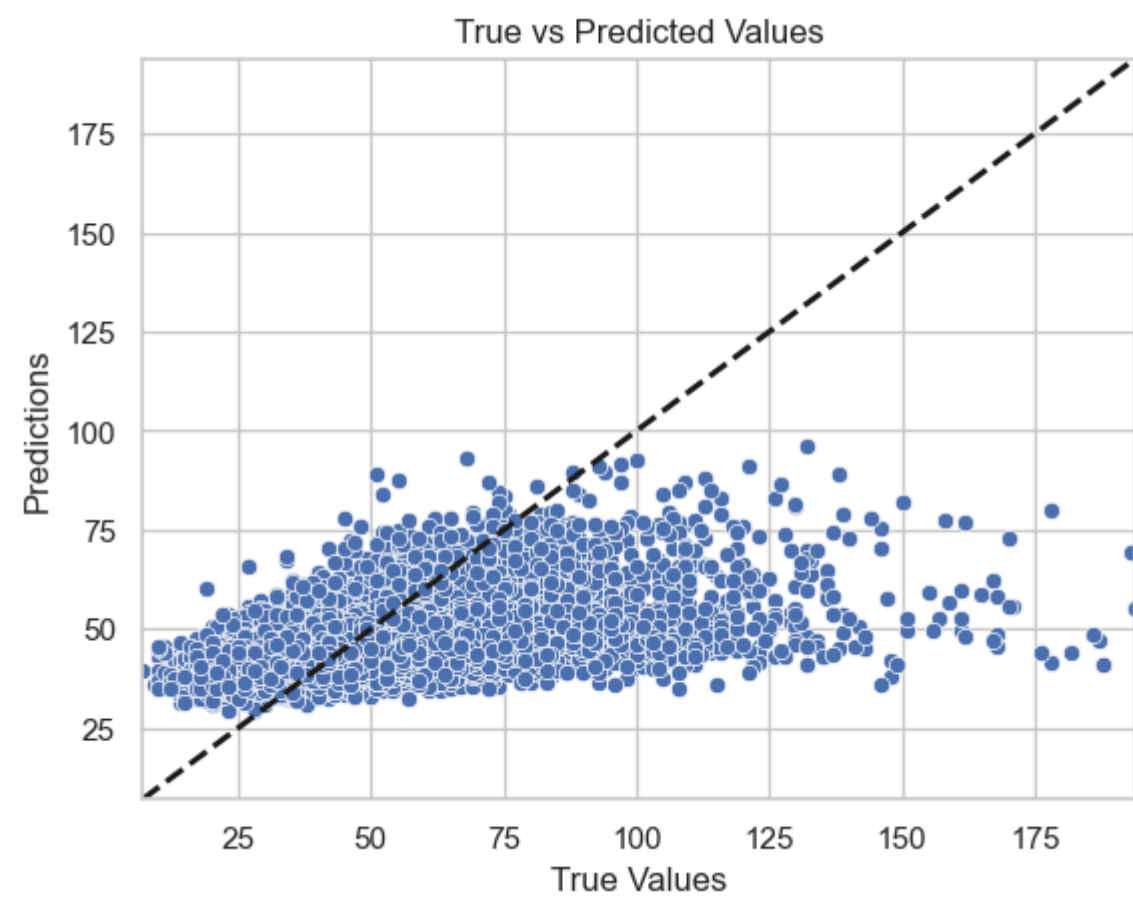
```
In [30]: residuals = y_test_o - predictions_o
residuals = residuals.flatten()
residuals.shape
plt.figure(figsize=(10, 6))
plt.hist(residuals, bins=100)
plt.title("Residuals Histogram")
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.show()
```



Observations

- We can see that residuals are normally distributed.

```
In [31]: sns.scatterplot(x=y_test_o.flatten(), y=predictions_o.flatten())
plt.xlabel("True Values")
plt.ylabel("Predictions")
plt.title("True vs Predicted Values")
plt.plot([y_test_o.min(), y_test_o.max()], [y_test_o.min(), y_test_o.max()], 'k--', lw=2)
plt.xlim(y_test_o.min(), y_test_o.max())
plt.ylim(y_test_o.min(), y_test_o.max())
plt.show()
```



Observations

- From above plot we can see that model is underperforming on the test set.

Insights

- There are 177544 rows in the dataset
- First orders were placed on 2015-01-21 09:52:03 and last orders were placed on 2015-02-18 00:30:44
- Majority of the orders are from market id 2
- Market id 6 has the least number of orders
- Order protocol 6 had a slightly higher delivery time
- Majority of the orders are from Order Protocol 1
- Order Protocol 7 has the least number of orders
- From above plot we can see that most of the total items count is between 0 and 15
- At any given time there are around 30-80 outstanding orders
- We can see that during the lunch hours 1-2pm and dinner hours 7-10pm the number of orders is high
- We can see that during the lunch hours 1-2pm and dinner hours 7-10pm the average number of orders is 25 and 45 respectively
- We can see that delivery time increases during peak hours
- We can see that there is a repeating pattern in the daily order data of a month
- We can see that orders are high in first 2 weeks and low in the last week of the month
- Most of the delivery times are under 100 minutes
- Most of the deliveries happen on Friday, Saturday and Sunday
- We can see that American, Pizza, and Mexican are the most ordered categories
- We can see that delivery time decreases as the number of total items increases
- We can see that the delivery time is less as the number of unique items increases

Recommendations

- The company should priorities higher markets that are receiving more orders like market id 2 and deploy more resources to these markets.
- Since Order Protocol 1 has the highest number of orders, the company should focus on improving the delivery time for this protocol.
- Increase the number of delivery partners during peak hours.
- And also during weekends to reduce the delivery time.
- This can be done by hiring more delivery partners or incentivizing existing ones to work during these hours.
- During the start of month, there are higher number of orders, so the company should focus on increasing the number of delivery partners during this time.
- Consider implementing targeted marketing strategies to boost orders in underperforming markets.

Questionnaire

1. Defining the problem statements and where can this and modifications of this be used?
 - The problem statement is to predict the delivery time for a given order based on various features such as order details, restaurant information, and delivery partner availability. This model can be used in various applications such as:
 - E-commerce platforms for estimating delivery times.
 - Food delivery services for providing accurate delivery time estimates to customers.
 - Logistics companies for optimizing delivery routes and times.
 - Ride-hailing services for estimating arrival times of drivers.
2. List 3 functions the pandas datetime provides with one line explanation.
 - `pd.to_datetime()` : Converts a string or number to a datetime object.
 - `pd.date_range()` : Generates a range of dates with specified frequency.
 - `pd.to_timedelta()` : Converts a string or number to a timedelta object.
3. Short note on datetime, timedelta, time span (period)
 - **Datetime**: Represents a specific point in time, including date and time.
 - **Timedelta**: Represents a duration or difference between two datetime objects.
 - **Time Span (Period)**: Represents a span of time, often defined by a start and end date.
4. Why do we need to check for outliers in our data?
 - Outliers can skew the results of statistical analyses and machine learning models, leading to inaccurate predictions and insights. Identifying and handling outliers helps improve model performance and interpretability.
5. Name 3 outlier removal methods?
 - **Z-Score Method**: Identifies outliers by measuring how many standard deviations a data point is from the mean.
 - **IQR Method**: Uses the interquartile range to determine outliers by identifying points that fall below $Q1 - 1.5/IQR$ or above $Q3 + 1.5IQR$.
 - **Isolation Forest**: An algorithm that isolates outliers instead of profiling normal data points.
6. What classical machine learning methods can we use for this problem?
 - **Linear Regression**: A method for modeling the relationship between a dependent variable and one or more independent variables.
 - **LightGBM**: A gradient boosting framework that uses tree-based learning algorithms.
7. Why is scaling required for neural networks?
 - Scaling is required for neural networks to ensure that all input features contribute equally to the model's learning process. It helps in faster convergence during training and prevents certain features from dominating the learning process due to their larger scale.
8. Briefly explain your choice of optimizer.
 - **Adam**: The Adam optimizer is chosen because it combines the advantages of two other extensions of stochastic gradient descent. It computes adaptive learning rates for each parameter and is computationally efficient, requiring little memory. Adam is well-suited for large datasets and high-dimensional spaces, making it a popular choice for training deep learning models.
9. Which activation function did you use and why?
 - **ReLU**: The Rectified Linear Unit activation function is used because it helps mitigate the vanishing gradient problem and allows models to learn faster and perform better.
 - **Elu (After parameter tuning)**: The Exponential Linear Unit activation function is used because it helps to avoid the vanishing gradient problem and has a non-zero mean, which can help with convergence.
10. Why does a neural network perform well on a large dataset?
 - Neural networks can learn complex patterns and relationships in large datasets, allowing them to generalize better and make accurate predictions. The more data available, the better the model can learn and adapt to the underlying distribution of the data.